TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH
HỆ ĐÀO TẠO CHẤT LƯỢNG CAO

THIẾT KẾ VÀ XÁC MINH HIGH-SPEED UART VỚI GIAO THỨC APB

SVTH: **NGUYỄN LỤC ĐỨC**
MSSV: 21119331
HUỲNH TẤN TÀI
MSSV: 21119354

GVHD: **TS ĐỖ DUY TÂN**

TP. HỒ CHÍ MINH – 06/2025

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

THIẾT KẾ VÀ XÁC MINH HIGH-SPEED UART VỚI GIAO THỨC APB

SVTH: **NGUYỄN LỤC ĐỨC**
MSSV: 21119331
HUỲNH TẤN TÀI
MSSV: 21119354

GVHD: **TS ĐỖ DUY TÂN**

TP. HỒ CHÍ MINH – 06/2025

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: THIẾT KẾ VÀ XÁC MINH HIGH-SPEED UART VỚI GIAO THỨC APB.

Sinh viên thực hiện:

Nguyễn Lục Đức	MSSV: 21119331
Huỳnh Tấn Tài	MSSV: 21119354

Giảng viên hướng dẫn: TS. Đỗ Duy Tân

Nhận xét bao gồm các nội dung sau:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đưa ra vấn đề, mục tiêu và yêu cầu cụ thể.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết, phân tích và đánh giá phù hợp.

3. Kết luận thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Kết luận thực hiện đáp ứng đủ các yêu cầu, mục tiêu phù hợp với yêu cầu của đề tài: phân tích thiết kế hoạt động và tiến hành kiểm thử chức năng tồn tại.

4. Kết luận và đề xuất:

Kết luận phù hợp với yêu cầu được đặt ra, phù hợp với hoạt động thiết kế thực tế.

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng yêu cầu định dạng.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện kỹ năng giao tiếp và kỹ năng làm việc nhóm trong việc thực hiện đề tài.

7. Tài liệu trích dẫn:

Tài liệu tham khảo được trích dẫn đúng theo chuẩn IEEE.

8. Đánh giá về sự trùng lặp của đề tài:

Đề tài có 21% trùng lặp và được kiểm tra bằng Turnitin.

9. Những ưu nhược điểm và thiết sót, những điểm cần được bổ sung và chỉnh sửa:

- Nội dung lý thuyết cần chuyên sâu hơn về phân tích kỹ thuật.
- Cần cập nhật danh sách viết tắt theo thứ tự Alphabet.
- Tất cả các hình ảnh và bảng cần được tham chiếu đến các đoạn văn liên quan.
- Cần bổ sung và cập nhật danh mục tài liệu tham khảo.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên:

Có tinh thần trách nhiệm cao, chủ động trong quá trình thực hiện.

Đề nghị của giảng viên hướng dẫn:

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư và được phép bảo vệ khóa luận tốt nghiệp.

Tp. Hồ Chí Minh, ngày 09 tháng 06 năm 2025

Người nhận xét

(Ký và ghi rõ họ tên)

ĐẠI HỌC SƯ PHẠM KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự Do – Hạnh phúc

Tp. HCM, ngày 07 tháng 07 năm 2025

BẢN GIẢI TRÌNH CHỈNH SỬA ĐỒ ÁN TỐT NGHIỆP

NGÀNH: CNKT MÁY TÍNH (HỆ ĐT CLC)

1. Tên đề tài: THIẾT KẾ VÀ XÁC MINH HIGH-SPEED UART VỚI GIAO THỨC APB.
2. Tên sinh viên: Nguyễn Lục Đức MSSV: 21119331
   Tên sinh viên: Huỳnh Tấn Tài MSSV: 21119354
3. GVHD: Ts. Đỗ Duy Tân
4. Hội đồng bảo vệ HĐ 1, phòng A2-401, ngày 26 tháng 06 năm 2025
5. Giải trình chỉnh sửa báo cáo đồ án tốt nghiệp:

TT	Nội dung góp ý của Hội đồng	Kết quả chỉnh sửa, bổ sung	Ghi chú
1	Cần kiểm tra lại cách trình bày: định dạng, biểu thức toán	Đã kiểm tra lại cách trình bày, căn dòng, đánh dấu các công thức toán.	
2	Cần bổ sung giải thích cách xây dựng các testcase	Đã bổ sung giải thích cách xây dựng testcase vào chương 4.	
3	Cần vẽ lại các sơ đồ khối	Đã chỉnh sửa các sơ đồ khối, làm rõ chữ, thêm chi tiết tín hiệu.	

Xác nhận của trưởng ngành
(Ký họ và tên)

Xác nhận của GVHD
(Ký họ và tên)

Nhóm thực hiện báo cáo
(Ký họ và tên)

LỜI CẢM ƠN

Trước tiên, nhóm thực hiện xin gửi lời tri ân sâu sắc đến Tiến sĩ Đỗ Duy Tân – người đã tận tình hướng dẫn, định hướng và đồng hành cùng nhóm trong suốt quá trình thực hiện đồ án tốt nghiệp với đề tài "Thiết kế và xác minh High-Speed UART với giao thức APB". Sự tận tâm, kiến thức chuyên môn sâu rộng và những đóng góp quý báu từ Thầy là nguồn động lực to lớn, giúp nhóm vượt qua nhiều thử thách và hoàn thành đề tài một cách hiệu quả.

Nhóm cũng xin chân thành cảm ơn quý Thầy Cô trong Khoa Kỹ thuật máy tính – Trường Đại học Sư phạm Kỹ thuật TP. Hồ Chí Minh, những người đã truyền đạt cho nhóm những kiến thức nền tảng quý giá trong suốt quá trình học tập. Đó chính là hành trang quan trọng giúp nhóm có thể thực hiện đồ án một cách độc lập và có hệ thống.

Bên cạnh đó, nhóm xin được bày tỏ lòng biết ơn đến gia đình và bạn bè, những người luôn ở bên động viên, khích lệ và tạo điều kiện thuận lợi cả về tinh thần lẫn vật chất trong suốt chặng đường học tập cũng như quá trình hoàn thiện đề tài.

Dù đã cố gắng thực hiện đề tài một cách nghiêm túc và chỉn chu nhất, nhóm vẫn không tránh khỏi những thiếu sót do hạn chế về kinh nghiệm và thời gian. Rất mong nhận được sự đóng góp ý kiến từ quý Thầy Cô để nhóm có thể hoàn thiện hơn trong những nghiên cứu và dự án sau này.

LỜI CAM ĐOAN

Nhóm sinh viên Nguyễn Lục Đức và Huỳnh Tấn Tài thực hiện đề tài **Thiết kế và xác minh High-Speed UART với giao thức APB** dưới dự hướng dẫn của thầy Đỗ Duy Tân xin cam đoan các nội dung như sau:

1. Sản phẩm của Đồ án tốt nghiệp là do nhóm sinh viên Nguyễn Lục Đức và Huỳnh Tấn Tài thực hiện, không mượn, thuê, mua từ người khác.
2. Quyển báo cáo Đồ án tốt nghiệp là do nhóm sinh viên Nguyễn Lục Đức và Huỳnh Tấn Tài tự viết, tỷ lệ trùng lắp là 21%, các nội dung tham khảo đã được trích dẫn đầy đủ.
3. Kết quả thực hiện trong quyển báo cáo bao gồm hình ảnh, độ chính xác của mô hình là hoàn toàn đúng với mô hình, phần cứng nhóm đã thực hiện.

Nhóm sinh viên cam đoan các nội dung trên là hoàn toàn chính xác và chịu trách nhiệm hoàn toàn với những cam đoan trên.

Sinh viên thực hiện đồ án tốt nghiệp

(ký và ghi rõ họ tên)

TÓM TẮT

Giao tiếp nối tiếp bất đồng bộ UART (Universal Asynchronous Receiver and Transmitter) là một giao thức truyền thông đơn giản, được sử dụng phổ biến trong các hệ thống nhúng và thiết bị điện tử. Đề tài "**Thiết kế và xác minh High-Speed UART với giao thức APB**" tập trung vào vấn đề thiết và xác minh mô hình truyền thông UART tốc độ cao đồng bộ với giao thức ngoại vi APB (Advanced Peripheral Bus). Đề tài được thực hiện bằng ngôn ngữ mô tả phần cứng Verilog, bao gồm các khối chức năng chính như APB Slave, bộ truyền Transmitter, bộ nhận Receiver, bộ tạo xung Baud Rate Generator và các thanh ghi chức năng. Quá trình xác minh kiểm thử chức năng được thực hiện bằng testbench mô phỏng được thiết lập thủ công theo từng chức năng, thành phần đảm bảo độ chính xác và ổn định của hệ thống trong nhiều tình huống hoạt động khác nhau.

Các module được thiết kế với khả năng cấu hình linh hoạt về tốc độ truyền, hỗ trợ tùy chỉnh với nhiều mức baud rate thông qua bộ tạo xung bên trong, đảm bảo quá trình truyền không bị tác động bởi các tín hiệu xung hệ thống gây nhiễu. Giao tiếp với hệ thống điều khiển được thực hiện thông qua chuẩn giao tiếp APB, hỗ trợ tích hợp vào các hệ thống SoC một cách hiệu quả và dễ dàng. Toàn bộ thiết kế được mô tả bằng Verilog và kiểm thử theo kế hoạch đề ra.

Sản phẩm cuối cùng của đề tài là mô hình UART tốc độ kết hợp chuẩn giao tiếp APB, hoạt động chính xác trong cả hai chiều truyền và nhận dữ liệu. Hệ thống được đánh giá bằng các mô phỏng chức năng, kiểm tra lỗi và kiểm thử giới hạn. Là nền tảng có thể mở rộng và ứng dụng vào các hệ thống nhúng, hệ vi xử lý khác và tài liệu về lĩnh vực thiết kế vi mạch.

MỤC LỤC

LỜI CẢM ƠN....1
LỜI CAM ĐOAN....5
TÓM TẮT....6
MỤC LỤC....7
DANH MỤC HÌNH....10
DANH MỤC BẢNG....12
CÁC TỪ VIẾT TẮT....13
CHƯƠNG 1: GIỚI THIỆU....14
1.1 Giới thiệu....14
1.2 Mục tiêu đề tài....15
1.3 Giới hạn đề tài....16
1.4 Phương pháp nghiên cứu....16
1.5 Đối tượng và phạm vi nghiên cứu....16
1.6 Bố cục đồ án....16
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT....17
2.1 FPGA và Verilog cho FPGA....17
2.1.1 FPGA....17
2.1.2 Verilog cho FPGA....18
2.2 APB....19
2.2.1 Tổng quan....19
2.2.2 Chế độ hoạt động....20
2.2.3 Quá trình ghi có thời gian chờ....22
2.3 High-speed UART....25
2.3.1 Tổng quan....25

2.3.2 Đặc trưng khung truyền 27
2.3.3 Tốc độ Baud 29
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG 31
3.1 Yêu cầu hệ thống 31
3.1.1 Yêu cầu chức năng 31
3.1.2 Yêu cầu kỹ thuật và kiến trúc 31
3.1.3 Yêu cầu hiệu năng 32
3.1.4 Yêu cầu mô phỏng và kiểm thử 32
3.2 Sơ đồ khối hệ thống 33
3.3 Thiết kế khối chức năng 35
3.3.1 APB Slave 36
3.3.2 Register 37
3.3.3 FIFO 43
3.3.4 Baud rate 44
3.3.5 Transmitter 46
3.3.6 Receiver 49
CHƯƠNG 4: KẾT QUẢ MÔ PHỎNG 52
4.1 Module mô phỏng 52
4.2 APB Master 54
4.3 Testcase 56
4.3.1 APB Interface 56
4.3.2 Baud Rate Control 59
4.3.3 Register 61
4.3.4 Interrupt 64
4.3.5 Transmitter Data Check 66

4.3.6 Receiver Data Check 73
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 82
5.1 Kết luận 82
5.2 Hướng phát triển 82
PHỤ LỤC 84
TÀI LIỆU THAM KHẢO 85

DANH MỤC HÌNH

Hình 2.1: Cấu trúc kết nối các giao thức [5] 19
Hình 2.2: FSM APB Protocol [3] 20
Hình 2.3: Write transfer with wait state [3] 23
Hình 2.4: Giao tiếp UART [4] 25
Hình 3.1: Sơ đồ khối tổng quát hệ thống 33
Hình 3.2: Sơ đồ logic tổ hợp của khối APB Slave 36
Hình 3.3: Sơ đồ logic của khối FIFO 43
Hình 3.4: Sơ đồ logic của khối Baud_rate_generator 45
Hình 3.5: FSM Transmitter 46
Hình 3.6: Lưu đồ FSM Transmitter 47
Hình 3.7: FSM Receiver 49
Hình 3.8: Lưu đồ FSM Receiver 50
Hình 4.1 Waveform của testbench 55
Hình 4.2: Kết quả mô phỏng Log của Testbench 55
Hình 4.3: Waveform của APB Slave 56
Hình 4.4: Kết quả log của APB 57
Hình 4.5: Kết quả ghi dữ liệu vào của APB 58
Hình 4.6: Kết quả ghi dữ liệu vào của APB 58
Hình 4.7: Waveform của Baud rate Control 1 59
Hình 4.9: Log Baud Rate Control 60
Hình 4.10: Waveform của Thanh ghi 61
Hình 4.11: Kết quả log của thanh ghi 62
Hình 4.12: Waveform của Interrupt 1 65
Hình 4.13: Waveform của Interrupt 2 65
Hình 4.14: Log của interrupt 65

Hình 4.15: Kết quả truyền dữ liệu không có Parity.....67
Hình 4.16: Kết quả Log truyền dữ liệu không có Parity 67
Hình 4.17: Truyền dữ liệu với Parity lẻ.....68
Hình 4.18: Kết quả log truyền dữ liệu với Parity lẻ 69
Hình 4.19: Truyền dữ liệu với Parity chẵn.....70
Hình 4.20: Kết quả log truyền dữ liệu với Parity chẵn 71
Hình 4.21 Truyền dữ liệu với FIFO đầy.....72
Hình 4.22: Nhận dữ liệu xen kẽ không có Parity 74
Hình 4.23: Nhận dữ liệu không có Parity.....74
Hình 4.24: Kết quả log của nhận dữ liệu xen kẽ không có Parity.....74
Hình 4.25: Kiểm tra ghi đầy dữ liệu vào với không Parity 75
Hình 4.26: Kết quả log của ghi đầy dữ liệu không có Parity 76
Hình 4.27: Nhận dữ liệu xen kẽ với Parity Lẻ.....77
Hình 4.28: Nhận dữ liệu Parity lẻ.....77
Hình 4.29: Kết quả log của nhận dữ liệu xen kẽ Parity lẻ.....77
Hình 4.30: Kiểm tra ghi đầy dữ liệu vào ở chế độ Parity lẻ.....78
Hình 4.31: Kết quả log của nhận đầy dữ liệu Parity lẻ.....78
Hình 4.32: Nhận dữ liệu xen kẽ với Parity Chẵn 79
Hình 4.33: Nhận dữ liệu với Parity Chẵn.....79
Hình 4.34: Kết quả log của nhận dữ liệu xen kẽ Parity chẵn 79
Hình 4.35: Kiểm tra ghi đầy dữ liệu vào ở chế độ Parity Chẵn 81
Hình 4.36: Kết quả log của nhận đầy dữ liệu Parity chẵn.....81

DANH MỤC BẢNG

Bảng 1: Ba chuẩn giao tiếp của AMBA [6] 14

Bảng 2.1: Sự khác biệt giữa quá trình có và không có thời gian chờ.....21

Bảng 2.2: Khung truyền dữ liệu UART [4].....27

Bảng 3.1: Bảng mô tả tín hiệu 35

Bảng 3.2: Bảng mô tả thanh ghi 38

Bảng 3.3*:* Bảng mô tả thanh ghi trạng thái.....39

Bảng 3.4: Bảng mô tả thanh ghi điều khiển 39

Bảng 3.5: Bảng thanh ghi cho phép ngắt.....40

Bảng 3.6: Bảng thanh ghi trạng thái ngắt 40

Bảng 4.1: Bảng tóm tắt testcase 52

Bảng 4.2: Bảng giá trị reset thanh ghi 62

Bảng 4.3*:* Bảng giá trị thanh ghi truyền (DATA_RX).....63

Bảng 4.4: Bảng giá trị tốc độ Baud 63

Bảng 4.5: Bảng giá trị thanh ghi IER 64

CÁC TỪ VIẾT TẮT

AHB	Advanced High-Performance Bus
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
ASIC	Application-Specific Integrated Circuit
AXI	Advanced Extensible Interface
CTS	Clear To Send
DMA	Direct Memory Access
DSR	Data Set Ready
DSP	Digital Signal Processing
DTR	Data Terminal Ready
FIFO	Fist In Firt Out
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
HDL	Hardware Description Language
LUTs	Look-Up Table
PIO	Programmed I/O
PLC	Programmable Logic Controller
RAM	Random Access Memory
RTS	Request To Send
SoC	System On Chip
UART	Universal Asynchronous Receiver/ Transmitter

CHƯƠNG 1: GIỚI THIỆU

1.1 Giới thiệu

Trong lĩnh vực thiết kế vi mạch và hệ thống nhúng hiện đại, việc lựa chọn giao thức truyền thông phù hợp đóng vai trò quan trọng trong việc tối ưu hiệu suất, chi phí và mức tiêu thụ năng lượng. Để hỗ trợ việc tích hợp các khối chức năng trong một hệ thống trên chip (SoC – System on Chip), ARM đã phát triển kiến trúc AMBA (Advanced Microcontroller Bus Architecture) với ba chuẩn giao tiếp chính là APB (Advanced Peripheral Bus), AHB (Advanced High-Performance Bus), và AXI (Advanced Extensible Interface) – mỗi loại phù hợp với những ứng dụng và yêu cầu khác nhau. Bảng 1 là sự so sánh giữa 3 chuẩn giao tiếp:

Bảng 1: Ba chuẩn giao tiếp của AMBA [6]

Tiêu chí	APB	AHB	AXI
Độ phức tạp	Đơn giản	Trung bình	Phức tạp
Duplex	Half-duplex	Half-duplex	Full-duplex
Thông lượng tối đa	0.5/clock (read/write)	1/clock (read/write)	1/clock (đồng thời read & write)
Tần số clock	Thấp	Trung bình	Cao
Gửi yêu cầu tiếp theo	Sau khi nhận xong	Trong khi nhận	Không giới hạn
Pipeline	Không có	2 bước	Nhiều bước
Thiết kế cơ bản	Mạch tổ hợp	Thanh ghi	FIFO

Theo Bảng 1 tùy vào độ phức tạp và yêu cầu thông lượng, mỗi giao thức trên đều có ứng dụng đặc trưng riêng. APB thường được sử dụng để kết nối các thiết bị ngoại vi đơn giản như GPIO, UART, SPI, I2C – nơi mà độ trễ thấp và chi phí phần cứng tối thiểu được ưu tiên. Ngược lại, AHB và AXI được dùng trong các module hiệu năng cao như DMA, bộ xử lý, hoặc các hệ thống truyền dữ liệu lớn như bộ điều khiển đồ họa hoặc Ethernet.

UART (Universal Asynchronous Receiver/Transmitter) là một giao thức nối tiếp phổ biến, đóng vai trò trung gian truyền thông giữa vi điều khiển và các thiết bị ngoại

vi. Việc kết hợp giao thức UART với APB mang lại sự đơn giản, hiệu quả và tính khả thi cao cho các hệ thống nhúng cần truyền dữ liệu với tốc độ vừa phải.

Do đó, đồ án lựa chọn chủ đề **"Thiết kế và xác minh bộ giao tiếp UART sử dụng chuẩn APB"** nhằm hướng đến một giải pháp nhẹ, dễ tích hợp, nhưng vẫn đảm bảo khả năng mở rộng trong các SoC thực tế.

So với các thiết kế UART cơ bản sử dụng giao tiếp APB hiện có, đề tài có những đóng góp nổi bật sau:

- **Thiết kế cấu trúc UART tốc độ cao (High-Speed UART)** có khả năng cấu hình tốc độ truyền linh hoạt, đáp ứng đa dạng ứng dụng thực tế.
- **Tích hợp bộ đệm FIFO hai chiều**, hỗ trợ truy xuất dữ liệu hiệu quả hơn và giảm thiểu nguy cơ mất dữ liệu trong quá trình truyền.
- **Xây dựng testbench mô phỏng chi tiết APB Master**, mô phỏng hoạt động truyền/nhận một cách hệ thống, phục vụ cho kiểm thử toàn diện.
- **Tập trung vào xác minh chức năng thông qua kịch bản thực tế và đánh giá hiệu năng truyền dữ liệu**, từ đó rút ra ưu điểm về độ ổn định và khả năng xử lý dữ liệu của hệ thống.

Bên cạnh đó, đề tài còn trình bày tổng quan về các kiến trúc và quy trình thiết kế vi mạch số, giúp cung cấp nền tảng lý thuyết cho việc phát triển và kiểm thử các khối chức năng của hệ thống.

1.2 Mục tiêu đề tài

Thiết kế, mô phỏng hoạt động bộ truyền nhận UART với giao thức APB; đồng thời, lên kế hoạch xác minh, kiểm thử các chức năng:

- Truyền/ nhận dữ liệu 8 bit với bộ chuyển đổi song song - nối tiếp.
- Quản lý hoạt động hệ thống với thanh ghi chính - chức năng, và bộ đệm FIFO - dữ liệu.
- Tùy chỉnh tốc độ truyền (Baud rate) nhằm tương thích với nhiều ứng dụng khác nhau.
- Khởi tạo bộ điều khiển chính, tương đương vai trò khối kiểm tra - testbench, APB Master.

1.3 Giới hạn đề tài

- Tập trung mô phỏng cấu trúc bộ truyền nhận dữ liệu UART với giao thức APB sử dụng Verilog.
- Thực hiện mô phỏng phần mềm, không triển khai trên phần cứng FPGA hoặc ASIC.
- Hỗ trợ chế độ truyền 8 bit, chưa xét đến các chế độ truyền khác như 9 bit hoặc sử dụng các mức độ bảo vệ khác chế độ cơ bản.

1.4 Phương pháp nghiên cứu

- Nghiên cứu lý thuyết: Tìm hiểu các tiêu chuẩn AMBA APB, giao thức UART, cũng như các phương pháp thiết kế và xác minh vi mạch số.
- Thiết kế phần cứng: Sử dụng Verilog để mô tả và hiện thực hóa hệ thống truyền nhận UART tương thích với APB.
- Mô phỏng và kiểm thử: Sử dụng các công cụ mô phỏng QuestaSim để kiểm tra hoạt động của thiết kế.
- Phân tích và đánh giá: Đánh giá hiệu suất của thiết kế dựa trên nhiều tiêu chí về độ ổn định, tốc độ truyền và khả năng xử lý dữ liệu.

1.5 Đối tượng và phạm vi nghiên cứu

- Đối tượng nghiên cứu: Bộ truyền nhận UART và giao thức AMBA APB.
- Phạm vi nghiên cứu:
    + Thiết kế và mô phỏng bộ UART hoạt động trên APB Bus.
    + Xây dựng bộ kiểm thử (testbench - APB Master) để xác minh hoạt động của thiết kế.
    + Đánh giá hiệu năng của hệ thống trên QuestaSimvới nhiều tiêu chí khác nhau dựa trên cấu trúc.

1.6 Bố cục đồ án

Chương 1: Giới thiệu.

Chương 2: Cơ sở lý thuyết.

Chương 3: Thiết kế hệ thống.

Chương 4: Kết quả mô phỏng.

Chương 5: Kết luận và hướng phát triển.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 FPGA và Verilog cho FPGA

2.1.1 FPGA

a. Khái niệm

FPGA (Field-Programmable Gate Array) là một loại vi mạch số có thể được lập trình lại sau khi sản xuất, mang đến khả năng tùy biến cao trong thiết kế phần cứng. Nhờ tính linh hoạt này, FPGA được ứng dụng rộng rãi trong nhiều lĩnh vực như viễn thông, công nghiệp ô tô, hàng không vũ trụ và thiết bị điện tử tiêu dùng [1].

Khác với vi điều khiển hoặc các mạch tích hợp chuyên dụng (ASIC) vốn có cấu trúc phần cứng và tập lệnh cố định, FPGA cho phép người dùng tùy chỉnh cấu trúc logic bên trong để phù hợp với những chức năng cụ thể của từng ứng dụng. Các khối logic có thể được cấu hình để thực hiện các phép toán logic như AND, OR, XOR… và có thể kết nối với nhau linh hoạt, từ đó hình thành nên những hệ thống số phức tạp theo nhu cầu thiết kế [1].

Một trong những điểm mạnh nổi bật của FPGA là khả năng tái lập trình nhiều lần. Không giống như ASIC chỉ phục vụ một chức năng duy nhất và không thể thay đổi sau khi sản xuất, FPGA có thể được sử dụng lại cho nhiều thiết kế khác nhau, giúp đẩy nhanh quá trình thử nghiệm và phát triển sản phẩm. Tuy nhiên, chính tính linh hoạt này cũng khiến chi phí triển khai FPGA cao hơn so với các giải pháp chuyên biệt [1].

b. Cấu trúc thành phần

FPGA bao gồm các khối chức năng chính sau:

- Look-Up Table (LUTs): thực hiện các phép tính logic.
- Flip-Flop: lưu trữ dữ liệu tạm thời.
- Khối I/O: giao tiếp giữa các thiết bị và ngoại vi.
- Routing Matrix: kết nối các thành phần bên trong FPGA.

c. Ứng dụng

Các lĩnh vực phổ biến ứng dụng rộng rãi:

- Viễn thông: máy thu - phát sóng, module…
- Xử lý tín hiệu số: DSP, image processing…
- Điện tử hàng không, quân sự.
- Tính toán tự động trong các trung tâm dữ liệu.

- Học sâu - Deep Learning Accelerator.

2.1.2 Verilog cho FPGA

a. Giới thiệu

Verilog là một trong những ngôn ngữ mô tả phần cứng (HDL – Hardware Description Language) phổ biến, được dùng để biểu diễn các hệ thống số như bộ xử lý, bộ nhớ, bộ chuyển mạch hay các phần tử như flip-flop. Thông qua HDL, người thiết kế có thể mô tả phần cứng kỹ thuật số ở nhiều cấp độ trừu tượng khác nhau, từ cấp cổng logic đến hệ thống hoàn chỉnh [1].

Một trong những ưu điểm của việc sử dụng Verilog là khả năng thiết kế và mô phỏng các hệ thống mà không phụ thuộc vào công nghệ chế tạo phần cứng cụ thể, giúp quá trình phát triển linh hoạt hơn. So với sơ đồ mạch, Verilog cho phép kiểm thử và chỉnh sửa dễ dàng hơn, đặc biệt hữu ích đối với các hệ thống có quy mô lớn.

Mặc dù có cú pháp tương tự một số ngôn ngữ lập trình như C, Verilog không phải là ngôn ngữ lập trình mà là ngôn ngữ chuyên biệt dùng để mô tả hành vi và cấu trúc phần cứng số [1].

Trong Verilog, phần cứng hoặc một chức năng cụ thể được biểu diễn thông qua cấu trúc ngôn ngữ gọi là “module”. Khi định nghĩa một module, ta cần chỉ rõ danh sách các cổng vào/ra cùng với các thanh ghi và hệ thống bus nội bộ. Bên trong module, có thể sử dụng các phép gán song song, lời gọi tới các linh kiện khác, cũng như các khối xử lý để mô tả hành vi phần cứng. Nhiều module có thể được kết hợp lại theo mô hình phân cấp nhằm xây dựng các cấu trúc phần cứng phức tạp. Các thành phần trong thiết kế phân cấp có thể bao gồm các module, linh kiện cơ bản hoặc những thành phần được người dùng định nghĩa. Để tiến hành mô phỏng, mỗi thành phần con trong kiến trúc phân cấp này cần được tổng hợp riêng biệt [1].

b. Cấu trúc

Cấu trúc chương trình Verilog bao gồm:

- Module: Khối chức năng cơ bản.
- Ports (input, output, inout): Xác định các chân kết nối.
- Data types: wire, reg, integer, logic…
- Operations: Toán tử số học, logic, bitwise…
- Always block: Xử lý theo clock.

- Assign statement: Gán giá trị liên tục.

c. Ứng dụng

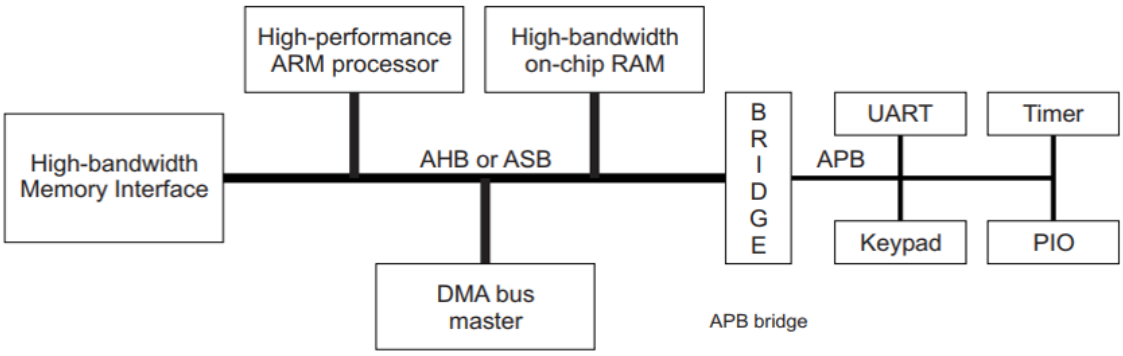
- Thiết kế và triển khai các bộ vi xử lý nhỏ (Soft-core CPU).
- Tối ưu hóa xử lý tín hiệu số (DSP).
- Mô phỏng và kiểm tra các giao thức truyền dữ liệu.
- Tích hợp và kiểm tra các giao thức như UART, SPI, I2C, APB…

2.2 APB

2.2.1 Tổng quan

APB (Advanced Peripheral Bus) là một phần của giao thức họ Advanced Microcontroller Bus Architecture (AMBA). Phiên bản mới nhất của APB là bản phát hành AMBA 4 v2.0. Đây là giao diện chi phí thấp, được tối ưu hóa để tiêu thụ điện năng tối thiểu và giảm độ phức tạp của giao diện. Không giống như AHB hay AXI, đây là giao thức Non-Pipelined, được sử dụng để kết nối các thiết bị ngoại vi băng thông thấp, đồng bộ đơn giản. Và mỗi lần truyền dữ liệu cần ít nhất hai chu kỳ xung nhịp (chu kỳ SETUP và chu kỳ ACCESS) để hoàn tất [3].

Giao diện APB được thiết kế để truy cập vào các thanh ghi điều khiển có thể lập trình của các thiết bị ngoại vi; đồng thời, APB cũng có thể giao tiếp với các giao thức khác như AHB, AXI… có tốc độ truyền khác nhau bằng cách sử dụng các cầu nối ở giữa [2]. Dưới đây là mô mình cấu trúc kết nối của các giao thức:



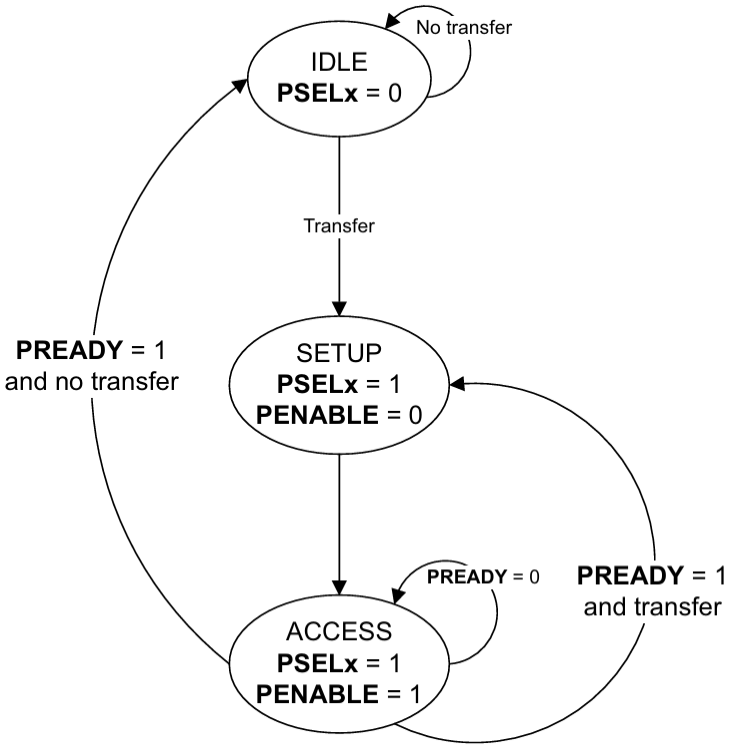
Hình 2.1: Cấu trúc kết nối các giao thức [5]

Trong hình 2.1, mô tả cấu trúc kết nối các giao thức truyền trong một hệ thống. Bộ xử lý ARM hiệu suất cao là Lõi của hệ thống và các thành phần khác như RAM trên chip băng thông cao, Bus master DMA và giao diện bộ nhớ băng thông cao được kết nối với Lõi bằng bus hệ thống, trong trường hợp này là AHB. Các thiết bị ngoại vi băng

thông thấp khác như UART, Timer, Keypad và PIO được kết nối với bus hệ thống thông qua các cầu nối bằng cách sử dụng bus ngoại vi, ở đây là APB Bus. Trong trường hợp này, cầu nối hoạt động như AHB Slave tương ứng với Core Master. Và nó cũng hoạt động như APB Master tương ứng với các thiết bị ngoại vi băng thông thấp còn lại [5].

2.2.2 Chế độ hoạt động

Dưới đây là sơ đồ máy của APB Protocol:



Hình 2.2: FSM APB Protocol [3]

Trong sơ đồ máy trạng thái của giao thức APB hình 2.2, bao gồm 3 trạng thái hoạt động chính:

- IDLE: Trạng thái mặc định của giao thức.
- SETUP: Giao dịch mới (đọc/ ghi) được khởi tạo, thiết lập các tín hiệu điều khiển và dữ liệu cần thực hiện; đồng thời đảm bảo độ ổn định cho giai đoạn truyền dữ liệu.
- ACCESS: Quá trình chính trong tiến trình giao dịch. Trong giai đoạn này, một số tín hiệu để đảm bảo tính ổn định của giao dịch, bao gồm: PADDR, PPROT, PWRITE, PWDATA, PSTRB,…

Ở trạng thái IDLE trong FSM, giao diện không có thực hiện hoạt động nào và luôn sẵn sàng tiếp nhận yêu cầu truyền dữ liệu mới từ APB Master. Trong trạng thái

này, hệ thống chỉ cần xác định giá trị tín hiệu PSELx và PENABLE ở mức THẤP mà không cần quan tâm đến giá trị của các tín hiệu khác.

Tại thời điểm nhận lệnh thực hiện giao dịch, chuyển đổi sang trạng thái SETUP - thiết lập, đây là giai đoạn đầu tiên của một quá trình thao tác, Slave vẫn chưa phản hồi, chỉ cho hệ thống biết rằng Master muốn thực hiện một giao dịch. Giai đoạn này ngoại trừ tín hiệu PENABLE còn ở mức THẤP, các tín hiệu điều khiển và dữ liệu khác cần phải được thiết lập ổn định các giá trị cần giao dịch. Trong thiết kế của cấu trúc APB, giai đoạn thiết lập SETUP chỉ tồn tại trong một chu kỳ xung hệ thống, sau một chu kỳ sẽ kích hoạt tín hiệu PENABLE [3]. Nguyên nhân giai đoạn SETUP chỉ cần tốn một chu kỳ xung hệ thống là vì: thứ nhất là tín hiệu tiếp nhận từ Master khi bắt đầu tiếp nhận giao dịch đến khi kết thúc sẽ không thay đổi, thứ hai là APB không phải là giao thức pipelined như AXI hay AHB, và cuối cùng là mọi tín hiệu điều khiển đều đồng bộ với xung clock. Vì thế, duy trì một chu kỳ xung trong trạng thái SETUP đã có thể đảm bảo độ ổn định tín hiệu cho quá trình truyền dữ liệu.

Khi Slave đã sẵn sàng dữ liệu cho yêu cầu của Master, kích hoạt tín hiệu PENABLE thực hiện giao dịch. Giao thức sẽ phân loại truyền dữ liệu thành hai loại khác nhau: có thời gian chờ (wait state) và không có thời gian chờ (no wait state). Việc lựa chọn có sử dụng thời gian chờ hay không phụ thuộc vào các thông số của ngoại vi, bao gồm: tốc độ phản hồi của Slave, độ phức tạp trong thiết kế, yêu cầu hiệu suất của hệ thống và cấu hình thiết kế của Bus. Theo lý thuyết thì thiết kế Master có thể đợi đến khi Slave sẵn sàng thực hiện; tuy nhiên, trong thực tế, cần phải xem xét hiệu suất hệ thống mà đặt một timeout để giới hạn thời gian hoạt động và đảm bảo hệ thống còn hoạt động kể cả khi xuất hiện lỗi ở một vài yêu cầu từ Master mà không gây ảnh hưởng đến các yêu cầu sau đó.

Thời điểm PREADY kích hoạt chính là thời điểm Slave đã chuẩn bị sẵn sàng các tín hiệu và dữ liệu phản hồi cho Master. Và sau một chu kỳ xung sẽ ngắt hành động đưa trạng thái của hệ thống về mặc định - IDLE; ngoài ra, nếu có yêu cầu khác từ Master thì trực tiếp chuyển từ trạng thái truy cập về thiết lập (ACCESS - SETUP) [3]. Bảng 2.1 sau là bảng phân biệt sự khác nhau giữa quá trình có và không có thời gian chờ.

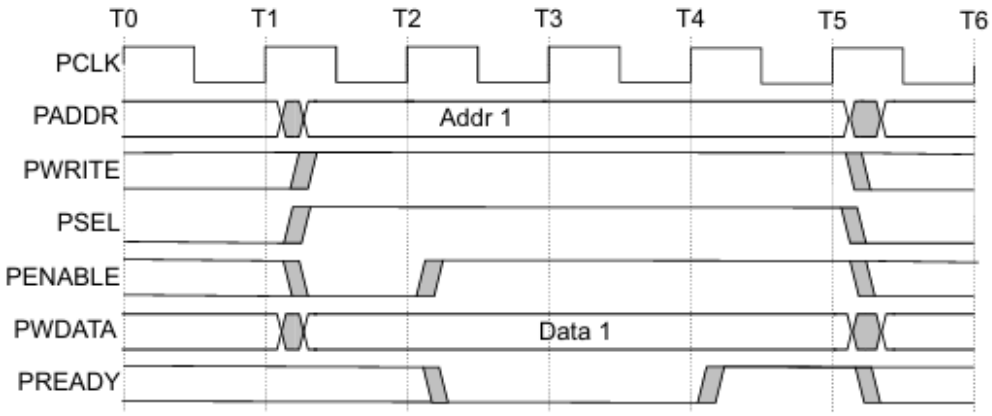
Bảng 2.1: Sự khác biệt giữa quá trình có và không có thời gian chờ

	With no wait state	With wait state
SETUP	1 chu kỳ	1 chu kỳ
ACCESS	1 chu kỳ	≥ 2 chu kỳ
PREADY	Luôn bằng 1	Có thể bằng 0 trong nhiều chu kỳ
Hiệu suất	Cao, tốc độ nhanh	Chậm vì có thời gian chờ
Ứng dụng	Các ngoại vi đơn giản, tốc độ phản hồi nhanh	Các ngoại vi phức tạp, thời gian xử lý dữ liệu và tín hiệu phức tạp.

Trong Bảng 2.1 cho thấy sự khác biệt trong thiết kế của hai quá trình là mức độ phức tạp trong thiết kế tương ứng với chức năng được ứng dụng bên trong thiết bị. Với những khối slave đơn giản như GPIO thì ta có thể không cần sử dụng thời gian chờ; tuy nhiên, những thiết kế phức tạp như bộ nhớ flash, ADC, hay UART có bộ nhớ đệm FIFO thì cần phải thiết kế sử dụng thời gian chờ bên trong bộ điều khiển. Sự khác biệt trong hai quá trình chính là quá trình xử lý dữ liệu diễn ra tại các khối slave có mức độ đơn giản hay phức tạp khác nhau, tùy thuộc vào thiết kế slave thường sẽ xuất hiện độ trễ khác nhau nên trong thiết kế sẽ chọn lựa thời gian chờ tối ưu nhất đảm bảo cho hệ thống hoạt động bình thường. Vì vậy nên trong thiết kế APB Slave thường kết hợp thêm tín hiệu PREADY có trách nhiệm xác định các dữ liệu đã sẵn sàng thực hiện giao dịch hay chưa.

2.2.3 Quá trình ghi có thời gian chờ

Dưới đây là Waveform của Quá trình ghi có thời gian chờ:



Hình 2.3: Write transfer with wait state [3]

Theo hình 2.3, quá trình ghi dữ liệu vào thiết bị bao gồm ba chu kỳ xung T1, T2 và T3 đến T5, trong đó có một trạng thái chờ do **PREADY ở mức THẤP** tại thời điểm đầu của pha truy cập.

Trong trường hợp này, tổng thời gian thực hiện quá trình ghi sẽ kéo dài hơn thời gian tối thiểu (tức hơn 2 chu kỳ xung), vì cần đợi cho đến khi **PREADY lên mức CAO** để hoàn tất truyền dữ liệu [3].

Các tín hiệu cần lưu ý:

- **PSEL**: Tín hiệu lựa chọn ngoại vi (thực tế là PSELx, với x là số hiệu ngoại vi).
- **PADDR, PWRITE, PWDATA**: Phải được thiết lập ổn định trước hoặc tại thời điểm PSEL được kích hoạt.

Chi tiết quá trình ghi:

Giai đoạn thiết lập (T1 đến T2):

- Tại T1, các tín hiệu PADDR, PWRITE, và PWDATA được thiết lập.
- PSEL được đưa lên mức CAO (1), đánh dấu việc chọn ngoại vi.
- Đây là pha chuẩn bị, chưa có truy cập thực sự diễn ra.

Giai đoạn truy cập bắt đầu (T2):

- PENABLE được đưa lên mức CAO tại T2, bắt đầu Access Phase.
- Tuy nhiên, tại thời điểm này PREADY vẫn ở mức THẤP, nghĩa là Slave chưa sẵn sàng nhận dữ liệu.

Trạng thái chờ (T3 đến T4):

- Vì PREADY = 0, Master phải giữ ổn định các tín hiệu (PADDR, PWDATA, PWRITE...) trong thời gian chờ.
- Việc truyền chưa thể hoàn tất.

Kết thúc truy cập (T4 đến T5):

- Tại cạnh lên T5, PREADY được đưa lên mức CAO từ phía Slave.
- Đây là thời điểm dữ liệu được chấp nhận, quá trình ghi hoàn tất.

Kết thúc truyền (sau T5):

- Sau cạnh lên T5, PSEL được đưa về mức THẤP, kết thúc phiên truyền ghi.
- Các tín hiệu khác có thể thay đổi, không còn bị ràng buộc.

Đối với quá trình ghi dữ liệu đã trình bày, được xem là hoạt động cơ bản của thiết kế. Ngoài sự ổn định khi thực hiện lần lượt từng quá trình riêng lẻ, thiết kế phải đảm bảo thực hiện liên tiếp nhiều tiến trình liên tiếp khi tín hiệu PSEL vẫn duy trì ở mức CAO.

Trong giai đoạn Truy cập (Access phase), khi tín hiệu PENABLE ở mức CAO, quá trình truyền sẽ được kéo dài bằng cách đưa PREADY xuống mức THẤP.

Các tín hiệu sau sẽ không thay đổi khi PREADY vẫn ở mức THẤP:

- **PADDR** – Tín hiệu địa chỉ.
- **PWRITE** – Tín hiệu hướng truyền.
- **PSELx –** Tín hiệu chọn.
- **PENABLE –** Tín hiệu cho phép.
- **PWDATA –** Tín hiệu dữ liệu ghi.
- **PSTRB –** Tín hiệu kích hoạt ghi.
- **PPROT –** Tín hiệu loại bảo vệ.
- **PAUSER –** Thuộc tính yêu cầu của người dùng.
- **PWUSER –** Thuộc tính dữ liệu ghi của người dùng.

PREADY có thể nhận bất kỳ giá trị nào khi PENABLE ở mức THẤP. Điều này đảm bảo rằng các ngoại vi có chu kỳ truy cập cố định là 2 chu kỳ có thể giữ PREADY ở mức CAO [3].

Tín hiệu PSTRB cho phép truyền dữ liệu trên bus dữ liệu ghi. Mỗi giá trị PSTRB tương ứng với 1 byte trên bus dữ liệu ghi. Khi được khẳng định ở mức CAO, PSTRB

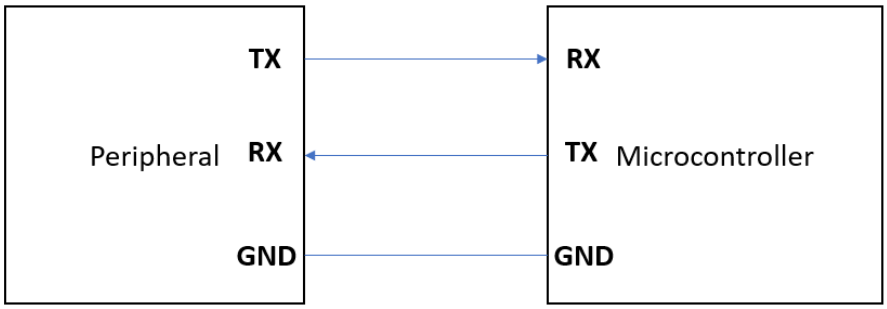
chỉ ra rằng byte tương ứng trên bus dữ liệu ghi chứa thông tin hợp lệ (các giá trị dữ liệu tương ứng với bit PSTRB = 0 sẽ mang giá trị mặc định).

Có một tín hiệu chốt ghi (write strobe) cho mỗi 8 bit của bus dữ liệu ghi, vì vậy PSTRB[n] tương ứng với PWDATA[(8n+7):(8n)] [3].

2.3 High-speed UART

2.3.1 Tổng quan

Dưới đây là sơ đồ khối giao tiếp UART:



Hình 2.4: Giao tiếp UART [4]

Trong các hệ thống nhúng và truyền thông nối tiếp hình 2.4, UART đóng vai trò quan trọng trong việc truyền và nhận dữ liệu giữa các thiết bị. Là giao thức truyền thông nối tiếp và không đồng bộ, đơn giản hóa trong quá trình kết nối giữa các vi điều khiển, máy tính và các thiết bị ngoại vi. Với thiết kế đơn giản, dễ triển khai thì UART trở thành lựa chọn phổ biến trong nhiều ứng dụng công nghệ khác nhau [4].

Giao thức bao gồm hai đường truyền dữ liệu độc lập là TX (truyền) và RX (nhận). Dữ liệu được truyền nhận thông qua hai đường truyền này dưới dạng một khung dữ liệu (data frame) được thiết kế theo tiêu chuẩn: start bit, data, parity bit và stop bit [4].

Trong thiết kế UART bao gồm nhiều thành phần quan trọng đảm việc truyền/ nhận diễn ra chính xác. Đầu tiên là bộ tạo xung nhịp (Baud rate Generator) giúp đảm bảo tốc độ truyền dữ liệu ổn định hơn dựa theo cấu hình mong muốn. Bộ phát (Transmitter) chịu trách nhiệm như một bộ chuyển đổi dữ liệu từ dạng song song sang nối tiếp; ngược lại, bộ nhận (Receiver) chuyển đổi các dữ liệu nối tiếp nhận vào thành dữ liệu nối tiếp, kiểm tra mức độ chính xác. Ngoài ra, UART còn có các thanh ghi đệm hỗ trợ lưu trữ các dữ liệu truyền đi/ nhận vào chuẩn bị cho các quá trình chuyển đổi, xử lý dữ liệu và kiểm soát các luồng dữ liệu trong hệ thống. Giao thức truyền này sử dụng các tín hiệu như CTS, DSR,RTS, DTR [4].

Quá trình truyền dữ liệu của UART hoạt động theo trình tự như sau: Đầu tiên, bộ phát sẽ tiếp nhận dữ liệu song song từ bộ xử lý và chuyển vào thanh ghi truyền. Sau đó, UART khởi tạo Start bit (mức THẤP) vào đầu chuỗi dữ liệu thông báo cho ngoại vi tiến hành quá trình truyền dữ liệu, và các bit dữ liệu sẽ lần lượt được gửi đi theo thứ tự từ bit có trọng số thấp nhất (LSB) đến bit có trọng số cao nhất (MSB). Khi toàn bộ dữ liệu được gửi đi, tùy theo yêu cầu của hệ thống mà sẽ khởi tạo bit Parity tương ứng với chế độ được lựa chọn với giá trị dữ liệu truyền đi. Cuối cùng, UART thêm một hoặc hai Stop bit (mức CAO) vào cuối chuỗi dữ liệu truyền đi để đánh dấu kết thúc quá trình truyền. Còn đối với quá trình nhận các dữ liệu, bộ nhận dữ liệu sẽ đọc các bit từ dòng tín hiệu, kiểm tra Parity bit (nếu có) để phát hiện lỗi, loại bỏ các bit điều khiển và chuyển đổi thành dữ liệu song song để xử lý [4].

Giao tiếp UART được sử dụng rộng rãi trong nhiều lĩnh vực nhờ tính linh hoạt và đơn giản khi triển khai trong thiết kế. Những ứng dụng phổ biến của nó là kết nối vi điều khiển với máy tính hoặc ứng dụng trong các module truyền thông không dây giữa các thiết bị [4]. Trong lĩnh vực công nghệ hiện nay, UART được áp dụng trong hệ thống nhúng và cảm biến, hỗ trợ các thiết bị đo lường với vi điều khiển hoặc bộ xử lý trung tâm. Một trong những ứng quan trọng cần chú ý chính là hệ thống điều khiển tự động, đặc biệt là trong các bộ điều khiển PLC và các thiết bị tự động hóa công nghiệp - nơi cần phương thức truyền dữ liệu đơn giản và đáng tin cậy.

Ưu điểm:

- Giao tiếp đơn giản: Chỉ cần hai đường truyền (TX, RX) giữa các thiết bị ngoại vi thực hiện chức năng truyền nhận dữ liệu.
- Không phụ thuộc tín hiệu xung hệ thống như các giao thức khác như SPI, I2C. Khiến cho cấu trúc đơn giản, giảm mức độ phức tạp cho hệ thống. Đồng thời giúp hệ thống hạn chế nhiễu tốt.
- Tài nguyên tiêu thụ ít (chân GPIO).
- Dễ dàng cấu hình tốc độ truyền (Baud rate).
- Đảm bảo độ uy tín dữ liệu: Khởi tạo Parity bit trong khung truyền dữ liệu đánh giá độ uy tín khi đã truyền đi.

Nhược điểm:

- Quy mô hệ thống nhỏ: chỉ truyền nhận 1-1 giữa hai thiết bị, hạn chế trong việc sử dụng khi thi công thiết kế mạng lưới dữ liệu một hệ thống nhiều thiết bị.
- Yêu cầu sự đồng bộ cấu hình: hai thiết bị giao tiếp phải thống nhất thông số truyền (tốc độ baud, cấu trúc gói dữ liệu, phương thức kiểm tra).
- Tốc độ truyền thấp hơn so với các giao thức khác: sử dụng truyền nối tiếp.

2.3.2 Đặc trưng khung truyền

Dưới đây là khung truyền (frame) dữ liệu UART:

Bảng 2.2: Khung truyền dữ liệu UART [4]

Start bit	Data	Parity bit	Stop bit
0	(5 to 8 bits)	(0 to 1 bit)	1

Trong giao tiếp của giao thức UART, dữ liệu sẽ được truyền theo từng khung (frame theo tiêu chuẩn chứa các bit điều khiển (start, parity và stop bit) và dữ liệu. Một khung truyền tiêu chuẩn như bảng 2.2) [4]. Cấu trúc cơ bản bao gồm:

Bit start (1 bit):

Trong UART, đường truyền khi ở trạng thái nhàn rỗi luôn ở logic mức CAO - khi dữ liệu không được gửi, đường truyền luôn duy trì ở mức "1". Nguyên nhân chính cho việc sử dụng mức logic CAO khi nhàn rỗi là để tránh nhiễu cho hệ thống, đảm bảo dữ liệu không xảy sai sót khi đến thiết bị ngoại vi đầu cuối. Mức "1" thường được duy trì bằng điện trở kéo lên (pull-up resistor), giúp ổn định tín hiệu và giảm thiểu xảy ra trường hợp nhiễu tín hiệu; đồng thời, nếu trạng thái nhàn rỗi được đặt mức "0" thì khi xuất hiện dao động nhỏ nào của tín hiệu cũng có thể làm cho hệ thống nhận nhầm dữ liệu. Ngoài ra, mức "1" tiêu thụ năng lượng ít hơn và thích hợp với nhiều chuẩn truyền thông khác (RS-232, TTL, CMOS) [4].

Khi có lệnh truyền dữ liệu từ phía Master, đường truyền tín hiệu sẽ được kéo xuống mức thấp trong một chu kỳ xung nhịp. Sự thay đổi điện áp từ mức cao xuống thấp này được bộ thu của UART nhận diện như một tín hiệu bắt đầu. Ngay khi phát hiện tín hiệu đó, bộ thu sẽ tiến hành đọc dữ liệu theo từng bit, với tốc độ đã được đồng bộ giữa hai thiết bị thông qua cấu hình tần số truyền.

Trường dữ liệu (5 – 8 bit):

Đây là phần chứa thông tin cần gửi. Số lượng bit có thể thay đổi từ 5 đến 8, tùy thuộc vào cấu hình của hệ thống hoặc yêu cầu cụ thể của ứng dụng, giúp tăng tính linh hoạt trong thiết kế truyền thông [4].

Parity bit:

Kiểm tra theo phương pháp xác định tính chẵn lẻ của dữ liệu (Even parity, Odd parity). Cấu hình có thể lựa chọn sử dụng bit kiểm tra vào khung truyền hoặc không. Trong phương pháp khởi tạo bit parity, để phù hợp với thiết kế của UART thì quá trình khởi tạo và kiểm tra parity dựa vào tổ hợp của nhiều cổng XOR, dữ liệu truyền đi được xem xét mức độ uy tín số lượng bit "0" và "1" trong chuỗi dữ liệu; hiểu đơn giản là nếu như trong chuỗi 8 bit dữ liệu của ký tự A (0100 0001), khi UART xác định chuỗi dữ liệu không xuất hiện lỗi thì dữ liệu đảm bảo cho người dùng là số lượng bit "1" là 2 và bit "0" là 6. Phương pháp này không thể đảm bảo dược thứ tự bit dữ liệu có chính xác hay không, mà chỉ là xét số lượng sai sót một loại bit là 1 [4]. Gồm hai cơ chế kiểm tra:

- Cơ chế bậc chẵn (Even parity): Nếu tổng số bit 1 của dữ liệu là số lẻ, bit parity được khởi tạo đi kèm chuỗi dữ liệu sẽ được thiết lập là 1. Khi dữ liệu được đọc ra, việc tính toán tổng các bit được thực hiện một lần nữa và so sánh với giá trị bit parity được nhận từ thiết bị ngoại vi đã gửi dữ liệu. Và ngược lại, khi số lượng bit "1" là 0 thì parity bit có giá trị là 0.
- Cơ chế bậc lẻ (Odd parity): Tương tự như nguyên lý hoạt động của Even parity, tuy nhiên sự khác biệt chính là giá trị parity bit chính là giá trị sẽ là nghịch đảo của cơ chế Even parity.

Tóm gọn lại phương pháp trên, trên phương diện logic, even parity là tổ hợp của cổng XOR và odd parity là kết quả ngược lại của even parity (đảo của tổ hợp XOR) [4].

Bit stop (1 bit):

Là bit giá trị thông báo kết thúc khung truyền của gói dữ liệu, UART gửi sẽ điều khiển đường truyền dữ liệu từ điện áp thấp lên mức điện áp cao. Khoảng thời gian được sử dụng từ một đến hai xung nhịp.

Sau thời gian duy trì bit stop, đường truyền mới sẵn sàng cho quá trình truyền tiếp theo.

2.3.3 Tốc độ Baud

Tốc độ baud là thông số kỹ thuật cho biết số lượng bit được truyền đi mỗi giây trong giao tiếp nối tiếp bất đồng bộ, tức là không có đồng hồ hệ thống dùng chung giữa hai thiết bị. Đơn vị đo của thông số này là bps (bits per second) [4].

Thông thường, tốc độ baud phản ánh tổng số bit trên đường truyền trong một giây, bao gồm toàn bộ các thành phần cấu thành khung dữ liệu như: bit bắt đầu (start bit), bit dữ liệu, bit kiểm tra (parity – nếu sử dụng), và bit kết thúc (stop bit). Mặc dù vậy, không phải tất cả các bit này đều là dữ liệu thực sự – phần lớn chỉ là bit điều khiển giúp đảm bảo quá trình truyền nhận chính xác và đồng bộ [4].

Ví dụ, nếu một khung dữ liệu gồm 7 bit thông tin, cộng thêm 1 bit bắt đầu và 1 bit dừng (khi không dùng parity) thì sẽ cần 9 bit để truyền một khung. Trong trường hợp có thêm 1 bit parity, khung dữ liệu sẽ bao gồm 10 bit. Với tốc độ truyền là 300 bps, sẽ có 30 khung được gửi trong một giây. Nếu mỗi khung chứa 7 bit dữ liệu, tổng số bit dữ liệu hữu ích là 30 × 7 = 210 bps, trong khi phần còn lại là các bit điều khiển phục vụ cho việc truyền nhận.

Công thức tính toán baud rate:

$$\text{Baud_rate} = \frac{F_{clk}}{16 * Baud_Divisor} \quad (1)$$

Với F_{clk} hệ thống là 50MHz.

16 là khoảng lấy mẫu

Do đó để có được tốc độ baud mong muốn ta có cần tìm Baud_divisor.

Tốc độ truyền chuẩn hiện nay gồm: 4800, 9600, 19200, 38400, 57600

Truyền với tốc độ cao gồm: 115200, 230400, 460800, 921600.

Vậy với các mốc Baud_rate, dựa trên công thức (1) ta tính được các Baud_Divisor:

+ Baud_rate 4800:

$$\text{Baud_Divisor} = \frac{F_{clk}}{16 * Baud_rate} = \frac{50\,000\,000}{16 * 4800} \approx 651$$

+ Baud_rate 9600:

$$\text{Baud_Divisor} = \frac{F_{clk}}{16 * Baud_rate} = \frac{50\,000\,000}{16 * 9600} \approx 325$$

+ Baud_rate 19200:

$$\text{Baud_Divisor} = \frac{F_{clk}}{16 * Baud_rate} = \frac{50\,000\,000}{16 * 19200} \approx 162$$

+ Baud_rate 38400:

$$\text{Baud_Divisor} = \frac{F_{clk}}{16 * Baud_rate} = \frac{50\,000\,000}{16 * 38400} \approx 81$$

+ Baud_rate 115200:

$$\text{Baud_Divisor} = \frac{F_{clk}}{16 * Baud_rate} = \frac{50\,000\,000}{16 * 115200} \approx 27$$

+ Baud_rate 230400:

$$\text{Baud_Divisor} = \frac{F_{clk}}{16 * Baud_rate} = \frac{50\,000\,000}{16 * 230400} \approx 14$$

+ Baud_rate 460800:

$$\text{Baud_Divisor} = \frac{F_{clk}}{16 * Baud_rate} = \frac{50\,000\,000}{16 * 460800} \approx 7$$

+ Baud_rate 921600:

$$\text{Baud_Divisor} = \frac{F_{clk}}{16 * Baud_rate} = \frac{50\,000\,000}{16 * 921600} \approx 3$$

+ Baud_rate 3125000:

$$\text{Baud_Divisor} = \frac{F_{clk}}{16 * Baud_rate} = \frac{50\,000\,000}{16 * 3125000} \approx 1$$

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu hệ thống

Để xây dựng và triển khai thành công một hệ thống UART sử dụng giao tiếp APB, cần xác định rõ các yêu cầu hệ thống nhằm đảm bảo tính đúng đắn, hiệu quả và khả năng mở rộng. Hệ thống cần đáp ứng các yêu cầu về mặt chức năng, kiến trúc, hiệu suất và khả năng tích hợp với các hệ thống lớn hơn như SoC. Cụ thể, các yêu cầu hệ thống được chia thành các nhóm như sau:

3.1.1 Yêu cầu chức năng

Truyền và nhận dữ liệu nối tiếp (UART):

Hệ thống phải có khả năng truyền và nhận dữ liệu dạng nối tiếp thông qua hai đường tín hiệu chính là tx (transmit) và rx (receive).

Hỗ trợ giao tiếp song song với bus APB:

Thiết bị ngoại vi UART phải giao tiếp được với CPU hoặc hệ thống điều khiển chính thông qua giao thức APB. Cụ thể, các tín hiệu điều khiển như PSEL, PENABLE, PWRITE, PADDR, PWDATA, PRDATA, PREADY phải được xử lý đầy đủ theo chuẩn AMBA APB.

Cấu hình tốc độ truyền (Baud rate):

Hệ thống phải cho phép lập trình tốc độ truyền thông qua một thanh ghi điều khiển baud rate (baud_div). Giá trị này được cấu hình bởi CPU thông qua APB.

Bộ đệm FIFO cho truyền và nhận:

Hệ thống phải có các bộ đệm FIFO riêng biệt cho cả quá trình truyền (Transmit FIFO) và nhận (Receive FIFO) để đảm bảo không mất dữ liệu do chênh lệch tốc độ xử lý giữa CPU và UART.

Báo trạng thái và ngắt:

Hệ thống cần cung cấp các tín hiệu trạng thái như FIFO full, FIFO empty, error, cùng với khả năng sinh ngắt (interrupt) khi có sự kiện cần CPU xử lý (ví dụ: dữ liệu đến, FIFO đầy...).

3.1.2 Yêu cầu kỹ thuật và kiến trúc

Tuân thủ chuẩn giao thức APB (AMBA 2.0):

Thiết kế phải tương thích với đặc tả chuẩn của giao tiếp APB, đảm bảo khả năng tích hợp với các hệ thống sử dụng AMBA bus của ARM.

Lập trình được qua địa chỉ thanh ghi:

Các chức năng như cấu hình baud rate, chế độ parity, truyền dữ liệu, đọc dữ liệu... phải được thực hiện thông qua việc truy cập vào các thanh ghi nội tại thông qua bus APB.

Độ rộng dữ liệu:

Hệ thống UART phải hỗ trợ truyền/nhận dữ liệu có độ rộng 8-bit (có thể mở rộng trong tương lai nếu cần). Bus dữ liệu APB cũng yêu cầu hỗ trợ tối thiểu 8-bit hoặc 32-bit để phù hợp chuẩn thiết kế hệ thống SoC.

Hoạt động đồng bộ với xung nhịp hệ thống:

Tất cả các khối chức năng bên trong phải hoạt động đồng bộ với xung nhịp hệ thống (i_Clock) nhằm đảm bảo độ ổn định và tránh lỗi timing.

3.1.3 Yêu cầu hiệu năng

Tốc độ truyền điều chỉnh được:

Hệ thống phải cho phép truyền dữ liệu với nhiều tốc độ khác nhau, từ thấp (9600 baud) đến cao (3125000 baud), tùy theo cấu hình thanh ghi baud_div.

Thời gian đáp ứng nhanh:

Đảm bảo PREADY được trả về đúng thời điểm (1CLK) để CPU không phải chờ đợi trong quá trình truyền/nhận hoặc cấu hình lâu.

Xử lý dữ liệu liên tục:

Nhờ vào FIFO, hệ thống phải có khả năng truyền và nhận dữ liệu liên tục mà không làm gián đoạn CPU hoặc làm trễ dữ liệu.

3.1.4 Yêu cầu mô phỏng và kiểm thử

Có khả năng mô phỏng Verilog:

Toàn bộ thiết kế phải viết bằng ngôn ngữ mô tả phần cứng Verilog và có thể chạy trên phần mềm mô phỏng QuestaSim

Viết testbench kiểm thử đầy đủ:

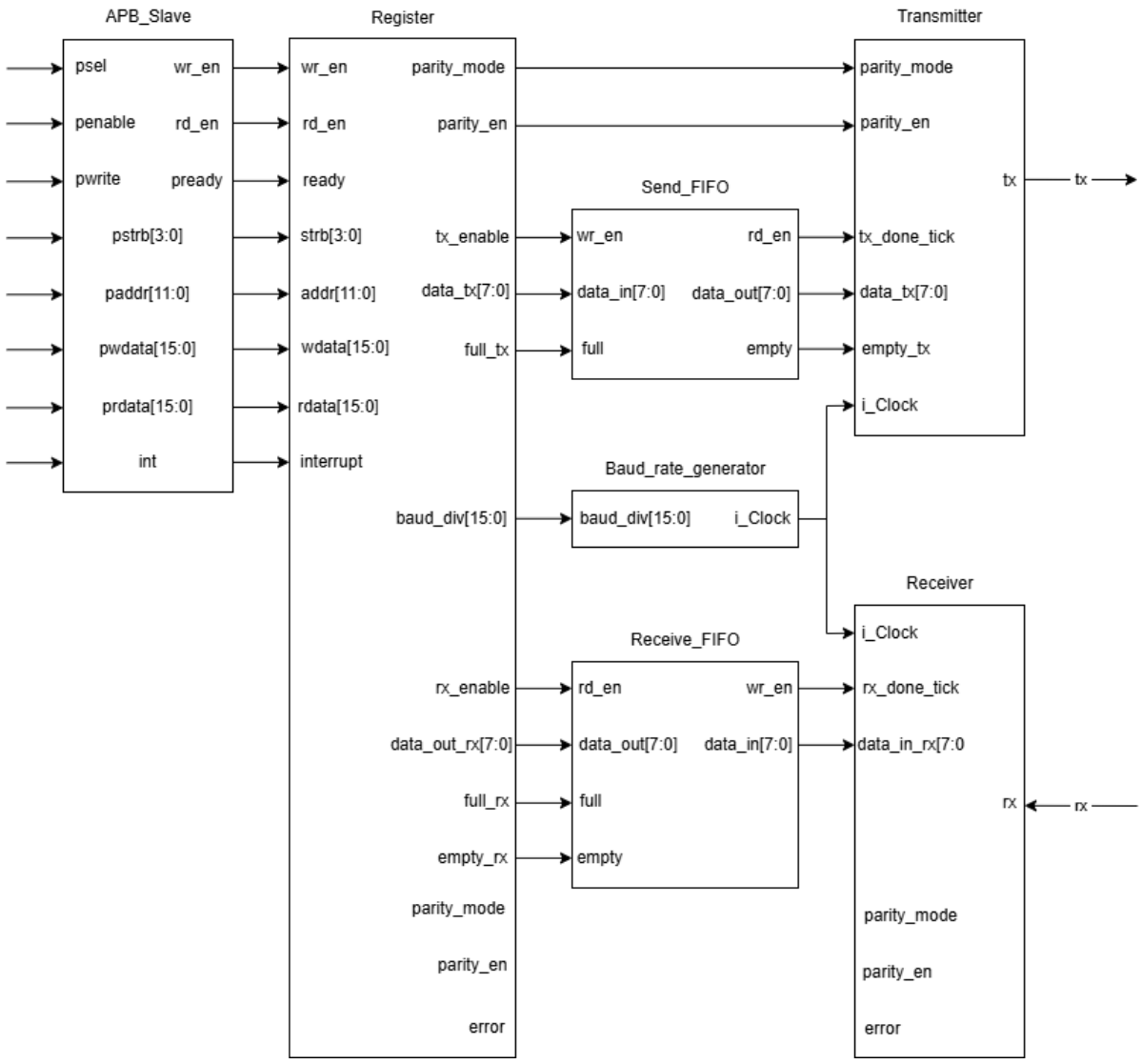
Hệ thống cần có testbench tự động để kiểm thử các tình huống như: truyền đúng dữ liệu, kiểm tra parity, FIFO đầy/rỗng, ngắt xảy ra đúng thời điểm...

Tái sử dụng và mở rộng dễ dàng:

Cấu trúc thiết kế cần được tổ chức mô-đun để dễ dàng tái sử dụng trong các dự án lớn hơn hoặc tích hợp vào hệ thống SoC phức tạp.

3.2 Sơ đồ khối hệ thống

Dưới đây là sơ đồ khối hệ thống:



Hình 3.1: Sơ đồ khối tổng quát hệ thống

Trong Hình 3.1 sơ đồ khối hệ thống, mô tả tổng quan các khối module chức năng bên trong giao diện gồm các chức năng được thiết kế theo yêu cầu xác định. Bao gồm 7 modules:

- APB Slave: là một trong hai thành phần trong cấu trúc của bus APB (master và slave). Modules này quản lý các tín hiệu điều khiển như đọc/ ghi (rd_en/ wr_en), địa chỉ (addr), các tín hiệu điều khiển khác (như psel, pwrite, pstrb, penable…); và các tín hiệu dữ liệu như wdata - dữ liệu ghi, rdata - dữ liệu đọc. APB Slave được thiết kế giao tiếp với thanh ghi của hệ thống.Master APB thường là một bus

khác có tốc độ cao hơn trong hệ thống bus trong bộ giao tiếp AMBA, ví dụ như AXI, AHB…

- Register: tập hợp tất cả những thanh ghi của hệ thống, lưu trữ các dữ liệu cấu hình như buad_div (tốc độ truyền dữ liệu), dữ liệu và các tín hiệu điều khiển khác (tx_enable, rx_enable).
- Baud_rate: khởi tạo tín hiệu điều khiển quá trình truyền/ nhận dữ liệu - i_Clock, xác định tốc độ truyền dữ liệu dựa trên giá trị baud_div được thiết lập từ thanh ghi.
- Send_FIFO và Receiver_FIFO: là các thanh ghi đệm lưu trữ dữ liệu truyền/ nhận, được thiết kế với chức năng lưu trữ các dữ liệu với kích thước tối đa là một byte; ngoài ra còn đảm bảo tín hiệu được xử lý theo thứ tự được đưa vào.
- Transmitter: là bộ truyền dữ liệu, còn có thể hiểu là bộ chuyển đổi tín hiệu song song từ khối Send_FIFO thành chuỗi dữ liệu nối tiếp theo tiêu chuẩn của một data frame tiêu chuẩn, truyền qua một đường truyền đếm các thiết bị khác thông qua cổng tx.
- Receiver: là bộ nhận dữ liệu, đối nghịch với chức năng của module transmitter, module này tiếp nhận chuỗi dữ liệu nối tiếp từ ngõ vào rx và chuyển đổi thành dữ liệu song song để lưu trữ vào receiver_FIFO để tiếp nhận xử lý (trong hệ thống, dữ liệu được lưu trữ trong thanh ghi Receiver_FIFO sẽ được lưu trữ tới khi APB Master tiến hành quá trình đọc dữ liệu truyền vào).

Sau đây là bảng mô tả I/O:

Bảng 3.1: Bảng mô tả tín hiệu

Tên	Độ rộng bit	I/O	Mô tả
sys_clk	1	Input	50 MHz
sys_rst_n	1	Input	Active-low Asynchronous reset. 0: reset 1: not reset
pselx	1	Input	Tín hiệu lựa chọn thiết bị - APB Slave. 0: Slave x không được chọn. 1: Slave x được chọn.
pwrite	1	Input	0: đọc - rd_en. 1: ghi - wr_en.
penable	1	Input	Tín hiệu xác nhận thực hiện giao dịch. 0: Không cho phép truyền dữ liệu. 1: Tiến hành thực hiện truyền dữ liệu.
paddr	12	Input	Địa chỉ thanh ghi hệ thống.
pwdata	16	Input	Dữ liệu ghi từ Master sang Slave.
prdata	16	Input	Dữ liệu đọc từ Slave vào Master.
pready	1	Output	Slave sẵn sàng cho quá trình truyền dữ liệu. 0: Chưa sẵn sàng. 1: Đã sẵn sàng.
tx	1	Output	Dữ liệu nối tiếp truyền đi.
rx	1	Output	Dữ liệu nối tiếp nhận vào.
int	1	Output	Ngắt.

Bảng 3.1 trình bày tổng quan thông tin về các tín hiệu I/O trong thiết kế. Những thông tin được nêu ra là những nội dung cần chú ý và tuân theo khi thiết kế các khối chức năng bên trong. Cần chú ý hướng di chuyển đường dữ liệu và quá trình cập nhật dữ liệu giữa các đường truyền phải đồng bộ nhau; bởi vì trong thiết kế luôn có các reserve bits không thể cập nhật dữ liệu vào đó.

3.3 Thiết kế khối chức năng

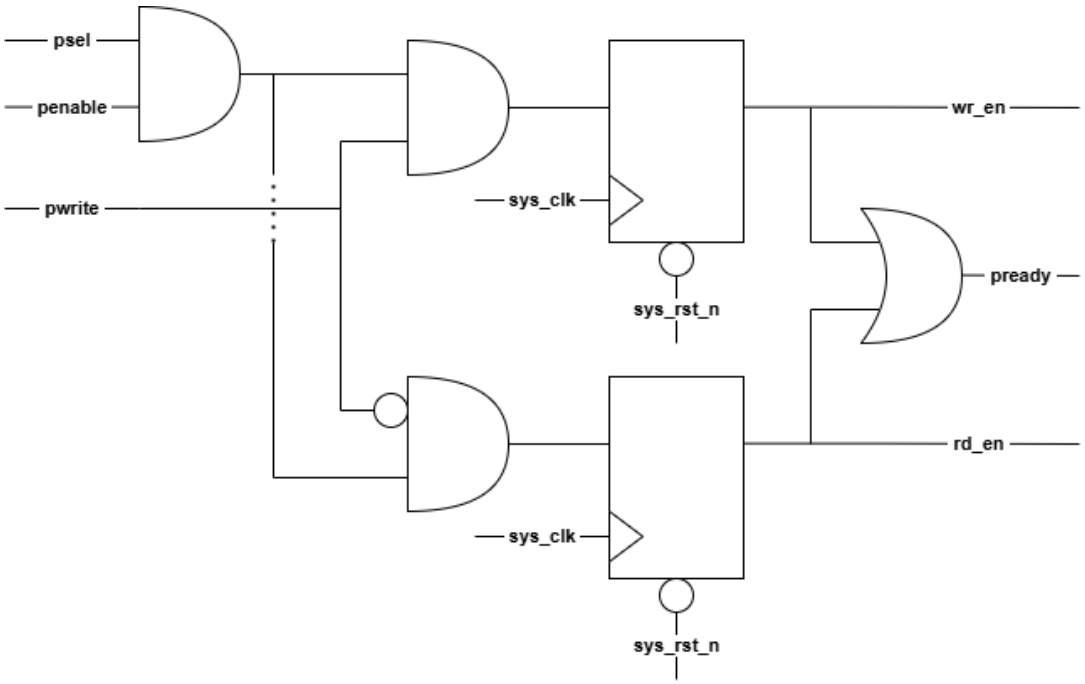
Sơ đồ khối tổng thể của hệ thống UART kết hợp giao tiếp APB được minh họa trong Hình 3.2. Hệ thống bao gồm các khối chức năng chính như: APB Slave, thanh ghi điều khiển (Register), bộ nhớ đệm FIFO, bộ tạo Baud rate, khối truyền (Transmitter) và khối nhận (Receiver). Mỗi khối thực hiện các chức năng chuyên biệt và phối hợp với nhau để đảm bảo quá trình truyền và nhận dữ liệu UART diễn ra chính xác, đồng bộ với giao thức APB.

3.3.1 APB Slave

Khối APB Slave đóng vai trò trung gian tiếp nhận tín hiệu điều khiển từ bus APB để tạo các tín hiệu điều khiển nội bộ cho hệ thống, bao gồm: đọc (rd_en), ghi (wr_en), và xác nhận (pready).

a. Sơ đồ cấu trúc

Hình dưới đây mô tả sơ đồ logic tổ hợp của khối APB Slave:



Hình 3.2: Sơ đồ logic tổ hợp của khối APB Slave

b. Mô tả hoạt động

Khối chức năng trong Hình 3.2 nhận các tín hiệu chuẩn của giao thức APB:

- **psel**: tín hiệu chọn slave.
- **penable**: tín hiệu cho phép giai đoạn truy cập.
- **pwrite**: xác định chu kỳ là ghi (1) hay đọc (0).

Dựa trên các tín hiệu này, khối tạo ra các tín hiệu điều khiển:

- **wr_en**: bật lên khi thực hiện ghi (psel & penable & pwrite).
- **rd_en**: bật lên khi thực hiện đọc (psel & penable & ~pwrite).
- **pready**: được bật khi có yêu cầu đọc hoặc ghi hợp lệ (psel & penable).

c. Thành phần logic

- **Cổng AND**: Kết hợp các điều kiện điều khiển để tạo wr_en và rd_en.
- **Cổng NOT**: Đảo giá trị pwrite phục vụ cho điều kiện đọc.
- **Cổng OR**: Dùng tạo tín hiệu pready khi có bất kỳ thao tác truy cập nào.

d. Vai trò và ý nghĩa

- Cho phép giao tiếp tuân theo chuẩn giao thức APB của ARM.
- Cung cấp tín hiệu điều khiển chính xác đến các khối con như Transmitter, baud_control , register.
- Đơn giản hóa điều kiện điều khiển và đồng bộ hóa các tín hiệu truy cập từ master đến slave.

3.3.2 Register

Khối Register đóng vai trò trung tâm trong việc quản lý và điều khiển hoạt động của High-Speed UART thông qua giao diện APB. Nó cung cấp một tập hợp các thanh ghi cho phép bộ vi xử lý cấu hình các tham số truyền nhận, truy cập dữ liệu và theo dõi trạng thái hoạt động của UART. Thiết kế khối Register đảm bảo sự tương tác hiệu quả giữa phần mềm và phần cứng của module UART.

a. Phân loại và Địa chỉ Thanh Ghi

Khối Register trong thiết kế này bao gồm các thanh ghi được ánh xạ vào không gian địa chỉ của APB Slave. Bảng 3.2 sau mô tả chi tiết từng thanh ghi, địa chỉ, kích thước và quyền truy cập:

Bảng 3.2: Bảng mô tả thanh ghi

Tên	Ký Hiệu	Địa Chỉ (Hex)	Kích Thước (bits)	Quyền Truy Cập	Mô Tả
Transmit Data Register	TDR	0x00	8	Chỉ Ghi	Chứa dữ liệu 8-bit được ghi bởi bộ vi xử lý để truyền đi.
Receive Data Register	RDR	0x04	8	Chỉ Đọc	Chứa dữ liệu 8-bit đã nhận được từ khối Receiver và có thể được đọc bởi bộ vi xử lý.
Status Register	SR	0x08	5	Chỉ Đọc	Chứa các cờ trạng thái của FIFO truyền (tx_empty, tx_full) và FIFO nhận (rx_empty, rx_full) cùng với cờ lỗi nhận (error từ Receiver).
Control Register	CR	0x0C	4	Đọc/Ghi	Cho phép/cấm bộ truyền (tx_enable), bộ thu (rx_enable), chế độ chẵn lẻ (parity_enable), và lựa chọn loại chẵn lẻ (parity_mode).
Baud Rate Divisor Register	BRDR	0x10	16	Đọc/Ghi	Chứa giá trị chia tần số xung clock hệ thống để tạo ra tốc độ baud mong muốn.
Interrupt Enable Register	IER	0x14	1	Đọc/Ghi	Bit cho phép ngắt (1: cho phép) khi có sự kiện ngắt xảy ra (từ FIFO đầy/rỗng hoặc lỗi nhận).
Interrupt Status Register	ISR	0x18	3	Đọc/Ghi (ghi để xóa)	Chứa các cờ trạng thái ngắt khi:

					- Bộ nhớ send_FIFO đầy dữ liệu (tx_fifo_int) - Bộ nhớ receive_FIFO đầy dữ liệu (rx_fifo_int) - Dữ liệu nhận vào lỗi parity (error_int). Ghi '1' vào bit 0 để khởi tạo lại cờ ngắt.

Bảng 3.2 mô tả chi tiết tên, ký hiệu, địa chỉ, kích thước, quyền truy cập và chức năng của từng thanh ghi.

b. Cấu Trúc Bit của Các Thanh Ghi Quan Trọng

- **Thanh ghi trạng thái (Status Register) Địa chỉ 0x08:**

Bảng sau đây mô tả chi tiết của thanh ghi trạng thái:

Bảng 3.3*:* Bảng mô tả thanh ghi trạng thái

Bit(s)	Tên Trường	Mô Tả
4	empty_rx	1: FIFO nhận rỗng, 0: FIFO nhận không rỗng
3	empty_tx	1: FIFO truyền rỗng, 0: FIFO truyền không rỗng
2	full_rx	1: FIFO nhận đầy, 0: FIFO nhận không đầy
1	full_tx	1: FIFO truyền đầy, 0: FIFO truyền không đầy
0	error	Cờ báo lỗi nhận từ khối Receiver.

Bảng 3.3 mô tả chi tiết tên, kích thước, và chức năng của thanh ghi trạng thái.

- **Thanh ghi Điều khiển (Control Register) Địa chỉ 0x0C:**

Bảng sau đây mô tả chi tiết của thanh ghi điều khiển:

Bảng 3.4: Bảng mô tả thanh ghi điều khiển

Bit(s)	Tên Trường	Mô Tả
3	parity_mode	Chọn chế độ chẵn lẻ (1: Lẻ, 0: Chẵn) - Chỉ có hiệu lực khi parity_enable = 1
2	parity_enable	Bit cho phép/cấm bit chẵn lẻ (1: Cho phép, 0: Cấm)
1	rx_enable	Bit cho phép/cấm bộ thu (1: Cho phép, 0: Cấm)
0	tx_enable	Bit cho phép/cấm bộ truyền (1: Cho phép, 0: Cấm)

Bảng 3.4 mô tả chi tiết tên, kích thước, và chức năng của thanh ghi điều khiển.

- **Thanh ghi cho phép ngắt (Interrupt Enable Register) Địa chỉ 0x14:**

Bảng sau đây mô tả chi tiết của thanh ghi cho phép ngắt:

Bảng 3.5: Bảng thanh ghi cho phép ngắt

Bit(s)	Tên Trường	Mô Tả
0	enable	0: Cấm ngắt tổng. 1: Cho phép ngắt tổng, kết hợp với các cờ ISR.

Bảng 3.5 mô tả chi tiết tên, kích thước, và chức năng của thanh ghi cho phép ngắt.

- **Thanh ghi trạng thái ngắt (Interrupt Status Register) Địa chỉ 0x18:**

Bảng sau đây mô tả chi tiết của thanh ghi trạng thái ngắt:

Bảng 3.6: Bảng thanh ghi trạng thái ngắt

Bit(s)	Tên Trường	Mô Tả
2	error_int	Cờ báo ngắt khi có lỗi nhận từ khối Receiver. Ghi '1' để xóa.
1	rx_fifo_int	Cờ báo ngắt khi FIFO nhận đầy. Ghi '1' để xóa.
0	tx_fifo_int	Cờ báo ngắt khi FIFO truyền đầy. Ghi '1' để xóa.

Bảng 3.6 mô tả chi tiết tên, kích thước, và chức năng của thanh ghi trạng thái ngắt.

- **Baud Rate Divisor Register (Địa chỉ 0x10):** Thanh ghi 16-bit này chứa giá trị chia tần số xung clock hệ thống để tạo ra tốc độ baud mong muốn. Giá trị này được gửi đến khối Baud Rate Generator.
- **Transmit Data Register (Địa chỉ 0x00):** Thanh ghi 8-bit này được ghi bởi bộ vi xử lý để chứa dữ liệu sẽ được truyền đi. Dữ liệu này sau đó được chuyển đến khối Send FIFO.
- **Receive Data Register (Địa chỉ 0x04):** Thanh ghi 8-bit này chứa dữ liệu đã nhận được từ khối Receive FIFO và có thể được đọc bởi bộ vi xử lý.

c. Cơ Chế Truy Cập Thanh Ghi

Khối Register hoạt động dựa trên các tín hiệu điều khiển từ giao diện APB (clk, rst_n, wr_en, rd_en, ready, strb, addr, wdata, rdata). Khi bộ vi xử lý muốn truy cập một thanh ghi, nó sẽ đặt địa chỉ của thanh ghi đó lên bus địa chỉ (addr) và kích hoạt tín hiệu chọn slave (psel, được ngầm hiểu trong addr hợp lệ).

- **Ghi (Write):** Khi tín hiệu wr_en và ready đồng thời được kích hoạt, và địa chỉ addr trùng với địa chỉ của một thanh ghi có quyền ghi, dữ liệu trên bus ghi (wdata) sẽ được lưu vào thanh ghi đó. Tín hiệu strb (strobe) cho phép ghi byte-enable, tuy nhiên trong thiết kế này, hầu hết các thanh ghi được truy cập theo đơn vị byte hoặc word (16-bit cho baud_div).
- **Đọc (Read):** Khi tín hiệu rd_en và ready đồng thời được kích hoạt, và địa chỉ addr trùng với địa chỉ của một thanh ghi có quyền đọc, dữ liệu từ thanh ghi đó sẽ được đưa lên bus đọc (rdata).

Quá trình giải mã địa chỉ được thực hiện bên trong khối Register để xác định thanh ghi nào đang được truy cập. Các tín hiệu data_tx_wr_sel, control_wr_sel, bauddiv_wr_sel, ier_wr_sel, và isr_wr_sel trong mã Verilog cho thấy cách địa chỉ và tín hiệu wr_en được kết hợp để chọn thanh ghi đích cho thao tác ghi. Tương tự, khối always *@(*)* với điều kiện rd_en & ready và cấu trúc case xác định dữ liệu nào sẽ được trả về trên rdata dựa trên địa chỉ đọc.

d. Tương Tác với Các Khối Chức Năng Khác

Khối Register đóng vai trò là cầu nối giữa bộ vi xử lý và các khối chức năng khác của UART:

- **Với khối APB Slave:** Khối Register nhận các yêu cầu đọc/ghi từ APB Slave thông qua các tín hiệu điều khiển và địa chỉ, sau đó cung cấp dữ liệu hoặc lưu trữ cấu hình tương ứng.
- **Với khối Send FIFO:** Thanh ghi **Transmit Data Register (TDR)** là nơi bộ vi xử lý ghi dữ liệu cần truyền. Dữ liệu này sau đó được chuyển đến khối Send FIFO thông qua tín hiệu data_tx. Thanh ghi **Status Register (SR)** cung cấp thông tin về trạng thái đầy/rỗng của Send FIFO (full_tx, empty_tx).
- **Với khối Receive FIFO:** Dữ liệu nhận được từ khối Receiver được lưu trữ trong Receive FIFO. Bộ vi xử lý đọc dữ liệu đã nhận thông qua thanh ghi **Receive Data Register (RDR)** (data_out_rx được gán cho rdata khi đọc địa chỉ 0x04). Thanh ghi **Status Register (SR)** cũng cung cấp thông tin về trạng thái đầy/rỗng của Receive FIFO (full_rx, empty_rx).

- **Với khối Baud Rate Generator:** Giá trị được ghi vào thanh ghi **Baud Rate Divisor Register (BRDR)** sẽ được xuất ra tín hiệu baud_div và được sử dụng bởi khối Baud Rate Generator để tạo ra tốc độ baud cần thiết.
- **Với khối Transmitter và Receiver:** Các bit điều khiển từ thanh ghi **Control Register (CR)** (tx_enable, rx_enable, parity_enable, parity_mode) được xuất ra để cấu hình hoạt động của bộ truyền và bộ thu.
- **Với khối Interrupt Handling:** Các cờ trạng thái từ FIFO (full_tx, full_rx) và tín hiệu lỗi từ Receiver (error) được theo dõi để tạo ra các cờ ngắt trong **Interrupt Status Register (ISR)**. Bit cho phép ngắt trong **Interrupt Enable Register (IER)** điều khiển việc tín hiệu interrupt được kích hoạt hay không.

e. Hiện Thực Hóa Mạch Logic

Khối Register được triển khai dựa trên mã Verilog bằng cách sử dụng các biến kiểu reg nhằm lưu giữ giá trị của các thanh ghi chức năng. Việc ghi dữ liệu vào các thanh ghi được xử lý thông qua khối *always @(posedge clk or negedge rst_n),* đảm bảo quá trình ghi diễn ra đồng bộ với xung clock và có khả năng khởi tạo lại khi tín hiệu reset hoạt động. Trong khi đó, thao tác đọc được hiện thực dưới dạng bất đồng bộ thông qua khối *always @(*),* sử dụng lệnh case để trả về giá trị tương ứng với địa chỉ thanh ghi được chọn khi có yêu cầu đọc hợp lệ. Các điều kiện so sánh địa chỉ, cùng với tín hiệu điều khiển wr_en và rd_en, được sử dụng nhằm xác định đúng thanh ghi đích cho hoạt động ghi hoặc đọc.

f. Vai trò của việc thiết kế khối Register

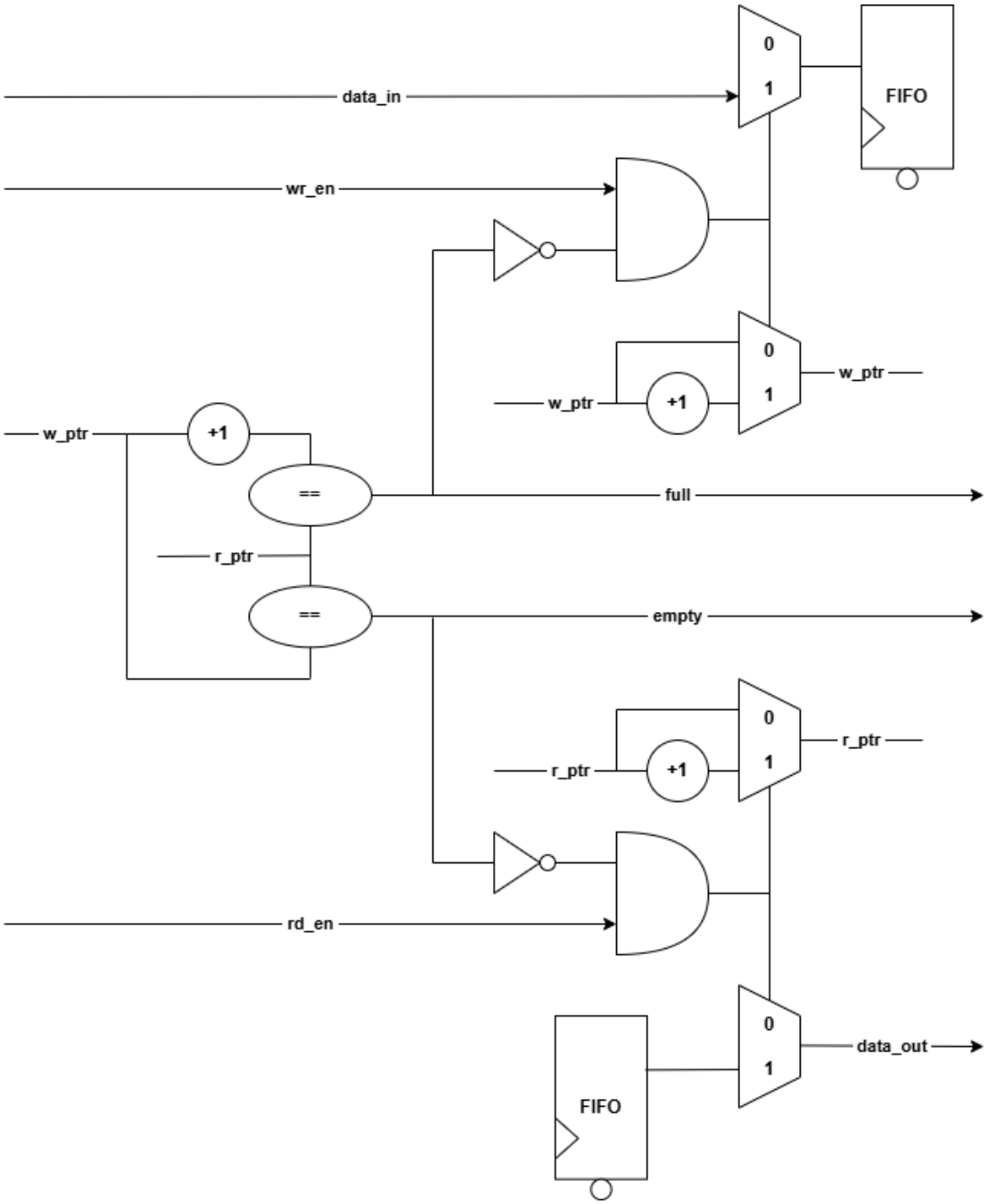
Việc xây dựng khối Register một cách logic, có tổ chức và dễ kiểm soát đóng vai trò then chốt trong việc quản lý toàn bộ hoạt động của module UART tốc độ cao. Việc phân loại rõ ràng các thanh ghi theo từng nhóm chức năng như: điều khiển, trạng thái, và dữ liệu – giúp cho bộ xử lý dễ dàng tương tác và giám sát hệ thống. Ngoài ra, việc tích hợp cơ chế xử lý ngắt thông qua các thanh ghi như IER (Interrupt Enable Register) và ISR (Interrupt Status Register) giúp hệ thống có khả năng phản hồi nhanh chóng với các sự kiện trong quá trình truyền nhận, từ đó nâng cao độ ổn định và hiệu suất vận hành.

3.3.3 FIFO

Khối **FIFO (First-In-First-Out)** là một thành phần quan trọng trong hệ thống truyền nhận dữ liệu như UART. FIFO giúp lưu trữ tạm thời dữ liệu vào/ra, tránh mất mát dữ liệu khi có sự chênh lệch tốc độ giữa quá trình ghi và đọc [7].

a. Sơ đồ cấu trúc

Hình sau biểu diễn sơ đồ logic của khối FIFO:



Hình 3.3*:* Sơ đồ logic của khối FIFO

b. Mô tả hoạt động

Sơ đồ logic của khối FIFO Hình 3.3 hoạt động dựa trên hai con trỏ chính:

- w_ptr (write pointer): trỏ đến vị trí ghi tiếp theo
- r_ptr (read pointer): trỏ đến vị trí đọc tiếp theo

Khối sử dụng hai tín hiệu điều khiển chính:

- wr_en: kích hoạt ghi dữ liệu vào FIFO (chỉ khi FIFO chưa đầy)
- rd_en: kích hoạt đọc dữ liệu từ FIFO (chỉ khi FIFO không rỗng)

c. Các tín hiệu điều khiển

- **Ghi dữ liệu (wr_en)**: Được kích hoạt khi wr_en = 1 và FIFO chưa đầy dữ liệu (~full). Dữ liệu data_in được ghi vào FIFO tại vị trí w_ptr. w_ptr được tăng thêm 1 sau mỗi lần ghi thành công
- **Đọc dữ liệu (rd_en)**: Được kích hoạt khi rd_en = 1 và FIFO có dữ liệu (~empty). Dữ liệu tại vị trí r_ptr được xuất ra data_out. r_ptr được tăng thêm 1 sau mỗi lần đọc thành công
- **Tín hiệu full**: Báo FIFO đầy khi w_ptr + 1 == r_ptr
- **Tín hiệu empty**: Báo FIFO rỗng khi w_ptr == r_ptr

d. Vai trò và ý nghĩa

- **Tạm lưu các dữ liệu** giữa các thành phần trong hệ thống truyền nhận
- **Tránh mất dữ liệu** khi thiết bị ghi nhanh hơn thiết bị đọc hoặc ngược lại
- **Đơn giản hóa điều khiển** trong các giao thức như UART, SPI, hay DMA

e. Ưu điểm thiết kế

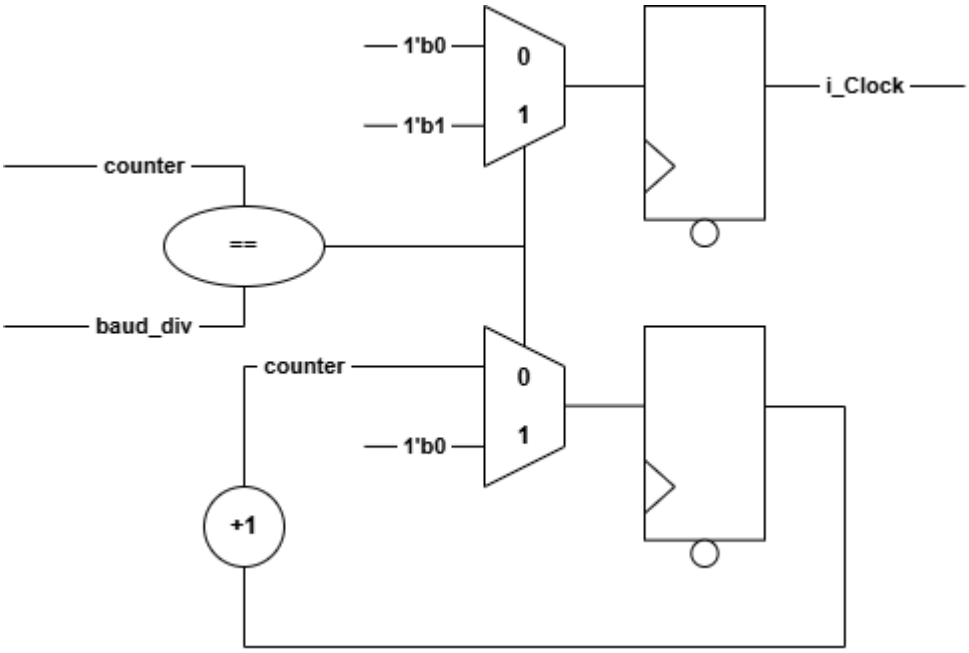
- Sử dụng logic tổ hợp đơn giản (AND, NOT, Multiplexer)
- Dễ mở rộng với nhiều bit dữ liệu
- Có khả năng tương thích với nhiều giao tiếp chuẩn khác

3.3.4 Baud rate

Khối **Baud rate generator** có nhiệm vụ tạo xung nhịp truyền dữ liệu theo đúng tốc độ baud yêu cầu từ tín hiệu xung hệ thống i_Clock. Đây là thành phần quan trọng giúp đảm bảo quá trình truyền nhận dữ liệu UART diễn ra đúng tần số.

a. Sơ đồ cấu trúc

Hình sau mô tả cấu trúc khối Baud rate generator:



Hình 3.4: Sơ đồ logic của khối Baud_rate_generator

b. Mô tả hoạt động

Khối Baud rate trong Hình 3.4 hoạt động dựa trên bộ đếm (counter) và tham số chia (baud_div) do người dùng lập trình.

- Mỗi chu kỳ xung i_Clock, bộ đếm counter sẽ tăng thêm 1.
- Khi giá trị counter đạt đến giá trị baud_div, một xung baud sẽ được sinh ra, dùng làm tín hiệu nhịp cho bộ phát và bộ thu UART.
- Sau đó, counter sẽ được reset về 0 và quá trình lặp lại.

c. Thành phần chức năng

- **Bộ so sánh**: So sánh giá trị của counter và baud_div.
- **MUX và D Flip-Flop**: Kết hợp để sinh ra xung baud đúng thời điểm.
- **Bộ đếm tăng**: Tăng counter lên mỗi chu kỳ i_Clock.

d. Ưu điểm thiết kế

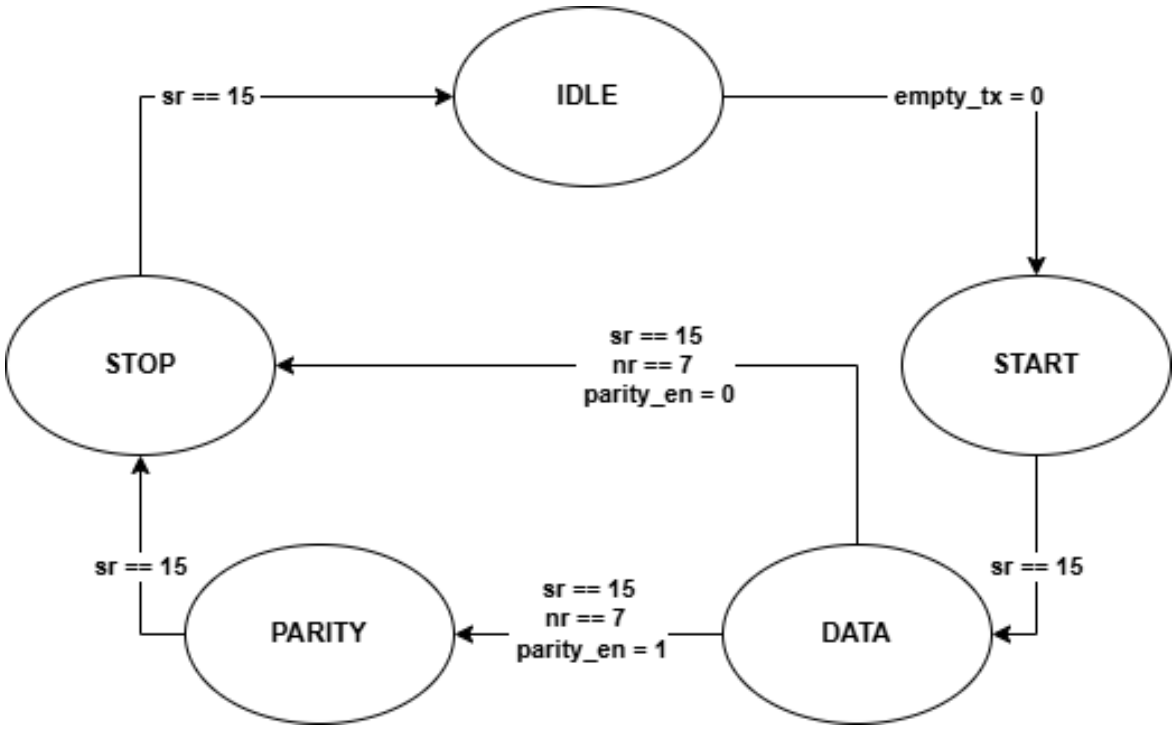
- Cho phép lập trình linh hoạt tốc độ baud bằng cách thay đổi giá trị baud_div.
- Đảm bảo tạo ra xung baud ổn định và đồng bộ với hệ thống.
- Hoạt động hiệu quả với mọi tốc độ baud trong phạm vi phần cứng cho phép (lên tới 3.125 Mbps với xung hệ thống 50 MHz).

3.3.5 Transmitter

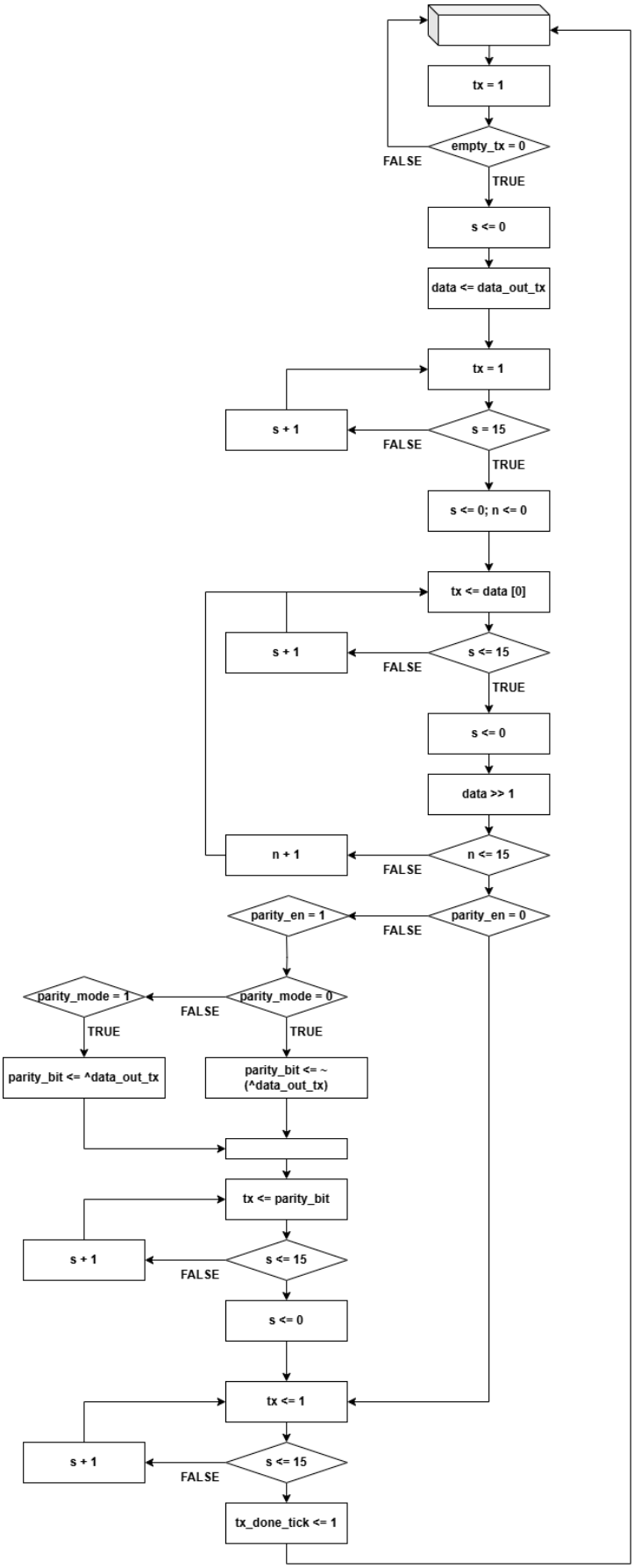
Khối **Transmitter (bộ phát dữ liệu)** chịu trách nhiệm truyền dữ liệu nối tiếp ra đường truyền UART. Dữ liệu từ FIFO sẽ được phát ra theo chuẩn UART gồm các bit: start, data, parity (nếu có) và stop. Quá trình truyền dữ liệu được điều khiển qua một **FSM (Finite State Machine)** – nghĩa là máy trạng thái hữu hạn.

a. Sơ đồ trạng thái

Hình sau minh họa FSM điều khiển quá trình truyền dữ liệu trong khối Transmitter:



Hình 3.5: FSM Transmitter



Hình 3.6: Lưu đồ FSM Transmitter

b. Các trạng thái và hoạt động

FSM của Transmitter Hình 3.5 và Hình 3.6 gồm 5 trạng thái chính:

- **IDLE**: Trạng thái chờ. Nếu emty_tx == 1'b0 (FIFO không rỗng), chuyển sang trạng thái START. Đây là trạng thái khởi đầu và kết thúc sau mỗi chu kỳ truyền.
- **START**: Truyền bit **start** (logic 0). Sau khi s_r == 15, chuyển qua trạng thái DATA.
- **DATA**: Truyền tuần tự 8 bit dữ liệu (sử dụng n_r để đếm số bit). Sau khi s_r == 15 và n_r == 7: Nếu parity_en == 1'b1, chuyển sang trạng thái PARITY. Nếu không, chuyển sang trạng thái STOP.
- **PARITY**: Truyền bit chẵn lẻ (odd hoặc even parity). Sau khi s_r == 15, chuyển sang trạng thái STOP.
- **STOP**: Truyền bit **stop** (logic 1). Sau khi s_r == 15, trở lại trạng thái IDLE để chờ khung dữ liệu kế tiếp.

c. Các tín hiệu điều khiển

- s_r: bộ đếm dùng để tạo độ trễ thời gian cho từng trạng thái
- n_r: bộ đếm bit dùng trong quá trình truyền dữ liệu
- parity_en: bật/tắt kiểm tra chẵn lẻ
- emty_tx: tín hiệu thông báo FIFO phát có dữ liệu

d. Vai trò trong hệ thống UART

- **Đảm bảo truyền đúng định dạng UART**: Start - Data - Parity - Stop.
- **Tương thích với chuẩn giao tiếp nối tiếp**: Giúp truyền dữ liệu sang thiết bị nhận (Receiver).
- **Tự động hóa quá trình truyền**: Bằng cách sử dụng FSM giúp đảm bảo chính xác và tuần tự.

e. Ưu điểm của thiết kế

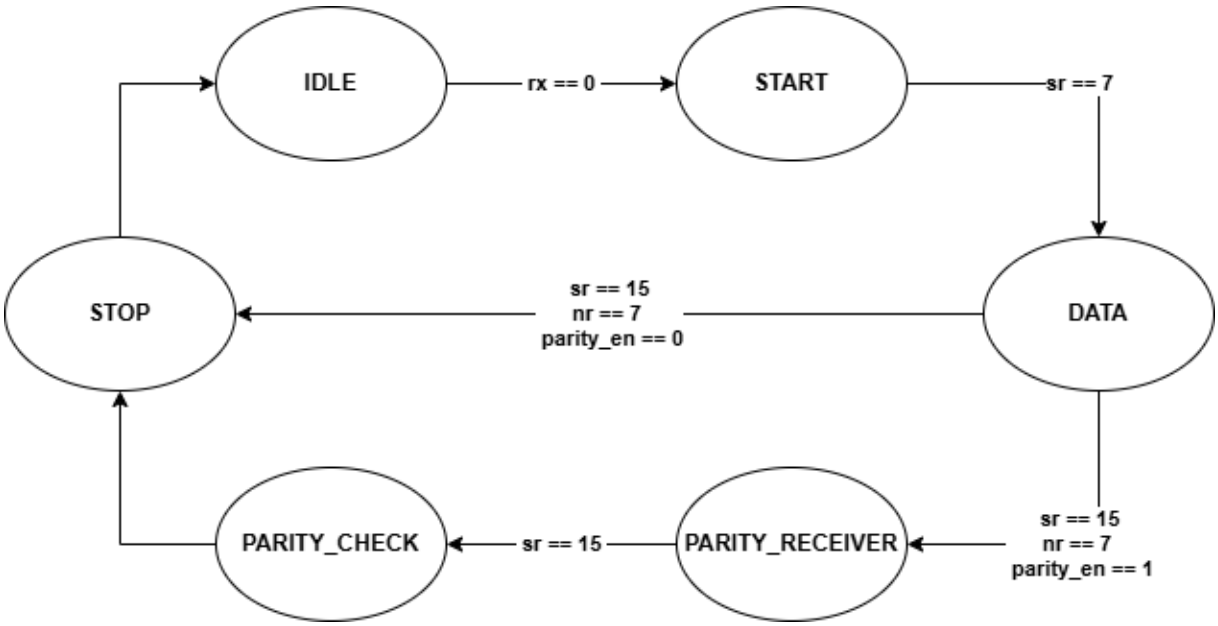
- Hoạt động tuần tự rõ ràng, dễ dàng kiểm soát và mở rộng.
- Có thể cấu hình bật/tắt kiểm tra parity linh hoạt.
- Tốc độ truyền có thể điều chỉnh thông qua bộ chia xung.

3.3.6 Receiver

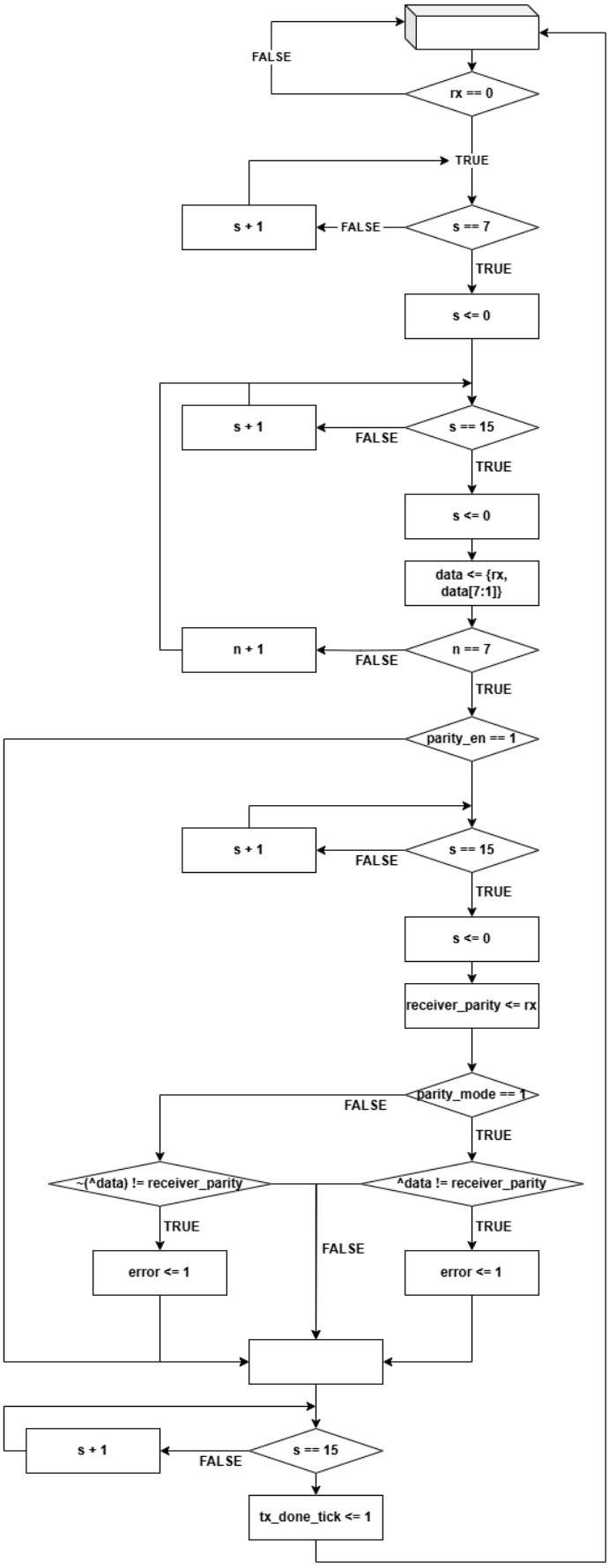
Khối **Receiver** chịu trách nhiệm nhận dữ liệu nối tiếp từ đường truyền UART (rx). Dữ liệu được thu thập tuần tự theo từng bit, kiểm tra bit **start**, đọc các bit **data**, kiểm tra **parity** (nếu có), và kết thúc bằng bit **stop**. Sau khi nhận đầy đủ một khung dữ liệu hợp lệ, dữ liệu sẽ được ghi vào FIFO nhận để xử lý sau.

a. Sơ đồ trạng thái

Quá trình nhận dữ liệu được điều khiển bởi một **FSM (Finite State Machine)** dưới đây là sơ đồ mô tả trạng thái máy:



Hình 3.7: FSM Receiver



Hình 3.8: Lưu đồ FSM Receiver

b. Các trạng thái và hoạt động

FSM của khối Receiver Hình 3.7 và Hình 3.8 gồm 5 trạng thái:

- **IDLE**: Trạng thái mặc định, chờ dữ liệu đến. Nếu phát hiện bit rx == 1'b0 (start bit), chuyển qua trạng thái START.
- **START**: Nhận bit start. Sau khi s_r == 7, chuyển sang trạng thái DATA.
- **DATA**: Nhận tuần tự 8 bit dữ liệu từ đường truyền rx. Bộ đếm n_r tăng từ 0 đến 7. Khi n_r == 7 và s_r == 15: Nếu parity_en == 1'b1, chuyển sang trạng thái PARITY. Nếu không, chuyển sang trạng thái STOP.
- **PARITY**: Nhận bit parity từ đường truyền. Sau khi s_r == 15, chuyển sang trạng thái STOP.
- **STOP**: Nhận bit stop (logic 1). Sau khi s_r == 15, chuyển về lại trạng thái IDLE.

c. Các tín hiệu điều khiển

- rx: tín hiệu đầu vào chứa dữ liệu nối tiếp cần nhận.
- s_r: bộ đếm tạo độ trễ thời gian cho từng trạng thái.
- n_r: bộ đếm bit dữ liệu trong trạng thái DATA.
- parity_en: bật/tắt kiểm tra chẵn lẻ.
- data_out: dữ liệu 8 bit được xuất ra sau khi nhận đủ và hợp lệ.
- wr_en: tín hiệu ghi dữ liệu vào FIFO nhận.

d. Vai trò trong hệ thống UART

- Phát hiện và thu nhận dữ liệu đúng định dạng UART: Start - Data - Parity - Stop.
- Đảm bảo tính toàn vẹn dữ liệu: kiểm tra chẵn lẻ nếu bật parity_en.
- Khi nhận dữ liệu lỗi sẽ báo error và bật bộ ngắt (intterupt).
- Giao tiếp với FIFO: lưu trữ dữ liệu nhận được để CPU truy xuất sau.

e. Ưu điểm của thiết kế

- Sử dụng FSM giúp quá trình nhận dữ liệu diễn ra chính xác và rõ ràng.
- Có khả năng cấu hình theo chuẩn UART có/không parity.
- Thiết kế đồng bộ, dễ dàng tích hợp với các module khác như FIFO và APB.

CHƯƠNG 4: KẾT QUẢ MÔ PHỎNG

4.1 Module mô phỏng

Trong chương 4, nhóm trình bày kết quả mô phỏng của từng khối chức năng trong hệ thống APB UART, bao gồm mô-đun APB Master, mô-đun UART và các tín hiệu giao tiếp giữa chúng. Việc mô phỏng được thực hiện trên phần mềm QuestaSim, đảm bảo các mô-đun hoạt động đúng theo mong đợi trong các trường hợp kiểm thử.

Trong quá trình xác minh hệ thống "Thiết kế và xác minh High-Speed UART với giao thức APB", các test case được xây dựng có hệ thống dựa trên việc phân tích và phân rã từng khối chức năng chính của thiết kế, bao gồm: giao tiếp APB (APB_Slave), hệ thống thanh ghi (Register), cơ chế ngắt (Interrupt), và khối truyền/nhận dữ liệu (Transmitter and Receiver). Mỗi khối được chia thành các mục kiểm thử chi tiết (Sub Item), phản ánh từng tình huống hoạt động cụ thể như khởi tạo, ghi/đọc thanh ghi, phản hồi tín hiệu, xác lập điều kiện sinh ngắt, kiểm tra tín hiệu dữ liệu đầu ra, xử lý khi reset hệ thống, hay hoạt động trong trạng thái rỗi. Đối với mỗi test case, nhóm thực hiện xác định rõ các tín hiệu đầu vào (Input), điều kiện ban đầu, bước kiểm thử cụ thể và kết quả đầu ra mong đợi (Expected Output). Sau đó, hệ thống được đưa vào mô phỏng trên công cụ kiểm thử số để ghi nhận kết quả thực tế (Actual Output) và đối chiếu với kết quả dự kiến để đánh giá tính đúng đắn. Những sai lệch (nếu có) sẽ được phân tích nguyên nhân để điều chỉnh thiết kế hoặc cập nhật lại testbench tương ứng. Cách xây dựng test case theo hướng chức năng cụ thể giúp đảm bảo việc kiểm thử có tính bao quát toàn diện, đồng thời dễ dàng truy vết lỗi trong từng khối logic, từ đó nâng cao tính tin cậy và độ chính xác của thiết kế trước khi đưa vào tổng hợp hoặc triển khai trên phần cứng.

Dưới đây là bảng tóm tắt các trường hợp testcase:

Bảng 4.1: Bảng tóm tắt testcase

Item	No	Sub Item 1	Sub Item 2
APB_Slave	**1**	**Initialization**	Write Operation
	2	**Write Operation**	Write Completion
	3		Read Operation
	4	**Read Operation**	Read Completion

	5		Timing check
	6	**PREADY**	
	7	**Idle state**	
	8	**Reset Behavior**	Write/ Read Operation
	9	**Multiple Operation**	
Register	**10**	**Default value**	Write/ Read register
	11	**DATA_TX**	Set baudrate for UART
	12	**BAUDRATE**	Set condition to interrupt
	13	**IER**	Read state interrupt
	14	**ISR**	Clear state interrupt
	15		
	16	**Reserved bits**	
Interrupt	**17**	**State**	Read state of interrupt in module
	18		Write 1 to clear
	19	**Enable**	
	20	**Interrupt**	
Transmitter And Receiver	**21**	**Data frame**	
	22	**Transmitter**	Update data prepared to send
	23		Send data
	24	**Receiver**	Update data into FIFO when received
	25		Receive data

Bảng 4.1 mô tả tổng quát các nhóm testbench chính được sử dụng trong quá trình mô phỏng và xác minh chức năng của hệ thống UART kết hợp giao tiếp APB. Mỗi nhóm

chức năng được phân chia theo các khối logic trong thiết kế, giúp theo dõi và đánh giá từng thành phần cụ thể một cách hệ thống và rõ ràng.

- **Khối APB_Slave**: Kiểm thử đầy đủ các trạng thái và giao dịch cơ bản theo chuẩn AMBA, gồm khởi tạo, đọc/ghi dữ liệu, trạng thái rỗi, tín hiệu PREADY và phản ứng reset.
- **Khối Register**: Kiểm tra giá trị mặc định, khả năng đọc/ghi từng thanh ghi điều khiển, đặc biệt là các thanh ghi cấu hình baudrate, ngắt, và thanh ghi dữ liệu.
- **Khối Interrupt**: Mô phỏng các tình huống phát sinh ngắt khi FIFO đầy hoặc có lỗi nhận, xác minh cơ chế bật/tắt ngắt và ghi để xoá cờ ngắt.
- **Khối Truyền và Nhận (Transmitter & Receiver)**: Đánh giá khả năng truyền nhận dữ liệu nối tiếp, cập nhật đúng dữ liệu vào FIFO khi nhận và phát đúng định dạng khung truyền.

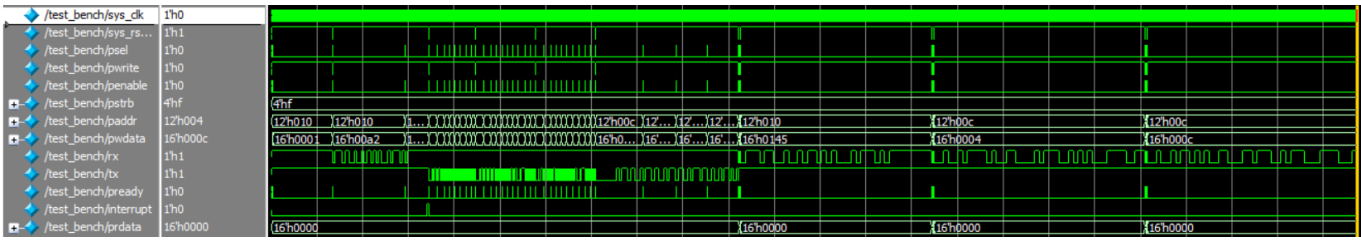
Nhận xét: Việc chia các testcase theo cấu trúc thiết kế giúp kiểm tra **toàn diện** các chức năng, đảm bảo rằng từng module trong hệ thống được xác minh một cách **độc lập và rõ ràng**, đồng thời có thể dễ dàng xác định nguồn gốc lỗi khi phát sinh trong quá trình kiểm thử.

4.2 APB Master

Để kiểm tra hoạt động chính xác của toàn bộ hệ thống, một mô-đun testbench đã được xây dựng, đóng vai trò kiểm thử và điều khiển quá trình mô phỏng. Testbench bao gồm toàn bộ các testcase:

- Bộ tạo xung clock: Tín hiệu clock được tạo với chu kỳ xác định để cung cấp cho hệ thống.
- Bộ tạo reset: Reset đồng bộ hoặc bất đồng bộ nhằm khởi tạo trạng thái hệ thống.
- APB Master giả lập: Gửi lệnh đọc/ghi đến slave UART theo chuẩn giao tiếp APB.
- Mô hình truyền nhận dữ liệu: Gửi các khung dữ liệu qua UART và kiểm tra đầu ra.
- Mô hình kiểm tra tốc độ baud: Nhập các giá trị baud rate mong muốn và kiểm tra đầu ra.
- Kiểm tra ngắt: Gửi các khung dữ liệu qua UART và kiểm tra đầu ra.

Dưới đây là kết quả mô phỏng và Waveform của testbench:



Hình 4.1 Waveform của testbench

```
# ========  Test Summary ========
# Total Tests:        127
# Total Errors:         0
# TEST PASSED
# ** Note: $finish    : test_bench.v(755)
#    Time: 23737290 ns  Iteration: 0  Instance: /test_bench
```

Hình 4.2*:* Kết quả mô phỏng Log của Testbench

Nhận xét Waveform Hình 4.1:

- **Giao tiếp APB**: Trong các đoạn đầu của waveform, tín hiệu psel, penable, pwrite, paddr, và pwdata thể hiện hoạt động **ghi và đọc dữ liệu qua APB bus**.
- **Kiểm tra thanh ghi UART**: Việc đọc, ghi các thanh ghi DATA_TX, STATE, CONTROL, BAUDDIV, IER, ISR được thực hiện nhằm đảm bảo dữ liệu phản hồi đúng như mong đợi. Các phép ghi lặp lại với pstrobe hoặc cấu hình các giá trị divisor tương ứng với tốc độ truyền (BAUD_9600 đến BAUD_3125000) cũng được thực hiện rõ ràng, kiểm tra hoạt động của bộ chia baudrate.
- **Kiểm tra ngắt và FIFO**: Các đoạn có tín hiệu interrupt lên mức cao cho thấy chức năng ngắt hoạt động chính xác khi có dữ liệu truyền hoặc nhận. Các testcase như Recever FIFO ×3 được dùng để kiểm tra hoạt động của hàng đợi FIFO, kể cả khi đầy (FIFO full check).
- **Kiểm tra chế độ parity**: Các đoạn sau mô phỏng quá trình **truyền – nhận dữ liệu với kiểm tra parity** (No parity, Odd parity, Even parity). Các testcase cho thấy UART xử lý chính xác cả ba chế độ parity, bao gồm việc sinh và kiểm tra bit parity trong truyền thông nối tiếp.

Kết luận:

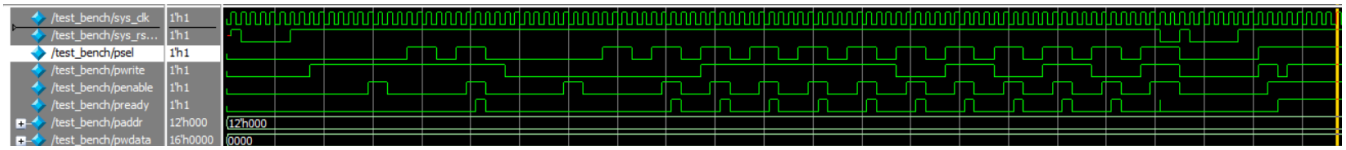
Waveform mô phỏng Hình 4.1 và kết quả Log Hình 4.2 tổng thể cho thấy **hệ thống APB UART đã đáp ứng đầy đủ các yêu cầu kiểm tra chức năng**, từ giao tiếp

APB chuẩn, phản ứng với reset, cấu hình thanh ghi đến xử lý truyền thông nối tiếp có parity. Không xuất hiện lỗi trong toàn bộ quá trình, cho thấy thiết kế đạt chất lượng và độ ổn định cao.

4.3 Testcase

4.3.1 APB Interface

Mục này mô tả các trường hợp kiểm thử được sử dụng để xác minh tính đúng đắn của giao tiếp giữa module APB Slave và bus APB Master. Các trường hợp kiểm thử bao gồm các kịch bản ghi, đọc, ghi liên tiếp, đọc-ghi xen kẽ, reset trong giao dịch và thay đổi tín hiệu pwrite giữa giao dịch. Kết quả mô phỏng được ghi lại và phân tích để đảm bảo module APB Slave hoạt động theo đúng đặc tả của giao thức APB. Kết quả Waveform thu được:



Hình 4.3: Waveform của APB Slave

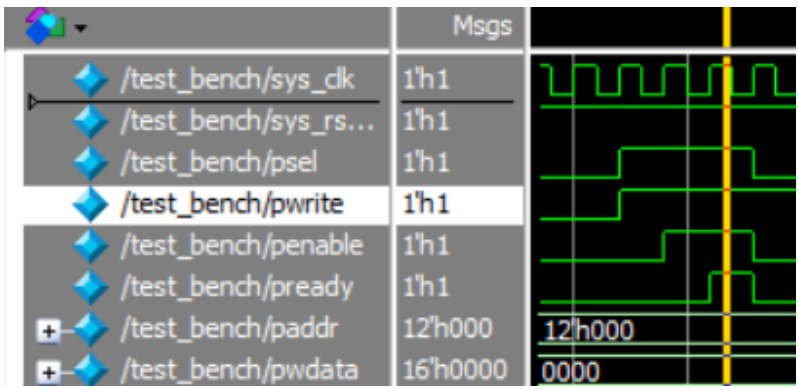
Hình 4.3 là kết quả mô phỏng waveform của module **APB Slave**, cho thấy rõ hoạt động trong các tình huống kiểm tra khác nhau. Kèm theo đó là log kết quả từng testcase được hiển thị trực tiếp từ testbench:

```
# <<<---------------- APB_SLAVE ---------------------->>>
# -------------------- APB Write --------------------------
# [212] PASS - Pready: Value = 0
# [312] PASS - Pready: Value = 0
# [412] PASS - Pready: Value = 0
# [512] PASS - Pready: Value = 1
# -------------------- APB Read ---------------------------
# [612] PASS - Pready: Value = 0
# [712] PASS - Pready: Value = 0
# [812] PASS - Pready: Value = 0
# [912] PASS - Pready: Value = 1
# -------------------- Back to Back Write -----------------
# [1012] PASS - Pready: Value = 1
# [1112] PASS - Pready: Value = 1
# [1212] PASS - Pready: Value = 1
# -------------------- Alternating Write-Read -------------
# [1312] PASS - Pready: Value = 1
# [1412] PASS - Pready: Value = 1
# [1512] PASS - Pready: Value = 1
# [1612] PASS - Pready: Value = 1
# [1712] PASS - Pready: Value = 1
# [1812] PASS - Pready: Value = 1
# -------------------- Reset During Transaction -----------
# [1932] PASS - Pready: Value = 0
# ---------- Hold PSEL HIGH for Multiple Cycles ----------
# [2152] PASS - Pwrite = 1
# [2152] PASS - Pready = 1
# [2172] PASS - Pread = 1
# [2172] PASS - Pready = 1
# [2192] PASS - Pwrite = 1
# [2192] PASS - Pready = 1
#
```

Hình 4.4: Kết quả log của APB

APB Write:

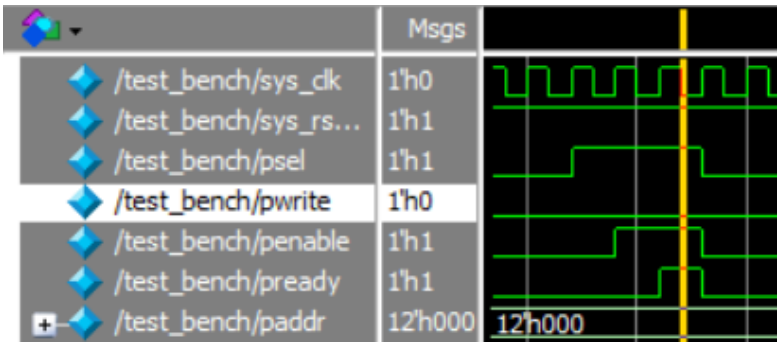
Trong Hình 4.5, cho thấy tín hiệu pready được giữ ở mức 0 trong các chu kỳ đầu (mô phỏng trạng thái **IDLE**) và khi tín hiệu psel được bật thì chuyển qua trạng thái lên mức 1 ở trạng thái chuẩn bị (**SETUP)** và tín hiệu pwrite ở mức 1 nghĩa là sẽ ghi dữ liệu, cuối cùng là penable bật lên 1 sau 1 chu kỳ thì pready được nâng lên mức 1 báo hiệu slave sẵn sàng nhận dữ liệu (**ACCESS**). Điều này chứng minh giao thức xử lý đúng trạng thái chờ khi ghi dữ liệu.



Hình 4.5: Kết quả ghi dữ liệu vào của APB

APB Read:

Tương tự như ghi, trong Hình 4.6, cho thấy tín hiệu pready được giữ ở mức 0 trong các chu kỳ đầu (mô phỏng trạng thái **IDLE**) và khi tín hiệu psel được bật thì chuyển qua trạng thái lên mức 1 ở trạng thái chuẩn bị (**SETUP)** và pwrite ở trạng thái 0 nghĩa là quá trình đọc, cuối cùng là penable bật lên 1 sau 1 chu kỳ thì pready được nâng lên mức 1 báo hiệu slave sẵn sàng nhận dữ liệu (**ACCESS**). Điều này chứng minh giao thức xử lý đúng trạng thái chờ khi ghi dữ liệu.



Hình 4.6: Kết quả ghi dữ liệu vào của APB

Back-to-Back Write:

Tương tự ở Hình 4.4, từ [1012] đến [1212], pready duy trì ở mức cao (1), cho phép thực hiện nhiều lần ghi liên tiếp mà không cần chờ đợi, điều này kiểm tra khả năng xử lý nhanh và liên tục của slave.

Alternating Write-Read:

Trong các chu kỳ [1312] đến [1812], hoạt động ghi và đọc được thực hiện xen kẽ, với tín hiệu pready luôn ở mức 1. Điều này cho thấy slave có thể xử lý tốt các hoạt động xen kẽ, không gây ra lỗi định thời hoặc xung đột.

Reset During Transaction:

Tại chu kỳ [1932], tín hiệu pready = 0, đúng với hành vi mong muốn: khi có tín hiệu reset xảy ra giữa quá trình giao tiếp, slave dừng phản hồi – giúp kiểm tra tính đúng đắn của cơ chế reset.

Hold psel HIGH for Multiple Cycles:

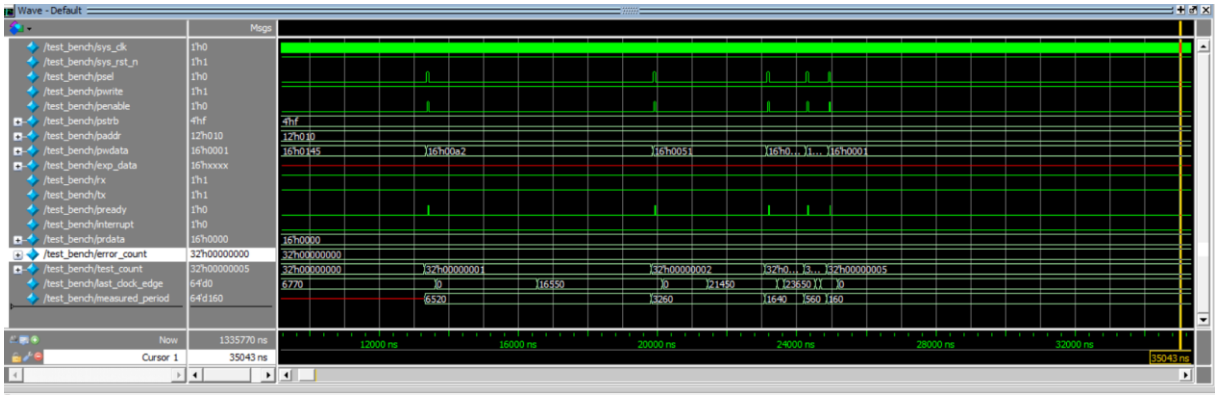
Ở các chu kỳ [2152], [2172], và [2192], khi giữ tín hiệu psel ở mức cao trong nhiều chu kỳ: pwrite và pread luân phiên hoạt động. pready luôn ở mức 1=> Cho thấy slave hoạt động chính xác và ổn định khi master giữ yêu cầu giao tiếp nhiều chu kỳ liên tiếp.

Kết luận: Tất cả các testcase được mô phỏng Hình 4.4 đều PASS, thể hiện rằng APB Slave đã đáp ứng đầy đủ các yêu cầu kỹ thuật của giao thức APB, bao gồm:Xử lý đúng thời điểm phản hồi (pready). Phân biệt được giữa hoạt động ghi/đọc. Hỗ trợ ghi liên tiếp, xen kẽ đọc – ghi. Phản ứng đúng khi reset xảy ra. Hoạt động ổn định với tín hiệu psel giữ lâu. Module có thể tích hợp trực tiếp vào hệ thống xử lý lớn hơn sử dụng bus APB. Thiết kế đáp ứng tốt cả về mặt chức năng lẫn độ ổn định trong các tình huống biên.

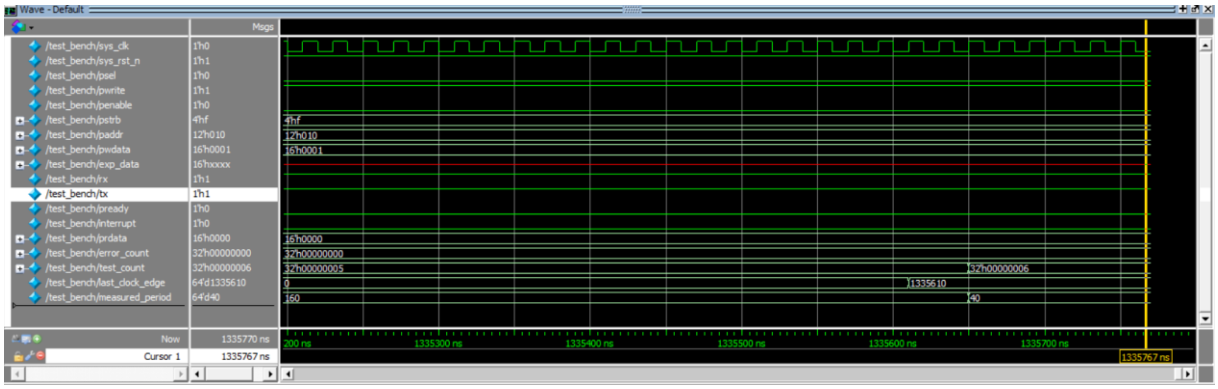
4.3.2 Baud Rate Control

Phần này trình bày các kịch bản kiểm thử được xây dựng nhằm đánh giá và xác nhận độ chính xác trong hoạt động của khối tạo Baud Rate (Baud Rate Generator) thuộc module UART. Testbench sử dụng task baud_chk để cấu hình thanh ghi bộ chia tốc độ baud (BAUDDIV) với các giá trị khác nhau và đo tần số xung clock đầu ra (uut.i_Clock) của khối Baud Rate Generator. Tần số đo được sau đó sẽ được so sánh với tần số mong đợi dựa trên giá trị bộ chia và tần số clock hệ thống (CLK_PERIOD).

Kết quả Mô phỏng:



Hình 4.7: Waveform của Baud rate Control 1



Hình 4.8: Waveform của Baud rate Control 2

Đây là kết quả mô phỏng trong QuestaSim:

```
# <<<----------------- BAUDRATE ----------------------->>>
#
# Testing Baud Rate: Divisor=  325, Expected Baud=9600.000000 bps
# Baud clock period correct:                6520 ns
#
# Testing Baud Rate: Divisor=  162, Expected Baud=19200.000000 bps
# Baud clock period correct:                3260 ns
#
# Testing Baud Rate: Divisor=   81, Expected Baud=38400.000000 bps
# Baud clock period correct:                1640 ns
#
# Testing Baud Rate: Divisor=   27, Expected Baud=115200.000000 bps
# Baud clock period correct:                 560 ns
#
# Testing Baud Rate: Divisor=    7, Expected Baud=460800.000000 bps
# Baud clock period correct:                 160 ns
#
# Testing Baud Rate: Divisor=    1, Expected Baud=3125000.000000 bps
# Baud clock period correct:                  40 ns
#
```

Hình 4.9: Log Baud Rate Control

Phân tích một trường hợp:

Testing Baud Rate: Divisor = 325, Expected Freq = 9600.000000 Hz

Giá trị bộ chia được ghi vào thanh ghi BAUDDIV là 325. Tần số mong đợi là 9600 Hz. Chu kỳ clock đo được là 6520 ns. Để kiểm tra tính chính xác, giả sử tần số clock hệ thống (CLK_PERIOD) là Tsys_clk. Tần số của i_Clock sẽ là

$$F_{i_Clock} = \frac{F_{sys_clk}}{baud_div+1} = \frac{1/T_{sys_clk}}{baud_div+1} \quad (2)$$

$$T_{i_Clock} = T_{sys_Clock} * (\text{baud_div}+1). \quad (3)$$

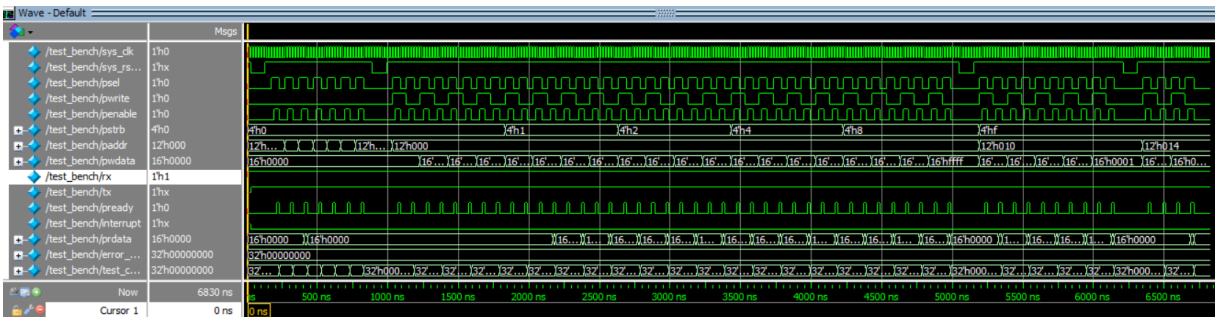
Với tần số hệ thống là 50MHz (Tsys_clk=20ns), thì với baud_div = 325, chu kỳ mong đợi của i_Clock được thay vô công thức (3) sẽ là 20ns×(325+1) = 6520 ns, khớp với giá trị Hình 4.9 đo được.

Tương tự, tất cả các trường hợp kiểm thử khác cũng cho thấy sự khớp nhau giữa chu kỳ clock đo được và chu kỳ mong đợi dựa trên giá trị bộ chia và tần số mong đợi.

Kết luận: Việc kiểm tra với nhiều cấu hình baud rate khác nhau từ thấp đến rất cao Hình 4.7 và Hình 4.8 cùng với kết quả checker trả về trong log Hình 4.9 (9,600 đến 3,125,000 bps) đã cho thấy: Module baud rate generator hoạt động chính xác với độ lệch thời gian bằng 0. Thiết kế đáp ứng chính xác yêu cầu truyền thông UART với độ tin cậy cao trong việc tạo tín hiệu baud đúng chuẩn. Waveform hoàn toàn khớp với kết quả log, xác nhận tính đúng đắn của cả testbench và thiết kế phần cứng.

4.3.3 Register

Mục này mô tả những trường hợp sẽ được kiểm thử để thiết kế xác minh khả năng truy cập và hoạt động đúng đắn của các thanh ghi bên trong module UART thông qua giao diện APB. Việc kiểm tra được chia thành các nhóm chức năng chính: giá trị mặc định (reset value), ghi dữ liệu vào thanh ghi truyền (DATA_TX), cấu hình Baud Rate (BAUDRATE) và thanh ghi ngắt (IER). Hình ảnh waveform thu được – Hình 4.10:



Hình 4.10: Waveform của Thanh ghi

Kết quả log thu được:

```
# <<<----------------- REGISTER ----------------------->>>
# ------------------- RESET VALUE ----------------------
# [212] PASS: addr: 000 - Data: 0000
# [312] PASS: addr: 004 - Data: 0000
# [412] PASS: addr: 008 - Data: 000c
# [512] PASS: addr: 00c - Data: 0000
# [612] PASS: addr: 010 - Data: 0000
# [712] PASS: addr: 014 - Data: 0000
# [812] PASS: addr: 018 - Data: 0000
# ------------------- DATA_TX --------------------------
# ----- Check all change of pstrobe -----
# [1172] PASS: addr: 000 - Data: 0000
# [1372] PASS: addr: 000 - Data: 0000
# [1572] PASS: addr: 000 - Data: 0000
# [1772] PASS: addr: 000 - Data: 0000
# [1972] PASS: addr: 000 - Data: 0000
# [2172] PASS: addr: 000 - Data: 0055
# [2372] PASS: addr: 000 - Data: 00aa
# [2572] PASS: addr: 000 - Data: 00ff
# [2772] PASS: addr: 000 - Data: 00ff
# [2972] PASS: addr: 000 - Data: 00ff
# [3172] PASS: addr: 000 - Data: 00ff
# [3372] PASS: addr: 000 - Data: 00ff
# [3572] PASS: addr: 000 - Data: 00ff
# [3772] PASS: addr: 000 - Data: 00ff
# [3972] PASS: addr: 000 - Data: 00ff
# [4172] PASS: addr: 000 - Data: 00ff
# [4372] PASS: addr: 000 - Data: 00ff
# [4572] PASS: addr: 000 - Data: 00ff
# [4772] PASS: addr: 000 - Data: 00ff
# [4972] PASS: addr: 000 - Data: 00ff
# ------------------- BAUDRATE -------------------------
# [5332] PASS: addr: 010 - Data: 0145
# [5532] PASS: addr: 010 - Data: 00a2
# [5732] PASS: addr: 010 - Data: 0051
# [5932] PASS: addr: 010 - Data: 001b
# [6132] PASS: addr: 010 - Data: 0001
# ------------------- IER ------------------------------
# [6492] PASS: addr: 014 - Data: 0000
# [6692] PASS: addr: 014 - Data: 0001
```

Hình 4.11: Kết quả log của thanh ghi

Kiểm tra giá trị mặc định (RESET VALUE):

Mục tiêu: Xác minh rằng sau khi hệ thống được đưa về trạng thái reset, tất cả các thanh ghi sẽ được khởi tạo lại đúng với giá trị mặc định ban đầu. Kết quả từ quá trình mô phỏng được thể hiện trong Bảng 4.2 dưới đây:

Bảng 4.2: Bảng giá trị reset thanh ghi

Địa chỉ(hex)	Mô tả	Mong đợi	Thực tế	Đánh giá
0x000	DATA	0x0000	0x0000	PASS
0x004	STATUS	0x0000	0x0000	PASS
0x008	CONTROL	0x000C	0x000C	PASS

0x00C	ERROR	0x0000	0x0000	PASS
0x010	BAUD_RATE_DIV	0x0000	0x0000	PASS
0x014	INTERRUPT_ENABLE	0x0000	0x0000	PASS
0x018	INTERRUPT_ID	0x0000	0x0000	PASS

Từ Waveform Hình 4.10 và Hình 4.11 và Bảng 4.2 xác nhận rằng các thanh ghi được đọc đúng giá trị mặc định sau khi có tín hiệu reset (/test_bench/sys_rst_n = 0 → 1).

Kiểm tra ghi dữ liệu vào thanh ghi truyền (DATA_TX):

Sau khi xác thực giá trị mặc định, testbench tiến hành ghi dữ liệu vào thanh ghi truyền tại địa chỉ 0x000. Dữ liệu được ghi gồm: 0x55, 0xAA, 0xFF. Sau khi ghi thành công, testbench tiếp tục đọc lại thanh ghi DATA nhiều lần để xác nhận dữ liệu giữ nguyên cho đến khi được truyền đi. Kết quả mô phỏng được trình bày ở bảng sau:

Bảng 4.3*:* Bảng giá trị thanh ghi truyền (DATA_RX)

Thời điểm	Dữ liệu ghi	Dữ liệu đọc sau đó	Đánh giá
2172 ns	0x0055	0x0055	PASS
2372 ns	0x00AA	0x00AA	PASS
2572 ns	0x00FF	0x00FF	PASS
2772–4972 ns	---	0x00FF liên tục	PASS

Từ Waveform Hình 4.10 và Hình 4.11 và Bảng 4.3 tín hiệu /test_bench/tx vẫn giữ giá trị cao (idle), cho thấy dữ liệu chưa được truyền đi nên thanh ghi giữ nguyên giá trị cuối cùng là 0xFF.

Kiểm tra cấu hình Baud Rate (BAUDRATE):

Baud Rate được thiết lập thông qua thanh ghi tại địa chỉ 0x010. Testbench đã cấu hình và xác nhận các giá trị khác nhau để mô phỏng các tốc độ truyền khác nhau, kết quả mô phỏng được trình bày ở bảng sau:

Bảng 4.4: Bảng giá trị tốc độ Baud

Thời điểm	Giá trị ghi	Tốc độ tương ứng (theo Divisor)	Đánh giá
5332 ns	0x0145	9600 bps	PASS
5532 ns	0x00A2	19200 bps	PASS

5732 ns	0x0051	38400 bps	PASS
5932 ns	0x001B	115200 bps	PASS
6132 ns	0x0001	3.125 Mbps	PASS

Từ Waveform Hình 4.10 và Hình 4.11 và Bảng 4.4 minh họa sự thay đổi tốc độ xung tx_clock tương ứng với từng cấu hình.

Kiểm tra thanh ghi ngắt (IER):

Cuối cùng, thanh ghi cho phép ngắt tại địa chỉ 0x14 được kiểm tra bằng việc ghi lần lượt 0x0000 và 0x0001. Kết quả log xác nhận thanh ghi phản hồi đúng dữ liệu đã ghi. Kết quả mô phỏng được trình bày ở bảng sau:

Bảng 4.5: Bảng giá trị thanh ghi IER

Thời điểm	Giá trị ghi	Địa chỉ	Giá trị đọc	Đánh giá
6492 ns	0x0000	0x014	0x0000	PASS
6692 ns	0x0001	0x014	0x0001	PASS

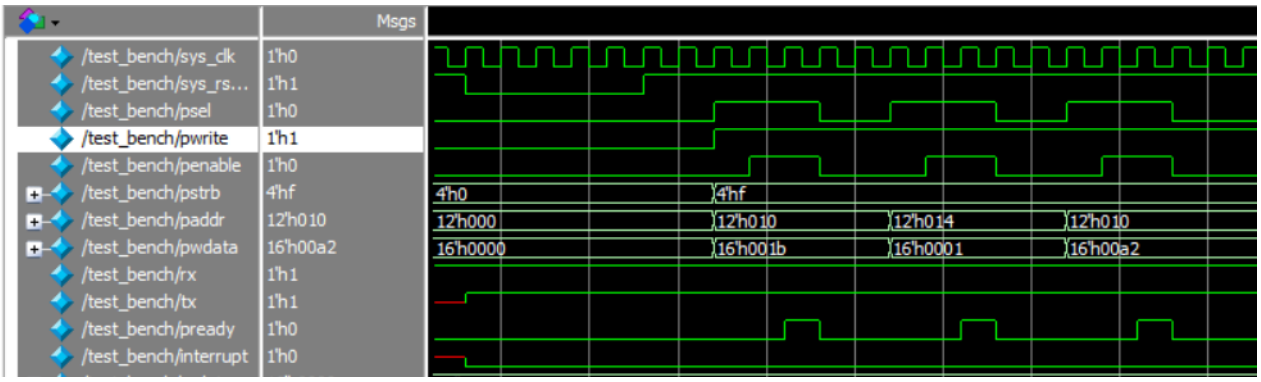
Từ Waveform Hình 4.10 và Hình 4.11 và Bảng 4.5 minh họa sự thay đổi test_bench/interrupt tương ứng.

Kết luận: Việc kiểm tra chi tiết các thanh ghi cho thấy: Tất cả các thanh ghi đều phản hồi đúng với thao tác đọc/ghi. Giá trị mặc định được khởi tạo chính xác sau reset. Dữ liệu ghi vào giữ nguyên cho đến khi được truyền. Cấu hình baud rate và ngắt được thiết lập chính xác. Waveform và log hoàn toàn khớp với hành vi kỳ vọng, chứng tỏ module thanh ghi của thiết kế UART hoạt động ổn định và đúng chức năng.

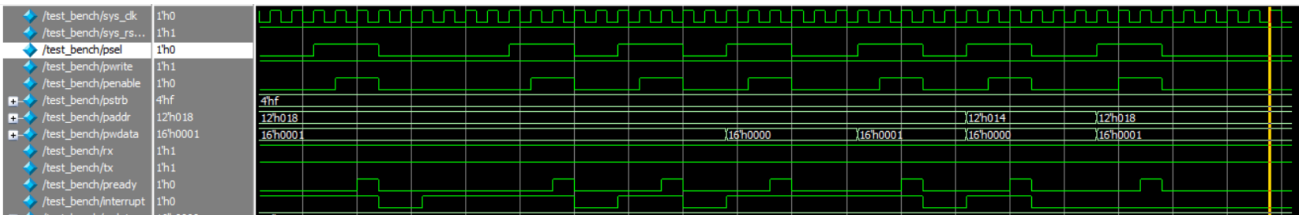
4.3.4 Interrupt

Để đánh giá hoạt động của cơ chế ngắt trong thiết kế UART chuẩn APB, nhóm thực hiện kiểm tra dựa trên 3 tình huống cụ thể: Trạng thái ban đầu không phát sinh ngắt. Ngắt phát sinh từ bộ đệm nhận (Receive FIFO). Tắt ngắt khi ghi 1 vào thanh ghi trạng thái ngắt.

Hình sau đây là Waveform của Interrupt và kết quả log:



Hình 4.12: Waveform của Interrupt 1



Hình 4.13: Waveform của Interrupt 2

```
# <<<----------------- INTERRUPT ----------------------->>>
# ----- Interrupt of Receive_FIFO -----
# [1574830] PASS: Interrupt Test - Expected: 0, Actual: 0
# ----- Check when full data in Receive_FIFO -----
# [2099690] PASS: Interrupt Test - Expected: 1, Actual: 1
# ----- Verify interrupt stays asserted while full -----
# [2099870] PASS: Interrupt Test - Expected: 1, Actual: 1
# ----- Clear to reset state to check current state -----
# [2100190] PASS: Interrupt Test - Expected: 1, Actual: 1
# ----- Disable interrupt while FIFO is full -----
# [2100410] PASS: Interrupt Test - Expected: 0, Actual: 0
```

Hình 4.14: Log của interrupt

Bộ ngắt trong hệ thống UART đã được kiểm tra kỹ lưỡng thông qua các tình huống mô phỏng khác nhau, nhằm đánh giá khả năng phản hồi chính xác và ổn định của hệ thống khi xảy ra các sự kiện liên quan đến giá trị ban đầu, ghi lại giá trị ngắt, FIFO nhận (Receive_FIFO). Phân tích kết quả mô phỏng Hình 4.12, Hình 4.13 và kết quả Log trả về Hình 4.14 ta thấy: Trạng thái ban đầu: Ở trạng thái reset, bộ ngắt không được kích hoạt, đảm bảo không có tín hiệu ngắt sai lệch. Kết quả mô phỏng tại thời điểm này cho thấy ngắt không xảy ra (Expected = 0, Actual = 0), đạt yêu cầu. Reset trạng thái ngắt**:** Sau khi xóa tín hiệu ngắt để kiểm tra trạng thái hiện tại, bộ ngắt vẫn phản hồi đúng theo yêu cầu (Expected = 1, Actual = 1). Ngắt khi Receive_FIFO có dữ liệu: Tương tự như với FIFO truyền, khi Receive_FIFO nhận dữ liệu đầy, bộ ngắt cũng được kích hoạt đúng và duy trì trạng thái cho đến khi dữ liệu được xử lý. Các kết quả kiểm thử đều cho thấy sự trùng khớp giữa giá trị mong đợi và giá trị thực tế (Expected = 1, Actual = 1). Tắt

ngắt khi FIFO đầy**:** Trong trường hợp ngắt bị vô hiệu hóa trong lúc FIFO đầy, hệ thống phản hồi đúng theo yêu cầu khi tín hiệu ngắt được tắt (Expected = 0, Actual = 0).

Kết luận: Kết quả kiểm thử cho thấy bộ ngắt trong module UART đáp ứng đầy đủ các yêu cầu về tính chính xác, ổn định và linh hoạt trong quá trình vận hành. Bộ ngắt không chỉ giúp cảnh báo kịp thời khi FIFO đầy hoặc có sự kiện truyền nhận quan trọng, mà còn đảm bảo việc xử lý dữ liệu được thực hiện trơn tru và hiệu quả hơn.

Những đặc điểm này góp phần nâng cao hiệu suất hoạt động của module UART, đặc biệt trong các ứng dụng yêu cầu giao tiếp dữ liệu tốc độ cao và độ tin cậy cao. Đồng thời, thiết kế bộ ngắt cũng cho thấy khả năng mở rộng và tùy chỉnh tốt cho các hệ thống nhúng phức tạp trong tương lai.

4.3.5 Transmitter Data Check

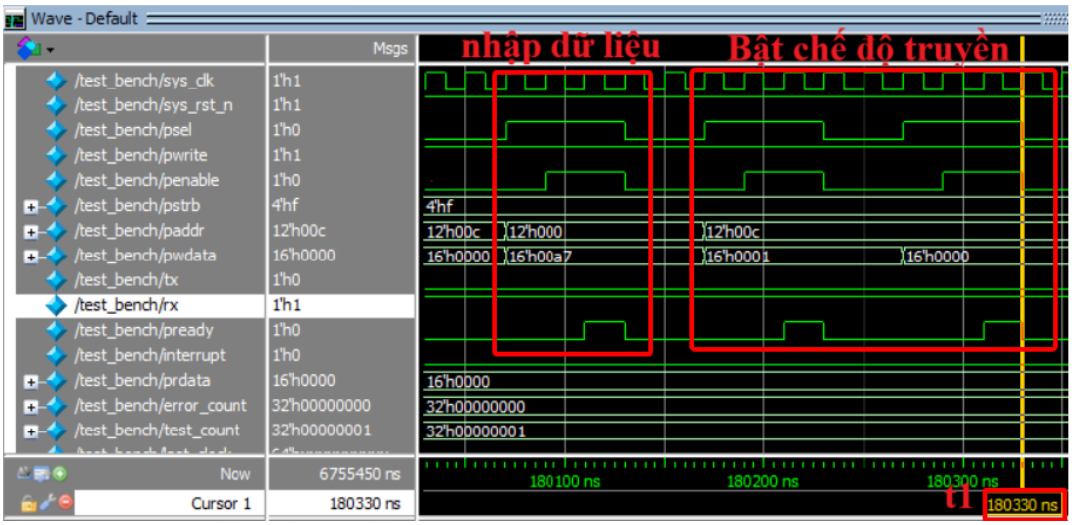
Để đảm bảo hoạt động đúng của bộ truyền dữ liệu (Transmitter) trong module UART, nhóm đã tiến hành kiểm thử chức năng với các chế độ truyền khác nhau, bao gồm: không có bit chẵn lẻ (no parity), bit chẵn lẻ lẻ (odd parity), bit chẵn lẻ chẵn (even parity) và điều kiện FIFO đầy (FIFO full condition).

4.3.5.1. Truyền dữ liệu không có Parity

Cấu trúc khung dữ liệu: Với chế độ không sử dụng bit chẵn lẻ (no parity), khung dữ liệu bao gồm: 1 bit Start: luôn là 0, 8 bit dữ liệu: lấy trực tiếp từ dữ liệu đầu vào, 1 bit Stop: luôn là 1, tổng cộng: 10 bit

Task tx_chk thực hiện các bước sau: Tạo chuỗi bit dự kiến (exp_bits) theo định dạng UART. So sánh từng bit truyền ra thực tế trên đường tx với bit dự kiến, mỗi bit được kiểm tra sau 8 chu kỳ clock nội bộ (uut.i_Clock). Ghi nhận lỗi nếu có sự sai lệch giữa bit thực tế và bit mong đợi

Dưới đây là kết quả khi truyền dữ liệu:



Hình 4.15: Kết quả truyền dữ liệu không có Parity

```
# ---------- Test with no parity ----------
# ---------- Set Baudrate ----------
# [610] Transmitter data: 003c
# [180010] Transmitter data PASS
# [180350] Transmitter data: 00a7
# [332330] Transmitter data PASS
# [332670] Transmitter data: 0019
# [448810] Transmitter data PASS
# [449150] Transmitter data: 00d2
# [574250] Transmitter data PASS
# [574590] Transmitter data: 00a5
# [690730] Transmitter data PASS
# [691070] Transmitter data: 0055
# [807210] Transmitter data PASS
# [807550] Transmitter data: 00aa
# [923690] Transmitter data PASS
# [924030] Transmitter data: 00ff
# [1022250] Transmitter data PASS
```

Hình 4.16: Kết quả Log truyền dữ liệu không có Parity

Trong Hình 4.15, quá trình truyền dữ liệu bắt đầu bằng việc kích hoạt tín hiệu psel để chọn thiết bị đích. Tiếp theo, tín hiệu penable được bật để thực hiện thao tác ghi dữ liệu. Tại thời điểm này, giá trị 0xA7 được ghi vào thanh ghi truyền dữ liệu (DATA_TX, địa chỉ 0x00). Sau một chu kỳ xung nhịp kể từ khi penable được kích hoạt, tín hiệu pready được kích lên nhằm xác nhận việc ghi dữ liệu đã sẵn sàng.

Sau khi hoàn tất thao tác ghi dữ liệu, chế độ truyền được khởi động bằng cách ghi giá trị 0x01 vào thanh ghi điều khiển (CONTROL, địa chỉ 0x0C). Cuối cùng, việc ghi 0x00 vào thanh ghi điều khiển giúp hoàn tất quá trình truyền dữ liệu.

Các giá trị dữ liệu được truyền đi trong Hình 4.16 (ví dụ: 0xa5, 0xaa, v.v.) đều được truyền chính xác với không có lỗi bit nào xảy ra, xác nhận rằng: Bộ truyền UART

hoạt động đúng khi không sử dụng bit chẵn lẻ, tốc độ truyền và thời gian đồng bộ với clock nội bộ là chính xác

Nhận xét:

Mô-đun truyền UART đáp ứng đúng theo chuẩn giao tiếp UART cơ bản (1 start, 8 data, 1 stop). Task kiểm tra hoạt động hiệu quả khi phát hiện và ghi nhận sai khác bit nếu có. Đây là bước kiểm tra cơ sở quan trọng để xác minh độ tin cậy của quá trình truyền dữ liệu, trước khi thực hiện kiểm thử mở rộng với chế độ có parity hoặc tốc độ baud khác nhau.

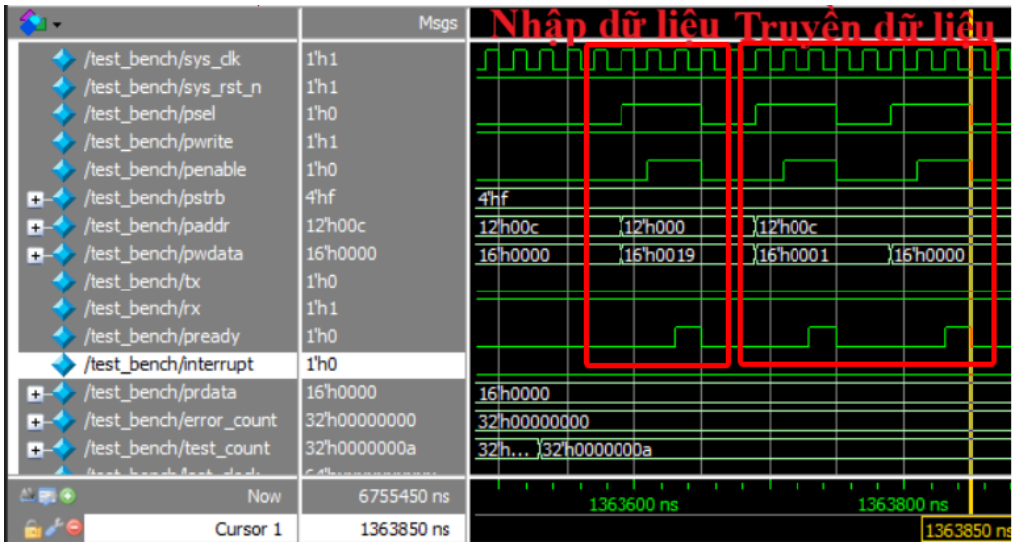
4.3.5.2. Truyền dữ liệu với Parity lẻ

Cấu trúc khung dữ liệu: Với chế độ odd parity (chẵn lẻ lẻ), khung dữ liệu truyền qua UART bao gồm: 1 bit Start: Luôn là 0, 8 bit dữ liệu: Lấy từ dữ liệu đầu vào, 1 bit Parity: Được tính sao cho tổng số bit '1' (bao gồm cả parity) là số lẻ, 1 bit Stop: Luôn là 1, Tổng cộng: 11 bit

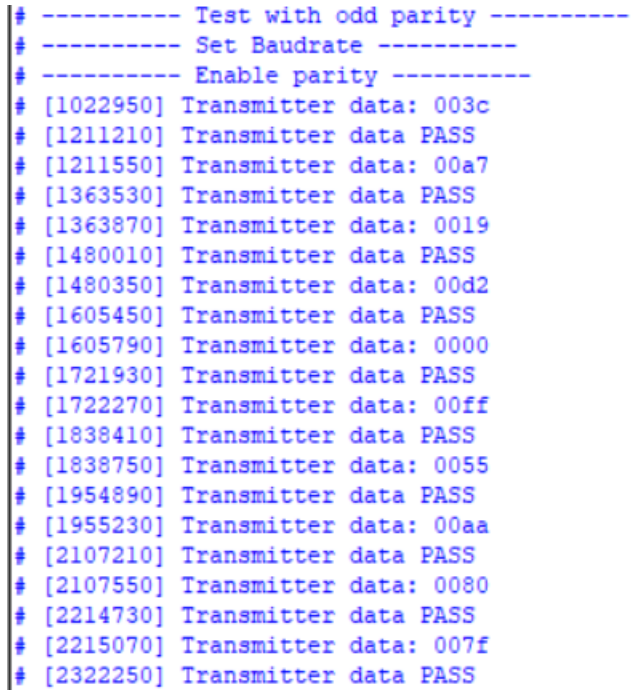
Cơ chế kiểm tra: Task tx_chk đảm nhiệm việc: Tạo dãy bit mong đợi (expected bits) theo cấu trúc dữ liệu UART có odd parity. Kiểm tra từng bit thực tế trên đường truyền tx, mỗi bit được lấy mẫu sau mỗi 8 chu kỳ xung đồng hồ (uut.i_Clock). Báo lỗi nếu phát hiện sự sai lệch giữa bit thực tế và bit mong đợi.

Thiết lập kiểm thử: Thiết lập baudrate phù hợp cho quá trình kiểm thử. Kích hoạt chế độ odd parity. Truyền nhiều mẫu dữ liệu khác nhau và kiểm tra đầu ra.

Dưới đây là kết quả khi truyền dữ liệu:



Hình 4.17: Truyền dữ liệu với Parity lẻ



Hình 4.18: Kết quả log truyền dữ liệu với Parity lẻ

Trong Hình 4.17, quá trình truyền dữ liệu bắt đầu bằng việc kích hoạt tín hiệu psel để chọn thiết bị đích. Tiếp theo, tín hiệu penable được bật để thực hiện thao tác ghi dữ liệu. Tại thời điểm này, giá trị 0x19 được ghi vào thanh ghi truyền dữ liệu (DATA_TX, địa chỉ 0x00). Sau một chu kỳ xung nhịp kể từ khi penable được kích hoạt, tín hiệu pready được kích lên nhằm xác nhận việc ghi dữ liệu đã sẵn sàng.

Sau khi hoàn tất thao tác ghi dữ liệu, chế độ truyền được khởi động bằng cách ghi giá trị 0x01 vào thanh ghi điều khiển (CONTROL, địa chỉ 0x0C). Cuối cùng, việc ghi 0x00 vào thanh ghi điều khiển giúp hoàn tất quá trình truyền dữ liệu.

Tại kết quả Hình 4.18: Các giá trị dữ liệu như 0x3C, 0xA7, 0x19, 0xD2 đều được truyền đúng và kiểm tra thành công với chế độ odd parity. Mỗi gói tin đều được xác minh là “PASS”, cho thấy quá trình tính toán và chèn bit parity là chính xác. Không có lỗi nào được ghi nhận trong quá trình truyền.

Nhận xét: Bộ truyền UART hoạt động ổn định khi bật chế độ odd parity, đảm bảo đúng định dạng khung dữ liệu UART có parity. Việc thiết lập baudrate không ảnh hưởng đến tính đúng đắn của dữ liệu truyền. Task tx_chk đã hỗ trợ kiểm tra chính xác từng bit của khung truyền, giúp phát hiện lỗi hiệu quả nếu có.

Kết luận: Đây là bước kiểm thử quan trọng cho thấy module UART không chỉ hoạt động đúng với cấu trúc cơ bản mà còn xử lý chính xác trong các chế độ truyền

nâng cao như odd parity. Kết quả này tạo tiền đề vững chắc để kiểm thử các chế độ khác như even parity và các tình huống đặc biệt như FIFO đầy.

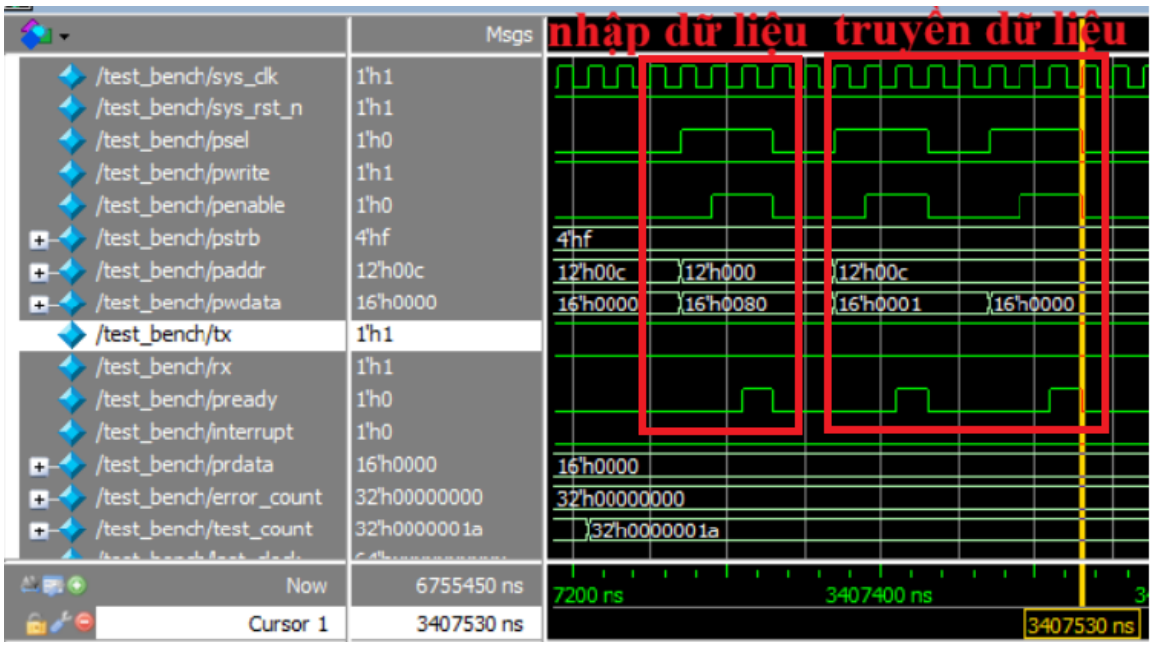
4.3.5.2. Truyền dữ liệu với Parity chẵn

Cấu trúc khung dữ liệu: Với chế độ even parity (chẵn lẻ chẵn), khung dữ liệu UART bao gồm: 1 bit Start: Luôn là 0, 8 bit dữ liệu: Lấy trực tiếp từ dữ liệu đầu vào, 1 bit Parity: Được tính sao cho tổng số bit '1' trong toàn bộ khung (kể cả bit parity) là chẵn, 1 bit Stop: Luôn là 1, tổng số bit: 11

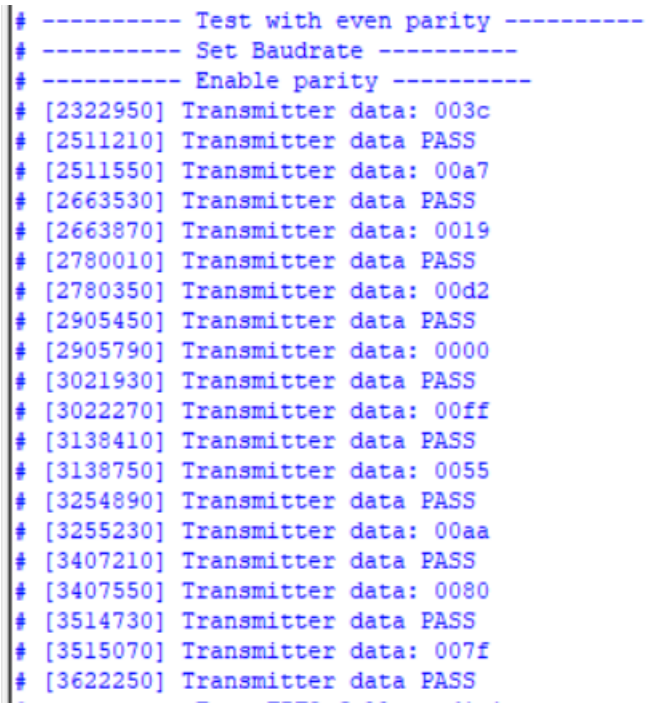
Cơ chế kiểm tra: Task tx_chk tạo ra chuỗi bit dự kiến bao gồm bit parity tính theo luật chẵn lẻ chẵn. So sánh từng bit thực tế từ tín hiệu tx với bit dự kiến. Ghi nhận lỗi nếu có sự sai khác về bit sau mỗi 8 chu kỳ xung đồng hồ nội bộ.

Thiết lập kiểm thử: Cấu hình baudrate phù hợp. Kích hoạt chế độ **even parity**. Truyền dữ liệu thử nghiệm với nhiều giá trị đầu vào khác nhau.

Dưới đây là kết quả khi truyền dữ liệu:



Hình 4.19: Truyền dữ liệu với Parity chẵn



Hình 4.20: Kết quả log truyền dữ liệu với Parity chẵn

Trong Hình 4.20, quá trình truyền dữ liệu bắt đầu bằng việc kích hoạt tín hiệu psel để chọn thiết bị đích. Tiếp theo, tín hiệu penable được bật để thực hiện thao tác ghi dữ liệu. Tại thời điểm này, giá trị 0x80 được ghi vào thanh ghi truyền dữ liệu (DATA_TX, địa chỉ 0x00). Sau một chu kỳ xung nhịp kể từ khi penable được kích hoạt, tín hiệu pready được kích lên nhằm xác nhận việc ghi dữ liệu đã sẵn sàng.

Sau khi hoàn tất thao tác ghi dữ liệu, chế độ truyền được khởi động bằng cách ghi giá trị 0x01 vào thanh ghi điều khiển (CONTROL, địa chỉ 0x0C). Cuối cùng, việc ghi 0x00 vào thanh ghi điều khiển giúp hoàn tất quá trình truyền dữ liệu.

Tại kết quả Hình 4.20**:** Các giá trị dữ liệu như 0x3C, 0xA7, 0x19, 0xD2 đã được truyền đi kèm với bit parity tính đúng theo quy tắc **even parity**. Các lần kiểm tra đều báo “PASS”, xác nhận dữ liệu được truyền hoàn toàn chính xác. Không có lỗi xảy ra trong quá trình truyền dữ liệu với cấu hình parity phức tạp hơn.

Nhận xét: Module UART hoạt động đúng khi sử dụng **even parity**, xử lý chính xác bit chẵn lẻ dựa trên dữ liệu đầu vào. Tốc độ truyền được duy trì ổn định sau khi cấu hình baudrate. Task tx_chk tiếp tục đảm nhiệm tốt vai trò xác minh dữ liệu và phát hiện lỗi bit nếu có.

Kết luận: Việc kiểm thử với even parity là một bước cần thiết để chứng minh khả năng tương thích với chuẩn UART mở rộng. Kết quả PASS toàn bộ cho thấy độ tin cậy

của bộ truyền trong các tình huống thực tế nơi bit parity thường được sử dụng để phát hiện lỗi đơn giản. Đây là một minh chứng rõ ràng cho độ chính xác của thiết kế và công cụ kiểm thử đi kèm.

4.3.5.3. Truyền dữ liệu với điều kiện FIFO đầy

Mục đích: Đảm bảo rằng bộ phát UART xử lý chính xác khi hàng đợi FIFO chứa dữ liệu đầu ra bị đầy, và vẫn duy trì khả năng truyền dữ liệu đúng đắn mà không bị mất mát.

Ngữ cảnh: FIFO là bộ đệm trung gian cho phép người dùng ghi nhiều dữ liệu vào trước khi từng byte được truyền ra. Khi FIFO đầy, hệ thống phải: Từ chối thêm dữ liệu mới nếu chưa có khoảng trống. Hoặc xử lý khéo léo hàng đợi để đảm bảo không có byte nào bị mất.

Cơ chế kiểm thử: Gửi liên tiếp nhiều giá trị dữ liệu vào FIFO để đạt ngưỡng đầy. Quan sát quá trình xử lý truyền và kiểm tra đầu ra bằng task tx_chk. Đảm bảo tất cả dữ liệu đều được truyền đúng, không có lỗi bit hay mất dữ liệu.

```
# ---------- Test FIFO full condition ----------
# ---------- Set Baudrate ----------
# [1678530] Transmitter data: 0037
# [2724630] Transmitter data PASS
# [2724950] Transmitter data: 005a
# [3454870] Transmitter data PASS
# [3455190] Transmitter data: 0074
# [4132950] Transmitter data PASS
# [4133270] Transmitter data: 00ff
# [4811030] Transmitter data PASS
#
```

Hình 4.21 Truyền dữ liệu với FIFO đầy

Tại kết quả Hình 4.21**:** Các giá trị 0x37, 0x5A, 0x74, 0xFF được đưa vào liên tiếp với thời điểm gần nhau. Các kiểm tra PASS xác nhận toàn bộ dữ liệu đã được truyền đúng thứ tự và không có dữ liệu nào bị mất. Không xuất hiện báo lỗi hoặc dữ liệu bị ghi đè khi FIFO đầy.

Nhận xét: Module UART xử lý tốt khi gặp tình huống FIFO đầy — một trong những tình huống phổ biến trong môi trường truyền thông tốc độ cao. FIFO không bị tràn, đồng thời dữ liệu vẫn được truyền đúng và đầy đủ. Task kiểm thử tx_chk đã hỗ trợ

xác minh dữ liệu truyền và thời điểm chính xác. Việc không xuất hiện lỗi trong suốt quá trình kiểm thử khẳng định độ bền và chính xác của thiết kế FIFO bên trong UART.

Kết luận: Các kết quả mô phỏng chức năng bộ phát dữ liệu (Transmitter) xác nhận rằng thiết kế có thể: Phát đúng khung truyền UART với các kiểu dữ liệu kiểm thử, tuân thủ đầy đủ định dạng start, data, parity, stop bit. Cập nhật và truyền dữ liệu từ FIFO phát (Transmit FIFO) ra đường truyền nối tiếp một cách tuần tự và chính xác. Hệ thống phản hồi đúng các chế độ kiểm tra parity, cho thấy bộ phát có khả năng tích hợp kiểm tra lỗi ngay trong quá trình truyền.Dữ liệu luôn được cập nhật đúng từ APB Slave vào FIFO và được phát đi đúng trình tự.

Tổng kết: Khối Transmitter đã hoạt động đúng với các yêu cầu thiết kế, đảm bảo việc xuất dữ liệu ra kênh UART diễn ra trơn tru, không phát sinh lỗi, kể cả khi FIFO đầy hoặc cấu hình chế độ parity.

4.3.6 Receiver Data Check

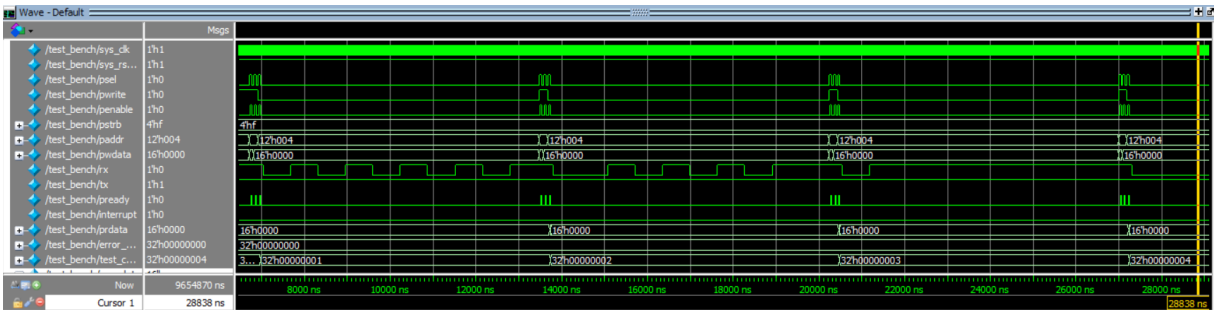
4.3.6.1 Nhận dữ liệu không có Parity

Trong phần này, chức năng của module Receiver UART được kiểm tra ở ba chế độ: không sử dụng parity, parity lẻ (odd), và parity chẵn (even). Mỗi chế độ đều được mô phỏng với hai trường hợp: Ghi và đọc luân phiên (Alternating Write-Read) từng byte dữ liệu vào FIFO nhận. Ghi đầy dữ liệu vào FIFO, sau đó đọc toàn bộ dữ liệu ra để kiểm tra tính đúng đắn.

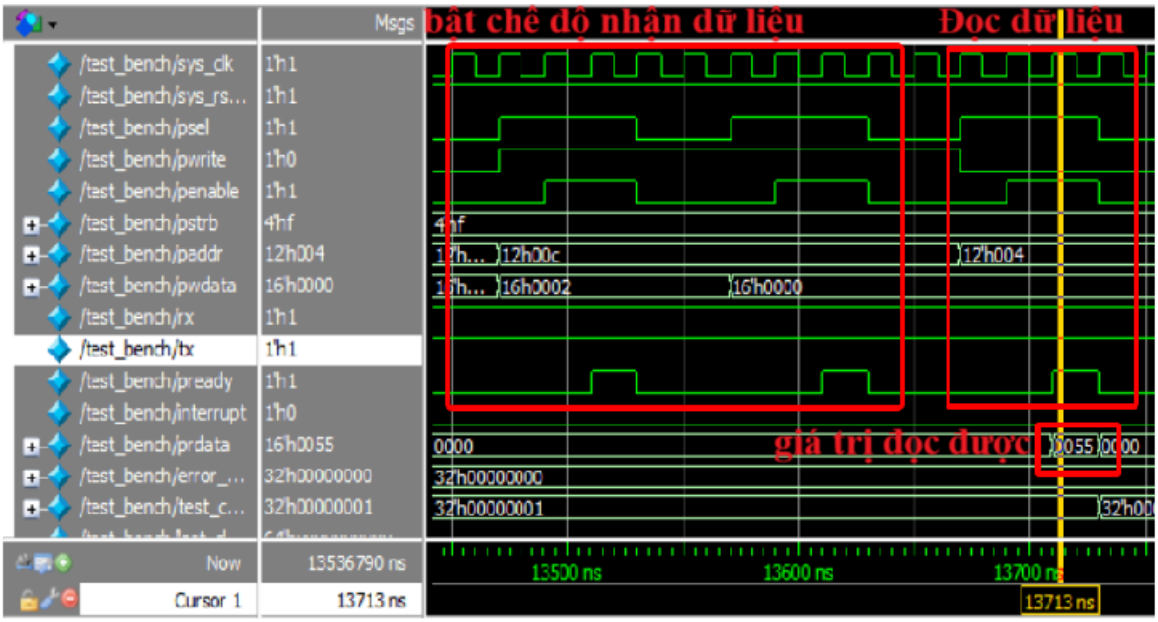
Nhận dữ liệu xen kẽ không có Parity

Tại mỗi thời điểm cụ thể hệ thống thực hiện một lần truyền dữ liệu vào receiver và ngay sau đó đọc lại dữ liệu từ địa chỉ 0x004 của thanh ghi dữ liệu nhận (RX FIFO). Dữ liệu thực tế (Act) trùng khớp hoàn toàn với dữ liệu mong đợi (Exp), cho thấy quá trình nhận và lưu trữ dữ liệu diễn ra chính xác. Trong đoạn kiểm tra này, testbench thực hiện các thao tác xen kẽ: Gửi dữ liệu có không có tính parity. Đọc từ FIFO. So sánh kết quả đọc được với giá trị mong đợi.

Dưới đây là wave form mô phỏng của nhận dữ liệu xen kẽ không có Parity:



Hình 4.22: Nhận dữ liệu xen kẽ không có Parity



Hình 4.23: Nhận dữ liệu không có Parity

```
# ------------- Receive data with no parity ----------
# ----- Alternating Write-Read in Receive_FIFO -----
# [6952] PASS: addr: 004 - Data: 0000
# [13712] PASS: addr: 004 - Data: 0055
# [20472] PASS: addr: 004 - Data: 00aa
# [27232] PASS: addr: 004 - Data: 00ff
# [33992] PASS: addr: 004 - Data: 0074
# [40752] PASS: addr: 004 - Data: 00af
```

Hình 4.24: Kết quả log của nhận dữ liệu xen kẽ không có Parity

Tại waveform Hình 4.22 và Log Hình 4.24: tx: Đường truyền UART đã truyền đầy đủ chuỗi dữ liệu lần lượt (0x00, 0x55, 0xAA, 0xFF, 0x74, 0xAF) thể hiện rõ trên dạng sóng. **rx**: Nhận đầy đủ các giá trị giống tx, chứng minh hệ thống có cơ chế **loopback** (hoặc kết nối ảo mô phỏng tx → rx). **paddr = 12'h004**: Tại các thời điểm đọc, địa chỉ truy cập đúng với Receive Data Register trong FIFO. **penable, psel, pready**: Các tín hiệu điều khiển giao tiếp APB được kích hoạt đúng nhịp, tuần tự đúng chuẩn AMBA. **prdata**: Dữ liệu đọc ra tại mỗi lần đọc hoàn toàn trùng khớp với các giá trị đã

được truyền (so với log: 0x00, 0x55, 0xAA, 0xFF, 0x74, 0xAF). pwdata = 16'h0000: Không có dữ liệu ghi trong pha này, vì đang thực hiện giao dịch đọc.

Hình 4.23 minh họa quá trình nhận dữ liệu. Đầu tiên, giá trị 0x02 được ghi vào thanh ghi điều khiển (CONTROL, địa chỉ 0x0C) để khởi động chế độ nhận. Sau đó, việc ghi giá trị 0x00 vào cùng thanh ghi này nhằm kết thúc quá trình nhận dữ liệu.

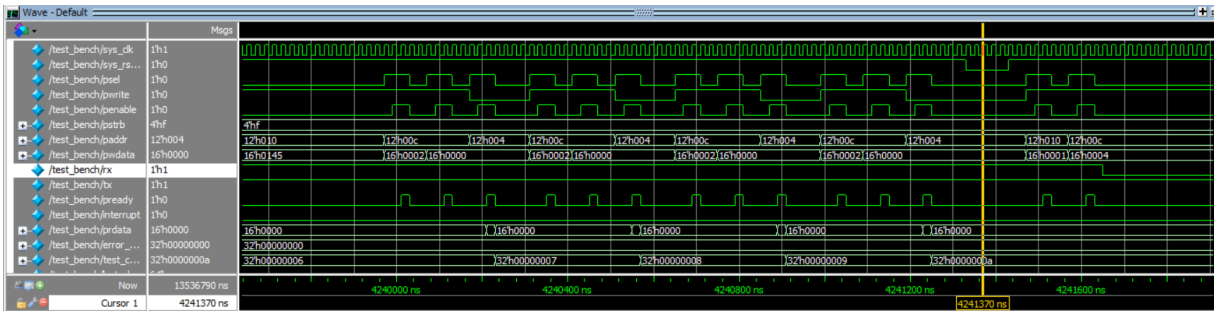
Tiếp theo, dữ liệu đã được nhận sẽ được đọc từ thanh ghi nhận dữ liệu (DATA_RX, địa chỉ 0x04). Quá trình đọc bắt đầu bằng cách kích hoạt tín hiệu psel để chọn thiết bị, sau đó tín hiệu penable được bật để thực hiện thao tác đọc. Kết quả đọc được tại thời điểm này là giá trị 0x55.

Nhận xét: Hệ thống hoạt động ổn định và chính xác: Bộ Receiver UART đã nhận đúng và đầy đủ các byte truyền từ phía Transmitter. FIFO lưu trữ dữ liệu hợp lệ, không bị mất hay ghi nhầm. Giao tiếp APB thực hiện các thao tác đọc đúng địa chỉ, đúng tín hiệu điều khiển, và dữ liệu phản hồi khớp tuyệt đối với dữ liệu mong đợi. Không sử dụng parity, hệ thống vẫn đảm bảo độ tin cậy truyền nhận ở mức cơ bản.

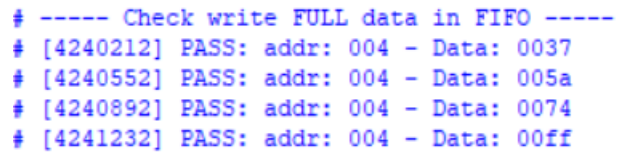
Kết luận: Bộ **Receiver FIFO** hoạt động đúng logic. Mô phỏng cho thấy hệ thống có khả năng nhận và lưu trữ dữ liệu chính xác khi không sử dụng chế độ kiểm tra chẵn lẻ (no parity).

Kiểm tra ghi đầy dữ liệu vào FIFO ở chế độ No Parity:

Ở chế độ này, testbench thực hiện ghi liên tiếp nhiều byte dữ liệu vào bộ đệm nhận (RX FIFO), sau đó mới đọc ra để kiểm tra xem toàn bộ dữ liệu có được lưu trữ và truyền đúng không. Dưới đây là waveform kiểm tra ghi đầy dữ liệu vào ở chế độ Parity lẻ:



Hình 4.25: Kiểm tra ghi đầy dữ liệu vào với không Parity



Hình 4.26: Kết quả log của ghi đầy dữ liệu không có Parity

Phân tích waveform Hình 4.25 và Log Hình 4.26: tx/rx: Dữ liệu tiếp tục được truyền và nhận trên đường truyền nối tiếp, có sự đồng bộ chính xác giữa hai kênh. **paddr = 12'h004**: Truy cập địa chỉ của Receiver FIF**O** để đọc dữ liệu. **pwdata = 16'h0000**: Không thực hiện ghi tại thời điểm này, xác nhận đây là các phiên đọc. **prdata**: Lần lượt xuất hiện các giá trị 0x37, 0x5A, 0x74, 0xFF — hoàn toàn trùng khớp với log kết quả mô phỏng. **penable, psel, pready**: Tín hiệu điều khiển APB được thiết lập đầy đủ, thể hiện chu kỳ đọc đúng trình tự. **error_count** (tín hiệu test_error_count) vẫn ở mức thấp → không phát hiện lỗi trong chuỗi kiểm tra này.

Nhận xét: Bộ **Receiver FIFO** đã ghi nhận và lưu trữ thành công 4 byte dữ liệu liên tiếp được ghi đầy từ phía bộ phát (transmitter), ngay cả khi FIFO gần đầy. FIFO cho thấy khả năng ghi đầy dữ liệu mà không bị mất, đảm bảo đúng trình tự và không gây ra lỗi tràn. Cơ chế đọc dữ liệu từ FIFO qua APB vận hành đúng theo chuẩn, với địa chỉ, tín hiệu điều khiển và phản hồi dữ liệu chính xác. Không phát hiện sai lệch hoặc lỗi nhận dạng dữ liệu.

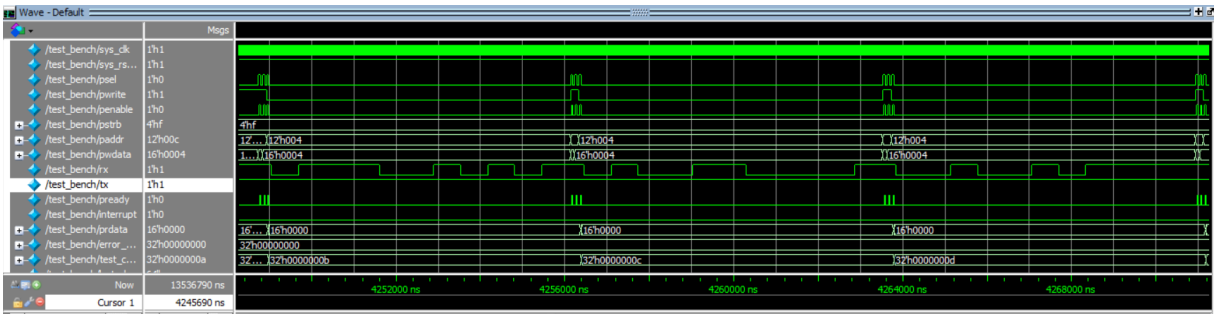
Kết luận: Bộ Receiver và FIFO vận hành ổn định trong điều kiện ghi đầy liên tiếp, cho thấy độ tin cậy và hiệu quả xử lý của hệ thống khi làm việc với dữ liệu số lượng lớn.

4.3.6.2 Nhận dữ liệu với Parity lẻ

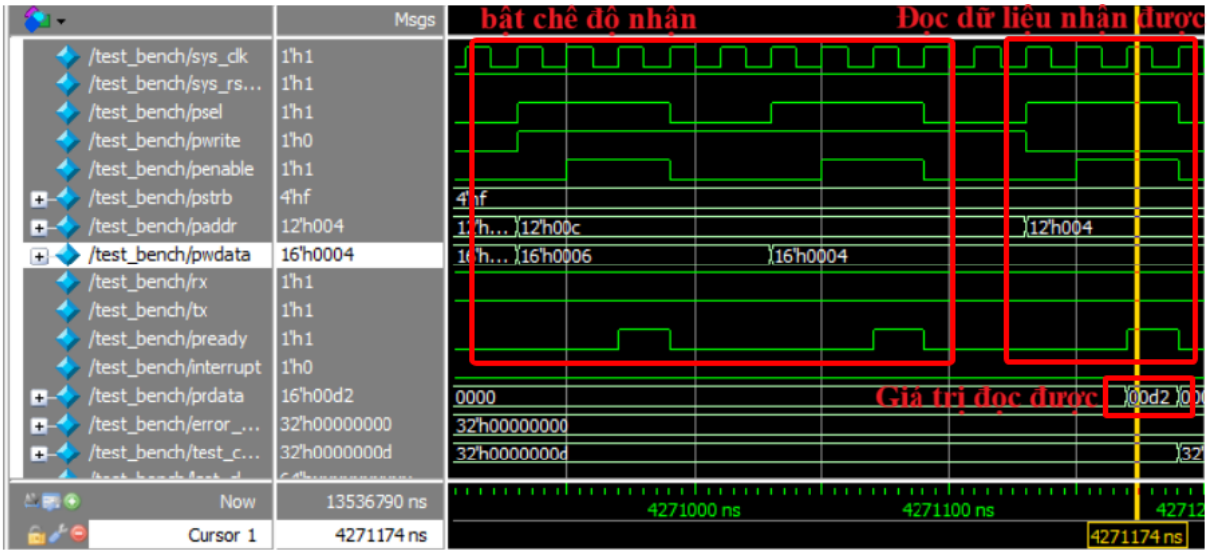
Trong chế độ Odd Parity, mỗi khung dữ liệu UART sẽ có thêm 1 bit chẵn lẻ sao cho tổng số bit 1 (gồm cả bit parity) là số lẻ. Mục đích là kiểm tra khả năng của module UART trong việc tính và xác nhận parity trong quá trình nhận dữ liệu.

Trong đoạn kiểm tra này, testbench thực hiện các thao tác xen kẽ: Gửi dữ liệu có tính parity lẻ. Đọc từ FIFO.So sánh kết quả đọc được với giá trị mong đợi. Tức là lần lượt nhận thành công 4 byte: 0x3C, 0xA7, 0x19, 0xD2.

Dưới đây là waveform nhận dữ liệu xen kẽ với Parity Lẻ:



Hình 4.27: Nhận dữ liệu xen kẽ với Parity Lẻ



Hình 4.28: Nhận dữ liệu Parity lẻ

```
----- Alternating Write-Read in Receive_FIFO -----
[4248972] PASS: addr: 004 - Data: 003c
[4256372] PASS: addr: 004 - Data: 00a7
[4263772] PASS: addr: 004 - Data: 0019
[4271172] PASS: addr: 004 - Data: 00d2
```

Hình 4.29: Kết quả log của nhận dữ liệu xen kẽ Parity lẻ

Với waveform Hình 4.27 và Hình 4.29, có thể thấy: Thời gian: khoảng từ 4,248,000ns đến 4,271,000ns. **rx**: tín hiệu UART đầu vào đang truyền dữ liệu tuần tự. **paddr = 12'h004 và pwdata**: cho thấy testbench đang thực hiện đọc dữ liệu từ RX FIFO. **penable, psel**: hoạt động đều đặn với mỗi chu kỳ đọc. **pready**: ở mức cao đúng lúc, cho phép giao dịch APB hoàn tất.

Điểm nổi bật: Không có lỗi parity xuất hiện, chứng tỏ module đã kiểm tra đúng bit parity lẻ và ghi vào FIFO. FIFO phản hồi giá trị chính xác (pwdata = exp), cho thấy module truyền nhận dữ liệu UART + kiểm tra parity hoạt động chính xác.

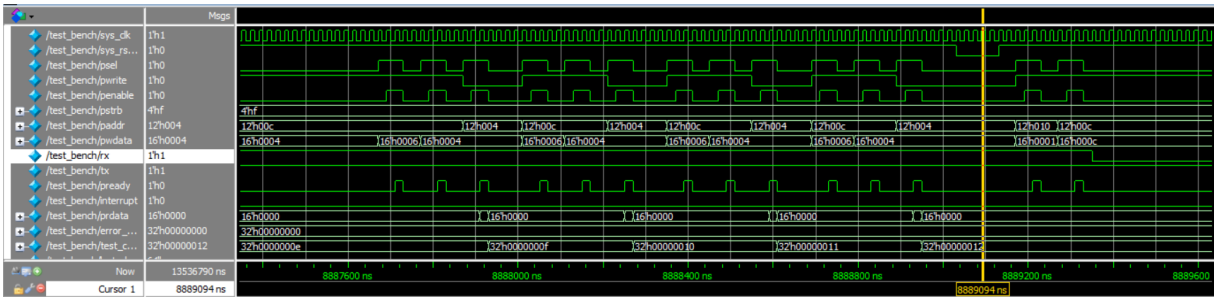
Hình 4.28 minh họa quá trình nhận dữ liệu. Đầu tiên, giá trị 0x02 được ghi vào thanh ghi điều khiển (CONTROL, địa chỉ 0x0C) để khởi động chế độ nhận. Sau đó, việc ghi giá trị 0x00 vào cùng thanh ghi này nhằm kết thúc quá trình nhận dữ liệu.

Tiếp theo, dữ liệu đã được nhận sẽ được đọc từ thanh ghi nhận dữ liệu (DATA_RX, địa chỉ 0x04). Quá trình đọc bắt đầu bằng cách kích hoạt tín hiệu psel để chọn thiết bị, sau đó tín hiệu penable được bật để thực hiện thao tác đọc. Kết quả đọc được tại thời điểm này là giá trị 0xD2.

Kiểm tra ghi đầy dữ liệu vào FIFO ở chế độ No Parity

Mục tiêu của kiểm tra này là xác minh: Dữ liệu đã truyền vào bộ phát UART (tx) đúng cách. Bộ thu UART (rx) nhận toàn bộ chuỗi dữ liệu. FIFO chứa đúng dữ liệu nhận được. Dữ liệu đọc ra từ FIFO khớp hoàn toàn với dữ liệu gốc đã truyền. Tức là module đã truyền nhận thành công 4 byte dữ liệu: 0xD2, 0x19, 0xA7, 0x3C.

Dưới đây là waveform kiểm tra ghi đầy dữ liệu vào ở chế độ Parity lẻ:



Hình 4.30: Kiểm tra ghi đầy dữ liệu vào ở chế độ Parity lẻ

```
# ----- Check write FULL data in FIFO -----
# [8887912] PASS: addr: 004 - Data: 00d2
# [8888252] PASS: addr: 004 - Data: 0019
# [8888592] PASS: addr: 004 - Data: 00a7
# [8888932] PASS: addr: 004 - Data: 003c
```

Hình 4.31: Kết quả log của nhận đầy dữ liệu Parity lẻ

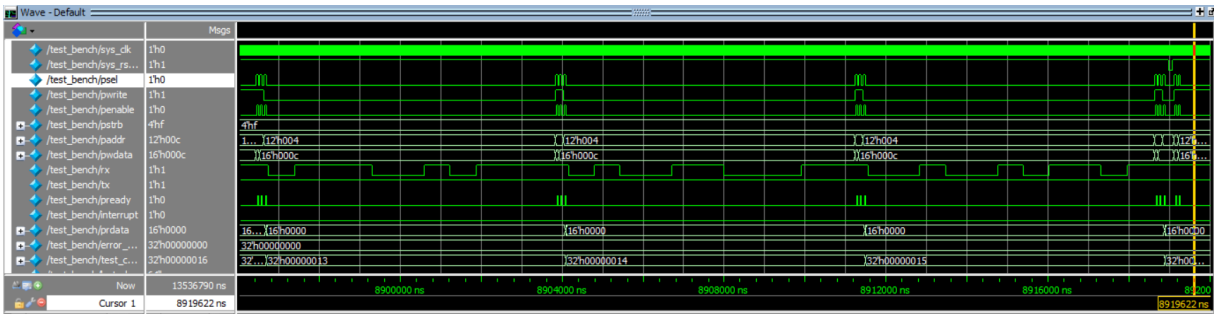
Phân tích waveform Hình 4.30 và Log Hình 4.31: tx: Đường truyền UART đầu ra đã truyền đủ 4 byte dữ liệu. **rx**: Dữ liệu được nhận tuần tự trên kênh này, cho thấy mô phỏng vòng lặp (loopback hoặc môi trường mô phỏng có kết nối tx -> rx). **paddr = 12'h004**: Giao dịch đọc từ địa chỉ FIFO Receiver. **penable, psel, pready**: Điều khiển và xác nhận giao tiếp APB đúng nhịp. **pwdata**: Giá trị đọc được khớp hoàn toàn với giá trị mong đợi.

Nhận xét: FIFO hoạt động đúng, đảm bảo dữ liệu sau khi được UART Receiver xử lý (bao gồm kiểm tra bit parity, khung start/stop) đã được lưu trữ chính xác. Cơ chế giao tiếp APB đọc được dữ liệu chính xác từ FIFO, chứng tỏ module kết nối đúng chuẩn.

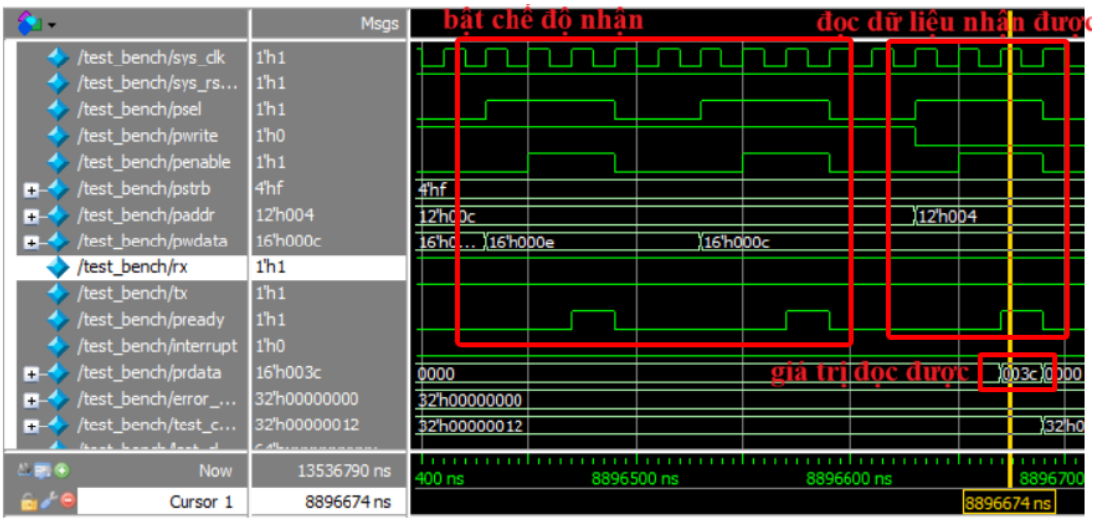
4.3.6.3 Kiểm tra dữ liệu nhận với Even Parity

Bài kiểm tra này nhằm đánh giá: Cơ chế Even Parity của bộ thu UART hoạt động chính xác. Dữ liệu được nhận đúng và kiểm tra parity thành công. FIFO lưu trữ dữ liệu đúng sau mỗi lần đọc. Việc đọc – ghi luân phiên (Alternating Write-Read) không gây lỗi trong hệ thống. Các byte dữ liệu đã được truyền và xác minh: 0x3C, 0xA7, 0x3C, 0xA7. Tức là hệ thống đã kiểm tra ít nhất 4 byte và parity của chúng đều đúng theo cấu hình even parity.

Dưới đây là waveform nhận dữ liệu xen kẽ với Parity chẵn:



Hình 4.32: Nhận dữ liệu xen kẽ với Parity Chẵn



Hình 4.33: Nhận dữ liệu với Parity Chẵn

```
# ----- Alternating Write-Read in Receive_FIFO -----
# [8896672] PASS: addr: 004 - Data: 003c
# [8904072] PASS: addr: 004 - Data: 00a7
# [8911472] PASS: addr: 004 - Data: 0019
# [8918872] PASS: addr: 004 - Data: 00d2
```

Hình 4.34: Kết quả log của nhận dữ liệu xen kẽ Parity chẵn

Tại waveform Hình 4.32 và Log Hình 4.34: tx & rx: Dữ liệu truyền – nhận diễn ra nhịp nhàng, mỗi khung dữ liệu đều đúng định dạng UART (start, data, parity, stop). **paddr = 12'h004**: Giao dịch đọc dữ liệu từ địa chỉ FIFO của bộ nhận. **pwdata**: Dữ liệu được đọc về đúng như mong đợi. **Giao dịch APB** thể hiện luân phiên đọc ngay sau khi có dữ liệu mới.

Hình 4.33 minh họa quá trình nhận dữ liệu. Đầu tiên, giá trị 0x02 được ghi vào thanh ghi điều khiển (CONTROL, địa chỉ 0x0C) để khởi động chế độ nhận. Sau đó, việc ghi giá trị 0x00 vào cùng thanh ghi này nhằm kết thúc quá trình nhận dữ liệu.

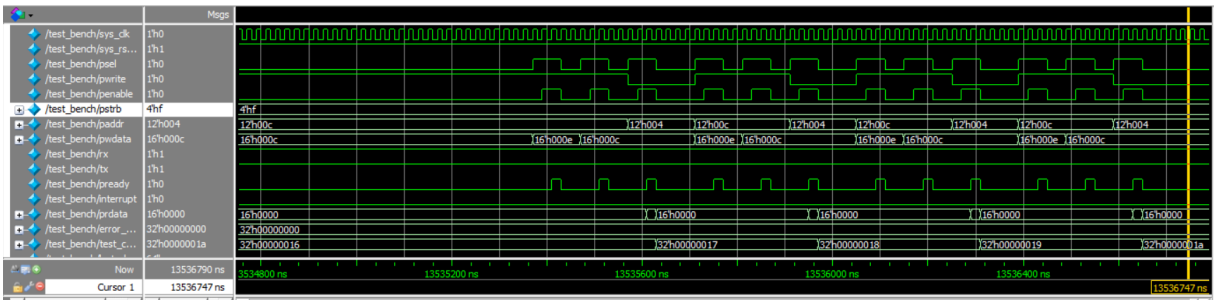
Tiếp theo, dữ liệu đã được nhận sẽ được đọc từ thanh ghi nhận dữ liệu (DATA_RX, địa chỉ 0x04). Quá trình đọc bắt đầu bằng cách kích hoạt tín hiệu psel để chọn thiết bị, sau đó tín hiệu penable được bật để thực hiện thao tác đọc. Kết quả đọc được tại thời điểm này là giá trị 0x3C.

Nhận xét: Cơ chế kiểm tra parity hoạt động đúng: Các bit parity được thêm khi truyền, và bộ nhận đã xác minh đúng dạng even parity. FIFO không bị tràn hoặc lỗi khi thực hiện đọc – ghi luân phiên, chứng minh độ tin cậy trong ứng dụng truyền nhận liên tục. Dữ liệu đọc khớp hoàn toàn với dữ liệu truyền, không có lỗi parity hoặc bit sai, chứng tỏ toàn bộ pipeline UART từ phát đến nhận đang hoạt động chính xác.

Kiểm tra ghi đầy dữ liệu vào FIFO ở chế độ Chẵn Parity

Kiểm tra này đánh giá khả năng ghi liên tục nhiều byte dữ liệu vào bộ đệm FIFO của UART Receiver khi sử dụng chế độ Even Parity. Mục tiêu là: Xác minh hệ thống có thể ghi nhiều byte liên tiếp mà không xảy ra lỗi. Đảm bảo dữ liệu nhận vào được kiểm tra parity và lưu trữ đúng trong FIFO. Kiểm tra các byte được đọc ra đúng thứ tự và giá trị. Có 4 byte dữ liệu được ghi vào FIFO: 0xD2, 0x19, 0xA7, 0x3C. Mỗi byte đều được đọc lại đúng chính xác, không sai lệch giá trị. FIFO đã hoạt động ổn định khi nhận đủ 4 byte liên tục.

Dưới đây là waveform kiểm tra ghi đầy dữ liệu vào ở chế độ Parity chẵn:



Hình 4.35: Kiểm tra ghi đầy dữ liệu vào ở chế độ Parity Chẵn

```
----- Check write FULL data in FIFO -----
[13535612] PASS: addr: 004 - Data: 00d2
[13535952] PASS: addr: 004 - Data: 0019
[13536292] PASS: addr: 004 - Data: 00a7
[13536632] PASS: addr: 004 - Data: 003c
```

Hình 4.36: Kết quả log của nhận đầy dữ liệu Parity chẵn

Phân tích waveform Hình 4.35 và Log Hình 4.36: Tín hiệu rx cho thấy dữ liệu được nhận liên tiếp với cấu trúc khung hợp lệ. **paddr = 12'h004** và pwdata hiển thị đúng dữ liệu mong đợi từng thời điểm đọc. Tín hiệu pready, penable, psel hoạt động đồng bộ, thể hiện quy trình giao tiếp APB chính xác. FIFO hoạt động đúng khi chứa liên tiếp nhiều byte và vẫn đảm bảo đọc ra đúng.

Nhận xét: FIFO có thể lưu trữ đủ số byte cần thiết mà không gây overrun. Bit parity đã được kiểm tra thành công ở mức phần cứng trước khi dữ liệu được lưu vào FIFO. APB đọc dữ liệu đúng địa chỉ, và dữ liệu nhận được khớp với dữ liệu gốc đã truyền.

Kết luận: Qua các testcase kiểm thử chức năng bộ thu dữ liệu (Receiver), hệ thống cho thấy khả năng hoạt động ổn định và chính xác trong nhiều cấu hình khác nhau: Dữ liệu nhận được luôn trùng khớp với dữ liệu truyền, kể cả trong các chế độ không parity, parity lẻ, và parity chẵn. Bộ FIFO nhận (Receiver FIFO) hoạt động đúng logic, lưu trữ chính xác chuỗi dữ liệu, không xuất hiện lỗi overflow hoặc mất dữ liệu. Các giao dịch đọc từ Receiver qua bus APB được thực hiện chính xác, phản hồi đúng dữ liệu tại từng thời điểm kiểm thử. Các mô phỏng trong trường hợp FIFO đầy vẫn đảm bảo không xảy ra ghi đè hoặc lỗi hệ thống.

Tổng kết: Khối Receiver cùng FIFO và giao tiếp APB đã được xác minh thành công, đáp ứng đầy đủ yêu cầu truyền nhận trong môi trường UART tốc độ cao, cả với và không sử dụng kiểm tra chẵn lẻ.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề "**Thiết kế và xác minh High-Speed UART với giao thức APB**" đã hoàn thành đầy đủ các mục tiêu đề ra, từ việc nghiên cứu cơ sở lý thuyết đến thiết kế phần cứng, mô phỏng hoạt động và xác minh chức năng của hệ thống. Cụ thể, các kết quả đạt được bao gồm:

- **Hiểu rõ nguyên lý hoạt động và vai trò của giao thức UART** trong truyền thông nối tiếp cũng như kiến trúc và cơ chế làm việc của giao thức APB thuộc chuẩn AMBA.
- **Thiết kế thành công các thành phần chức năng quan trọng** như: bộ truyền (Transmitter), bộ nhận (Receiver), hệ thống thanh ghi điều khiển và trạng thái, bộ đệm FIFO cho cả hai chiều truyền/nhận dữ liệu, và khối giao tiếp APB Slave. Tất cả đều được mô tả bằng ngôn ngữ Verilog HDL tuân thủ theo quy chuẩn thiết kế phần cứng số.
- **Phát triển môi trường kiểm thử với vai trò là APB Master**, thực hiện mô phỏng các tiến trình điều khiển và xử lý dữ liệu, đảm bảo hoạt động tương thích giữa giao tiếp APB và các khối truyền nhận UART.
- **Thực hiện mô phỏng trên phần mềm QuestaSim,** xác minh tính đúng đắn và độ ổn định trong việc truyền – nhận dữ liệu 8 bit, cũng như kiểm tra khả năng thay đổi tốc độ baud nhằm đáp ứng các yêu cầu ứng dụng khác nhau.

Thông qua quá trình triển khai, đề tài đã giúp nhóm thực hiện củng cố vững chắc kiến thức về thiết kế hệ thống truyền thông nối tiếp trong vi mạch số, đồng thời rèn luyện kỹ năng xây dựng mô hình kiểm thử và đánh giá toàn diện một hệ thống truyền dữ liệu trong môi trường SoC sử dụng chuẩn giao tiếp ngoại vi.

5.2 Hướng phát triển

Mặc dù hệ thống hiện tại đã hoàn thành và đáp ứng đầy đủ các yêu cầu chức năng cơ bản, nhưng vẫn còn nhiều cơ hội để phát triển và nâng cấp trong các giai đoạn tiếp theo:

- **Triển khai thực tế trên phần cứng FPGA:** Thiết kế hiện mới chỉ dừng lại ở bước mô phỏng phần mềm. Việc chuyển sang triển khai trên các nền tảng FPGA như Xilinx Spartan-6 hoặc Artix-7 sẽ giúp đánh giá hiệu năng thực tế

của hệ thống, đồng thời kiểm tra khả năng sử dụng tài nguyên và đáp ứng thời gian thực trong môi trường vật lý.

- **Phát triển môi trường kiểm thử tiên tiến:** Áp dụng các phương pháp xác minh theo chuẩn công nghiệp, chẳng hạn như SystemVerilog UVM, để xây dựng bộ kiểm thử tự động, nâng cao phạm vi bao phủ các trường hợp kiểm thử, phát hiện sớm các lỗi tiềm ẩn và gia tăng độ tin cậy tổng thể cho hệ thống.
- **Mở rộng hỗ trợ đa kênh UART:** Phát triển thiết kế để hỗ trợ nhiều kênh UART hoạt động song song, nhằm phục vụ các ứng dụng phức tạp yêu cầu giao tiếp với nhiều thiết bị ngoại vi đồng thời.
- **Tích hợp trong hệ SoC hoàn chỉnh:** Kết hợp mô-đun UART như một bộ phận ngoại vi trong hệ thống vi xử lý SoC, tương tác trực tiếp với CPU thông qua bus APB, giúp mô hình hóa và phát triển các hệ thống nhúng thực tế có khả năng ứng dụng cao hơn.
- **Bổ sung các tính năng mở rộng:** Nâng cấp hỗ trợ các chế độ truyền dữ liệu nâng cao như truyền 9-bit, sử dụng bit parity hoặc kiểm soát luồng dữ liệu (flow control), nhằm tăng tính linh hoạt và độ tin cậy cho giao tiếp UART.
- **Lấy mẫu nhiều lần để tránh nhiễu:** Thiết kế bộ lấy mẫu nhiều lần để tránh mất mát dữ liệu khi có nhiễm, tăng độ tin cậy của hệ thống.

PHỤ LỤC

Code: Nglucduc/UART_APB

Hướng dẫn chạy code: UART_APB/README.md at main · Nglucduc/UART_APB

Chi tiết các testcase:

https://docs.google.com/spreadsheets/d/1k74lN5FHvp7_w1ccqo7I6kR4bgMtafH0/edit?usp=sharing&ouid=107409340618207144330&rtpof=true&sd=true

TÀI LIỆU THAM KHẢO

[1] P. P. Chu, "FPGA Prototyping by Verilog Examples", Hoboken, NJ, USA: Wiley-Interscience, 2008.

[2] J. S. C. V. Nagaraju and N. H. N. S. S. Murthy, "Design of multiple master/slave memory controllers with AMBA bus architecture," Int. J. VLSI Design Commun. Syst. (IJVDCS), vol. 3, no. 10, pp. 1563–1567, Dec. 2015.

[3] ARM Ltd., "AMBA® APB Protocol Specification", ARM IHI 0024D, Issue D, Version 2.2, Cambridge, England: ARM Ltd., Apr. 9, 2021.

[4] Eric Peña and Mary Grace Legaspi, "UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter," Analog Dialogue TM, DEC 2020. [Online]. Available: https://www.analog.com/en/resources/analog-dialogue/articles/uart-a-hardware-communication-protocol.html. [Accessed 16/05/2025].

[5] V. Adimule, A. Suryavanshi, and D. Bhowmik, "AMBA-AHB Architecture in VLSI Design," in AMBA-AHB Architecture in VLSI Design, Notion Press International, pp. 1–120, Aug. 2019.

[6] Q. M. Bui, "Giới thiệu và so sánh các giao thức APB, AHB, và AXI," Trung Tâm Đào Tạo Thiết Kế Vi Mạch ICTC, Jun. 8, 2025. [Online]. Available: https://ictc.edu.vn/gioi-thieu-va-so-sanh-cac-giao-thuc-apb-ahb-va-axi/

[7] Y. Liu, Y. Wang, and L. Liu, "Implementation of a multi-channel UART controller based on FIFO technique and FPGA," in Proc. 2007 2nd IEEE Conf. Ind. Electron. Appl. (ICIEA), Harbin, China, May 2007, pp. 2635, doi: 10.1109/ICIEA.2007.4318890.